

Matplotlib for Storytellers

By: [Alexander Clark](#)

This version: September 25, 2026



This text is released under the [Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International](#) License.

The code is released under the MIT license.

The front cover image (on [Leanpub](#)) is from The First Book of Urizen, Plate 8, “In Living Creations Appear’d....” by William Blake (public domain).

Find more material on this book’s [GitHub](#) page, online Jupyter Book, and on YouTube.

github.com/alexanderthclark/Matplotlib-for-Storytellers

alexanderthclark.github.io/Matplotlib-for-Storytellers

youtube.com/@alexanderthclark

Contents

Preface	vii
Technical Notes and Prerequisites	vii
Why Matplotlib?	vii
Good Visualization is like Good Writing	viii
Resources and Inspiration	ix
Text Organization	x
I Prose	1
1 The Object-oriented Interface	3
1.1 Figure, Axes	4
1.2 Mixing the Interfaces	6
2 Axes Appearance, Ticks, and Grids	11
2.1 Axis Aspect and Limits	11
2.2 Axis Lines and Spines	14
2.3 Ticks	18
2.4 Grids	22
3 Plot Elements and Coordinate Systems	27
3.1 Primitives and Containers	27
3.1.1 Ordering with <code>zorder</code>	29
3.2 Coordinate Systems and Transformations	33
3.3 Use Window Extents	37
4 Text and Titles	41
4.1 Simple Titles	41
4.2 Text and Placement	43
4.2.1 Text Formatting for Numbers	47
4.3 Legends	49

4.4	Annotations	54
4.4.1	Labeling and Arrows	54
4.5	Fancy Titles	58
4.5.1	Multi-colored Titles	59
4.6	Fonts	62
4.6.1	Importing Fonts with Font Manager	63
5	Dates	65
5.1	Plotting	65
5.1.1	Time Zone Handling	66
5.2	Ticks and Formatting	67
5.2.1	Date Formats	67
6	Colors	71
6.1	Colormaps	71
6.2	Red, Green, Blue, Alpha	72
7	Multiple Axes and Plots	79
7.1	Multiple Axes	79
7.1.1	Using <code>twinx()</code> and <code>twinny()</code>	81
7.2	Multiple Plots	86
7.2.1	Using <code>subplots</code>	86
7.2.2	Using <code>add_subplot</code>	89
7.2.3	Figure Annotations and Legends	89
7.3	GridSpec	92
8	Style Configuration	95
8.1	<code>rcParams</code>	95
8.2	Defining Your Own Style	97
8.2.1	Temporary Configurations	99
8.3	A Final Prose Example	101
8.3.1	A First Go	102
8.3.2	Reconfigured, Refactored, and Reusable	105
II	Mathematical Interlude	109
9	Math	111
9.1	Circles	111
9.1.1	The Unit Circle	111
9.1.2	Non-unit Circles	113
9.1.3	Rotations and Ellipses	115
9.2	Right Triangles	117

10 Applications	121
10.1 Sloping Text	121
10.2 Circular Arrangements	123
10.3 Network Graphs	125
10.4 Tony Hawk's Vertical Loop	128
 III Poetry	 131
11 Poetry	133
12 Applications	135
12.1 Activity Calendar	135
12.2 Heatmaps	138
12.2.1 Google Trends	138
12.2.2 NHL Regular Season Records	143
12.3 Directed Graphs	147
12.4 Speedometer	150
 IV Special Topics	 153
13 Ternary Plots	155
13.1 Ternary	155
13.2 Application: Rock, Paper, Scissors	159
14 Intro Statistics	167
14.1 Probability Diagrams	168
14.2 Distributions	173
 15 Multi-dimensional Scaling	 181

All code and data files are available on the book's [GitHub repository](#). Note I exclude imports from the Python files in the main text. The imports below should cover the entire text. All of these should be included if you installed Anaconda, except for the ternary library, which I comment out below. When saving figures, I also sometimes run `plt.tight_layout()`, which is not always included in the Python files. See `Prose-Figure-Dev.ipynb`, `Math-Interlude-Figure-Dev.ipynb`, `Special-Figure-Dev.ipynb`, and `Poetry-Figure-Dev.ipynb` which contain the complete code for every figure.

```
1 import numpy as np
2 import pandas as pd
3 import math
4 from itertools import combinations
5 from itertools import product
6 from sklearn.datasets import load_iris
7
8 # matplotlib specific
9 import matplotlib as mpl
10 import matplotlib.pyplot as plt
11
12 # For Special Topics
13 # import ternary # requires install
14 # from ternary.helpers import simplex_iterator
15 from sklearn.manifold import MDS
16 from sklearn.decomposition import PCA
17 from scipy import stats
18
19 # Made redundant in the text
20 from matplotlib.patches import ConnectionPatch
21 from matplotlib.patches import Rectangle
22 import matplotlib.gridspec as gridspec
23 from matplotlib.ticker import MultipleLocator
24 from matplotlib.colors import colorConverter
25 from mpl_toolkits.mplot3d import Axes3D
26 from mpl_toolkits.mplot3d.art3d import Poly3DCollection
27 import matplotlib.dates as mdates
28 from matplotlib import font_manager
```

[imports.py](#)

Preface

Technical Notes and Prerequisites

The examples were originally developed with Python 3.9 and matplotlib 3.7.1. Because Python 3.9 has reached end of life, readers should use a currently supported Python 3 release and a current matplotlib release. I also make use of Jupyter notebook cell magics. I assume familiarity with basic Python programming, NumPy, pandas, and even matplotlib. In Part **I**, the premise is that you can make a plot, but now you want to polish it. Other parts assume less background knowledge. For those needing to review some Python before approaching this text, I recommend *A Whirlwind Tour of Python* and *Python Data Science Handbook*, both by Jake VanderPlas.

Why Matplotlib?

Though a bit aged, matplotlib is the standard in Python. matplotlib is integrated with pandas and Seaborn is based off matplotlib. You might prefer Plotnine if you already know R's ggplot2. You might prefer to leave Python and use D3 if you know javascript. You might prefer Microsoft Excel if you want consultants in your audience to feel at home.

I recommend matplotlib to anyone who is already committed to working in Python (and with the Python community) and values reproducibility and customizability. By the time we get to Part **III**, we'll be drawing more than plotting. This allows for more creativity than Excel allows and we'll maintain a reproducible Python-only workflow.

Good Visualization is Like Good Writing

This book isn't a guide to visualization design, but we must consider, at least briefly, what makes for good visualization and then why you might find matplotlib useful in that pursuit.

Data visualization is a form of communication not much different than writing. Cole Nussbaumer Knafl's *Storytelling with Data* parallels writing style guides like Sir Ernest Gowers' *The Complete Plain Words*. They both emphasize clarity and stripping out what is not essential. Matplotlib doesn't offer any unique advantage in pursuing clarity. Instead, the advantage is a tactical one. Matplotlib will expand your options. Sometimes straightforward prose is appropriate and sometimes only poetry will be stirring enough to capture your audience's attention. There exist prosaic visualizations and poetic visualizations with all the same tradeoffs.

Prose is precise and direct. Poetry has a certain beauty that invites interest and mediates higher truths. The familiar bar chart is prose, plainly reporting the numbers that need to be reported. Your boss will appreciate prose in a routine meeting. But imagine the king must wrestle with a difficult truth. Prose won't do. Only a jester or a Shakespearean fool can deliver the message and only by rhyme and riddle. So it may be with your C-level audience. The small truths of your bar charts don't matter to a busy CEO. Easier said than done, but capture your CEO's attention with a poetic visualization that might sacrifice some precision for its larger message.

A hurdle to crafting good visualizations is being limited to a short menu of cookie cutter graphics, whatever is available in Excel, a dashboard tool, or from a limited knowledge of matplotlib. Ahead of us is the chance to break free from those cookie cutter, ready-made visuals. In writing, George Orwell made good note of the "invasion of one's mind by ready-made phrases," in his worthwhile essay *Politics and the English Language*:

[Ready-made phrases] will construct your sentences for you—even think your thoughts for you, to a certain extent—and at need they will perform the important service of partially concealing your meaning even from yourself.

The important point here is that the unimaginative application of ready-made visualizations, just like phrases, can conceal your

meaning from yourself, not to mention your audience, and create a monotonous presentation of bar chart after bar chart.

The parallels between writing and making visuals go one level further. If you want to *become* a good writer, you will learn grammar, read good writers who came before you, write a lot, and skirt the rules a bit as you find your voice. In other words, you will do many things. Data visualization is no different. In what follows, you will begin to master just one thing, the technical grammar of matplotlib.

Resources and Inspiration

Before you dive in, you ought to get excited about data visualization. While there is a glaring lack of major museum space devoted to data visualization (I just recall a disappointing exhibit at the Cooper Hewitt), you will find many wonderful displays if you only keep your eyes peeled.

If you like to listen to people talk about data visualization, I recommend the [Data Stories](#) podcast.

If you'd like to start by reading one of the pioneers, check out [Edward Tufte](#), who continues to write new material. For more explicit or domain-specific guidance than Tufte might provide, see [Storytelling with Data](#) by Cole Nussbaumer Knaflc or [Better Data Visualization](#) by Jonathan Schwabish. Many of Schwabish's main themes are also communicated more briefly in Schwabish 2014. I have limited patience for how-to guides when they edge toward being overly prescriptive (I've never read any books on how to write well either), but I've profited from these titles. They are useful for their treatment of fundamentals like preattentive processing and surfacing more variety in visualizations, helping to inspire a richer repertoire. Knaflc's book is oriented toward business professionals and Schwabish adds his own public policy background. As a result, Knaflc concentrates on what I call prosaic visuals and Schwabish pushes further into the realm of poetry. Schwabish discusses the tradeoffs between standard and nonstandard graphs, noting that novelty can encourage more active processing, providing further justification for using a less accurate graph in select, exploratory cases.

Media outlets like the New York Times and Wall Street Journal usually make good use of data visualization. Take appropriate inspiration from these sources and from the [r/DataIsBeautiful](#) and

[r/DataIsUgly](#) subreddits.

The official matplotlib documentation continues to be a great resource, especially with new tutorials and galleries being added since I began work on this book. There is also a good Data Visualization section in *Coding for Economists* by Arthur Turrell. For a more advanced treatment of matplotlib, check out [Scientific Visualization: Python + Matplotlib](#).

Text Organization

Continuing the [parallel to writing](#), I have built this text around two main parts: [Prose](#) and [Poetry](#), though the distinction between prose and poetry is surely less exact than the division I've created. Prose, or Part [I](#), focuses on the fundamentals of customizing plots through the object-oriented interface. This section attempts to be reasonably thorough in breadth while providing only a minimal effective dose in depth. Then, after a mathematical interlude in Part [II](#), we reach poetry in Part [III](#). There can be no comprehensiveness to this section. I provide a guide to drawing in matplotlib, mostly with various [artist](#) objects. The mathematical interlude is there for those who would like to review some trigonometry I use. Then, I introduce two special (for fun) topics in Part [IV](#), multi-dimensional scaling and ternary plots.

Part I

Prose



Still Life with Apples and Grapes by Claude Monet (Public Domain)

Chapter 1

The Object-oriented Interface

Matplotlib offers two interfaces: a MATLAB-style interface and the more cumbersome object-oriented interface. If you count yourself among the matplotlib-averse, you likely never had the stomach for object-oriented headaches. Still, we are using the object oriented interface because we can do more with this.

The MATLAB-style interface looks like the following.

```
1 import matplotlib.pyplot as plt
2 x = 1,0
3 y = 0,1
4
5 plt.plot(x,y)
6 plt.title("My Plot")
```

[matlab-plot.py](#)

The object-oriented interface looks like this.

```
1 fig, ax = plt.figure(), plt.axes()
2 ax.plot(x,y)
3 ax.set_title("My Plot")
```

[oop-plot.py](#)

There is no such thing as a free lunch, so you will observe this interface requires more code to do the same exact thing. Its virtues will be more apparent later. Object-oriented programming (OOP) also requires some new vocabulary. OOP might be contrasted with procedural programming as another common method of programming. In procedural programming, the MATLAB-style interface

being an example, the data and code are separate and the programmer creates procedures that operate on the program's data. OOP instead focuses on the creation of *objects* which encapsulate both data and procedures.

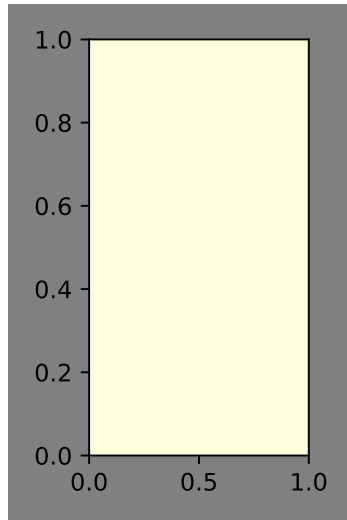
An object's data are called its *attributes* and the procedures or functions are called *methods*. In the previous code, we have figure and axes objects, making use of axes methods `plot()` and `set_title()`, both of which add data to the axes object in some sense, as we could extract the lines and title from `ax` with more code. Objects themselves are instances of a *class*. So `ax` is an object and an instance of the Axes class. Classes can also branch into subclasses, meaning a particular kind of object might also belong to a more general class. A deeper knowledge is beyond our scope, but this establishes enough vocabulary for us to continue building an applied knowledge of matplotlib. Because `ax` contains its data, you can think of `set_title()` as changing `ax` and this helps make sense of the `get_title()` method, which simply returns the title belonging to `ax`. Having some understanding that these objects contain both procedures and data will be helpful in starting to make sense of intimidating programs or inscrutable documentation you might come across.

1.1 Figure, Axes

A plot requires a figure object and an axes object, typically defined as `fig` and `ax`. The figure object is the top level container. In many cases like in the above, you'll define it at the beginning of your code and never need to reference it again, as plotting is usually done with axes methods. A commonly used figure parameter is `figsize`, to which you can pass a sequence to alter the size of the figure. Both the figure and axes objects have a `facecolor` parameter which might help to illustrate the difference between the axes and figure.

```
1 fig = plt.figure(figsize = (2,3),
2                   facecolor = 'gray')
3 ax = plt.axes(facecolor = 'lightyellow')
```

`figparams.py`



The axes object, named `ax` by convention, gets more use in most programs. In place of `plt.plot()`, you'll use `ax.plot()`. Similarly, `plt.hist()` is replaced with `ax.hist()` to create a histogram. If you have experience with the MATLAB interface, you might get reasonably far with the object-oriented style just replacing the `plt` prefix on your pyplot functions with `ax` to see if you have an equivalent axes method.

This wishful coding won't take you everywhere though. For example, `plt.xlim()` is replaced by `ax.set_xlim()` to set the *x*-axis view limits. To modify the title, `plt.title()` is replaced with `ax.set_title()` and there is `ax.get_title()` simply to get the title. The axes object also happens to have a `title` attribute, which is only used to access the title, similar to the `get_title()` method. Many matplotlib methods can be classified as *getters* or *setters* like for these title methods. The plot method and its logic is different. Later calls of `ax.plot()` don't overwrite earlier calls and there is not the same getter and setter form. There's a `plot()` method but no single `plot` attribute being mutated. Whatever has been plotted can be retrieved, or gotten (getter'd?), but it's more complicated and rarely necessary. Use the code below to see what happens with two calls of `plot()` and two calls of `set_title()`. The second print statement demonstrates that the second call of `set_title()` overwrites the title attribute, but a second plot does not nullify the first.

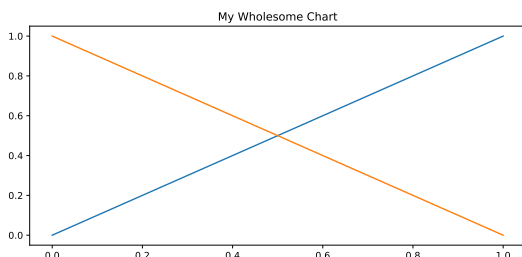
```
1 x = np.linspace(0,1,2)
```

```

2 fig, ax = plt.figure(figsize = (8,4)), plt.axes()
3 ax.plot(x, x)
4 ax.plot(x, 1 - x)
5 ax.set_title("My Chart")
6 print(ax.title)
7 print(ax.get_title()) # Similar to above line
8 ax.set_title("My Wholesome Chart")
9 print(ax.get_title()) # long

```

[gettersetter.py](#)



Axes methods `set_xlim()` and `get_xlim()` behave just like `set_title()` and `get_title()`, but note there is no attribute simply accessible with `ax.xlim`, so the existence of getters and setters is the more fundamental pattern.¹

1.2 Mixing the Interfaces

You can also mix the interfaces. Use `plt.gca()` to get the current axis. Use `plt.gcf()` to get the current figure.

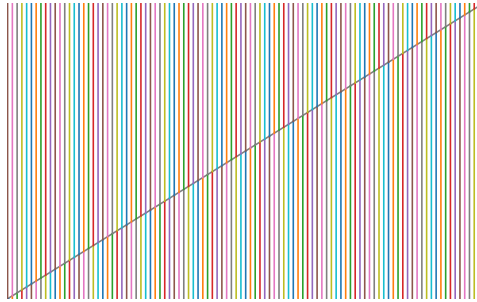
```

1 x = np.linspace(0,1,2)
2 plt.plot(x,x)
3 plt.title("My Chart")
4
5 ax = plt.gca()
6 print(ax.title)
7
8 ax.plot(x, 1 - x)
9 ax.set_title('My Wholesome Chart')
10 print(ax.title)

```

[chart.py](#)

¹Getters and setters are thought of as old-fashioned. It's more Pythonic to access attributes directly, but matplotlib doesn't yet support this.



In the above, we started with MATLAB and then converted to object-oriented. We can also go in the opposite direction, though it's not always ideal, especially when working with subplots. Below, we start with our figure and axes objects, and then revert back to the MATLAB style with the `axvline()` functions (producing vertical lines across the axes), toggling off the axis lines and labels, and then saving the figure. This graph would appear unchanged if you replaced `plt.axvline()` with `ax.axvline()`, `plt.axis()` with `ax.axis()`, and `fig.savefig()` would do the same as `plt.savefig()`.

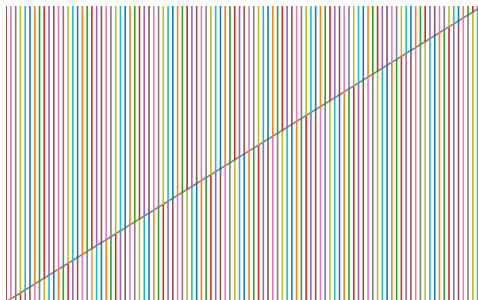
```

1 # OOP Start
2 fig, ax = plt.figure(figsize = (8,5)), plt.axes()
3
4 x = np.linspace(0,100,2)
5 ax.plot(x, x, color = 'gray')
6
7 ax.set_xlim([0,100])
8 ax.set_ylim([0,100])
9
10 # Back to pyplot functions
11 for i in range(101):
12     plt.axvline(i,0, i / 100, color = 'C' + str(i))
13     plt.axvline(i, i/100, 1, color = 'C' + str(i+5))
14
15 plt.axis('off')
16 plt.savefig('colorful.pdf')
```

colorful.py

Matplotlib is also integrated into pandas, with a `plot()` method for both Series and DataFrame objects, among other functionalities. There is excellent documentation [available](https://pandas.pydata.org/pandas-docs/stable/user_guide/visualization.html).² These plots can be mixed with the object-oriented interface. You can use a plot

²https://pandas.pydata.org/pandas-docs/stable/user_guide/visualization.html



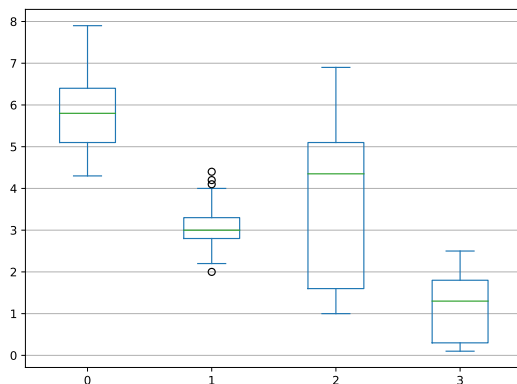
method and specify the appropriate axes object as an argument. Below we import the iris dataset and make a boxplot with a mix of axes methods and then pyplot functions.

```

1 from sklearn.datasets import load_iris
2 data = load_iris()['data']
3 df = pd.DataFrame(data)
4
5 fig, ax = plt.figure(), plt.axes()
6
7 df.plot.box(ax = ax)
8 ax.yaxis.grid(True)
9 ax.xaxis.grid(False)
10
11 plt.tight_layout()
12 plt.savefig('irisbox.pdf')

```

[irisbox.py](#)



The above capability is handy, especially with subplots, where every subplot will have its own axes object as we will see later.

Chapter 2

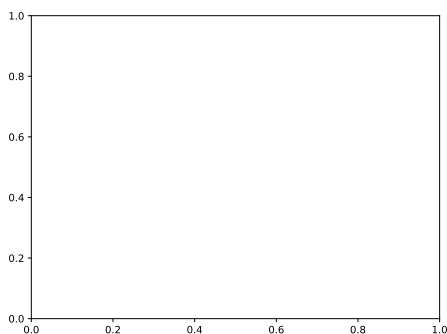
Axes Appearance, Ticks, and Grids

2.1 Axis Aspect and Limits

The most basic plot is the empty plot.

```
1 fig, ax = plt.figure(), plt.axes()
```

[empty.py](#)



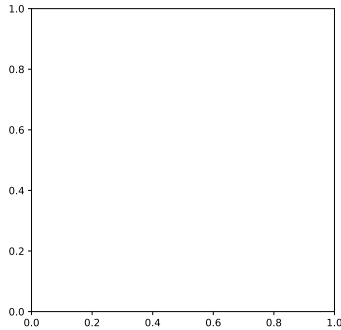
You'll notice this defaults to plotting the square region between data points (0,0) and (1,1). However, the plot is not square by default. That is to say the *aspect* is not one, where the aspect is the ratio of height to width. This can be changed with the axes method `set_aspect()`. For equal scaling, use `ax.set_aspect('equal')` or `ax.set_aspect(1)`.

```

1 fig, ax = plt.figure(), plt.axes()
2 ax.set_aspect('equal')

```

empty-square.py



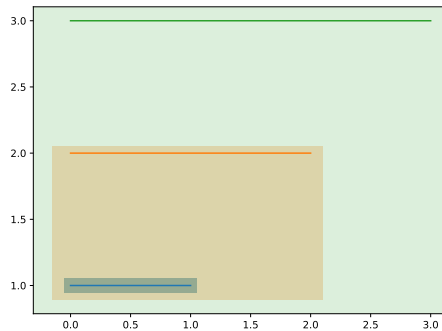
As we already covered in Chapter 1, the x and y limits can be adjusted with axes methods `set_xlim()` and `set_ylim()`, taking a sequence for the minimum and maximum values. If you don't explicitly set the limits, matplotlib will set the limits automatically based on the data. You can retrieve those limits with the getter methods, `get_xlim()` and `get_ylim()`. The program below makes use of both methods. We plot a few lines, and after each plot call, matplotlib is quietly updating the axes limits. Using the `fill_between()` method, which creates a color fill in the defined region, the expanding limits are shown. The colors are chosen automatically by matplotlib because I haven't explicitly specified a color value.

```

1 fig, ax = plt.figure(), plt.axes()
2
3 for i in range(1,4):
4     ax.plot([0,i], [i,i])
5     bottom_y, top_y = ax.get_ylim()
6     left_x, right_x = ax.get_xlim()
7     ax.fill_between(x = [left_x, right_x],
8                     y1 = bottom_y,
9                     y2 = top_y,
10                    alpha = 0.5/i)
11
12 # Prevent limits from automatically stretching further
13 # The last fill_between would stretch limits again
14 ax.set_ylim(bottom_y, top_y)
15 ax.set_xlim(left_x, right_x)

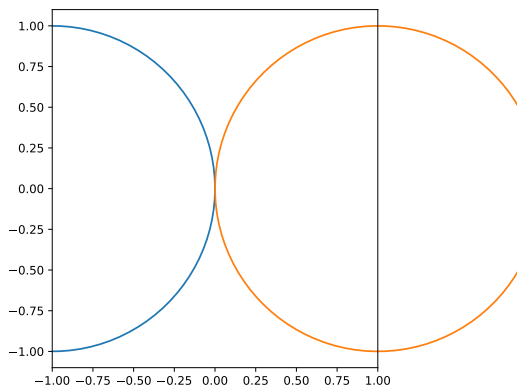
```

expanding-lims.py



If your axes limits are too restrictive, plot elements will be cut off. If you want your plot element to break past the end of the axes, spilling into the outer figure space, you can change this by setting `clip_on = False` in the appropriate method. Below, we create two circles with `ax.plot()` and set restrictive *x*-axis limits. The first circle, in blue, would extend further to the left if the limits were more generous. By default, it is clipped so we only see half of a circle. In the next call to `ax.plot()`, we create an orange circle and toggle `clip_on = False`. As a result, the circle extends to the right of the axes limits into the remaining figure space.

```
1 fig, ax = plt.figure(), plt.axes()
2 ax.set_aspect(1)
3
4 # Create a unit circle
5 u = np.linspace(0, 2*np.pi, 100)
6 x = np.cos(u)
7 y = np.sin(u)
8
9 # Default, clip_on = True
10 ax.plot(x-1, y)
11
12 # Unclipped, extends beyond the axes
13 ax.plot(x+1, y, clip_on = False)
14
15 ax.set_xlim(-1, 1)
```



2.2 Axis Lines and Spines

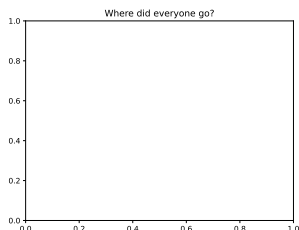
You might be used to plots that aren't surrounded by a box. Those enclosing lines, included by default, are called the *spines*. The default might also be jarring if you're used to the typical x - and y -axis lines at $y = 0$ and $x = 0$, like in most math textbook plots. In this section we'll cover how to modify these.

First, you might just eliminate everything with `ax.axis('off')`. We saw `plt.axis('off')` used similarly in Chapter 1 with a program that alternated between pyplot functions and the object-oriented approach. Below is a simple plot, empty but for a title, that becomes even emptier by eliminating the axis lines and labels. For reference, on the right is the same plot if `ax.axis('off')` were excluded from the program.

```
1 fig, ax = plt.figure(), plt.axes()
2 ax.set_title("Where did everyone go?")
3 ax.axis('off')
```

no-axis.py

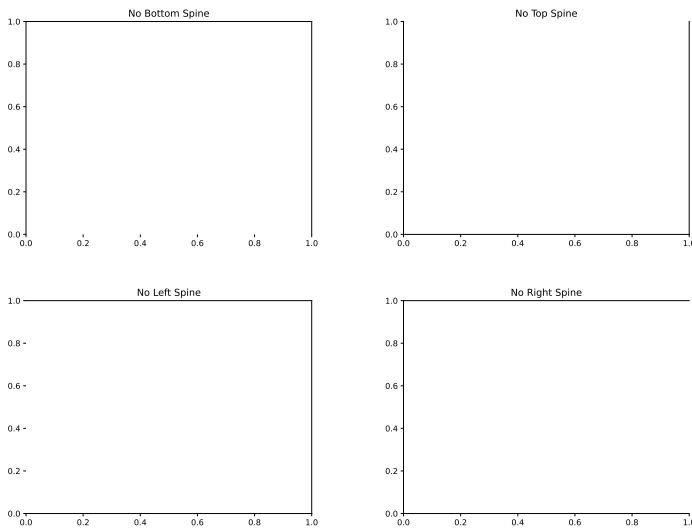
Where did everyone go?



Next, we can access and modify specific spines through `ax.spines`, which returns an `OrderedDict`. Access a specific spine using the appropriate key: `"left"`, `"right"`, `"top"`, or `"bottom"`. A spine can be toggled on or off by passing the appropriate boolean value to `set_visible()`.

```
1 for spine in 'bottom', 'top', 'left', 'right':
2     fig, ax = plt.figure(), plt.axes()
3     ax.set_title("No " + spine.title() + " Spine")
4     ax.spines[spine].set_visible(False)
5     plt.show()
```

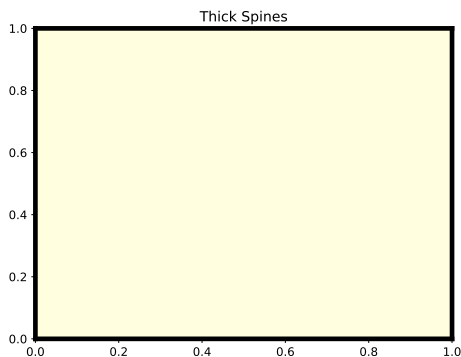
spine-vis.py



Other spine modifications might be their width and color. Again, we access a particular spine and then make use of setter methods, `set_color` and `set_linewidth` in particular.

```
1 fig, ax = plt.figure(), plt.axes(facecolor = '
   lightyellow')
2 ax.set_title("Thick Spines")
3 for spine in 'bottom', 'top', 'left', 'right':
4     ax.spines[spine].set_color('black')
5     ax.spines[spine].set_linewidth(4)
6 ax.set_xlim(0,1)
7 ax.set_ylim(0,1)
```

thick-spines.py



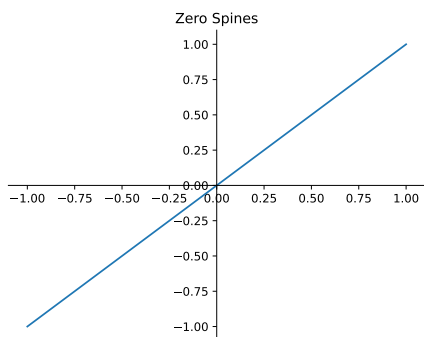
It's easy to get this far imagining that spines are simply the pieces of the box enclosing your plot. But they don't have to enclose the plot if we alter them with the `set_position` method. Below, we set the bottom spine to be along the usual x -axis and the left spine to be along the usual y -axis by passing `'zero'` to `set_position`. The right and top spines are removed.

```

1 fig, ax = plt.figure(), plt.axes()
2 ax.set_title("Zero Spines")
3 ax.plot([-1,1], [-1,1])
4 for spine in 'top', 'right':
5     ax.spines[spine].set_visible(False)
6 for spine in 'bottom', 'left':
7     ax.spines[spine].set_position('zero')

```

[zero-spines.py](#)



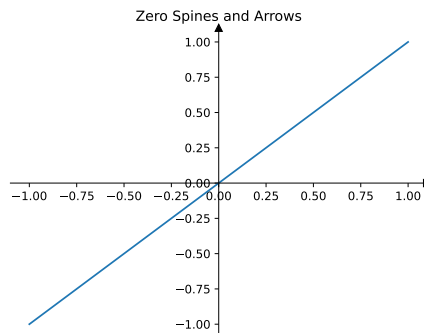
We can go a step further and add arrows at the ends of our axis lines with some clever plotting.

```

1 fig, ax = plt.figure(), plt.axes()
2 ax.set_title("Zero Spines and Arrows")
3 ax.plot([-1,1], [-1,1])
4 for spine in 'top', 'right':
5     ax.spines[spine].set_visible(False)
6 for spine in 'bottom', 'left':
7     ax.spines[spine].set_position('zero')
8
9 # get current limits
10 xlims = ax.get_xlim()
11 ylims = ax.get_ylim()
12
13 # Add arrows
14 ax.plot(xlims[1], 0, ">k", clip_on = False)
15 ax.plot(0, ylims[1], "^k", clip_on = False)
16
17 # revert limits to before the arrows
18 ax.set_xlim(xlims)
19 ax.set_ylim(ylims)

```

[arrow-axes.py](#)



The tick labels do clutter the graph above. This can be solved after we cover Section 2.3. Knaflitz [2015](#) recommends removing the top and right spines as part of the imperative to declutter and remove unnecessary chart border. I think it is arguable. I'm used to default spines enclosing the data. Removing them can seem untidy, like the plot guts might spill out onto the page, or as if the plot is now vulnerable to intruders without any fencing. Arrows on axis lines subtly prod the reader to imagine what happens outside

of the plotted region. I don't like that if, for example, I don't want to create the impression that a linear trend in a time series graph will continue into the future.

2.3 Ticks

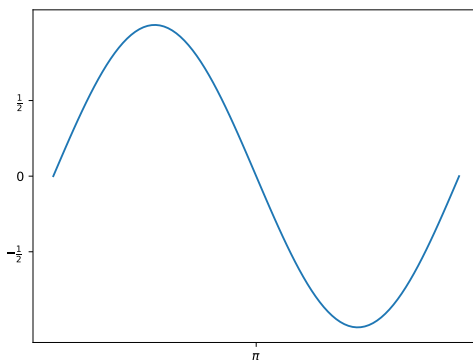
The important axes methods for ticks are `set_xticks`, `set_xticklabels`, and the natural y -axis counterparts. One may also use the general `set_ticks` and `set_ticklabels` with `ax.xaxis` or `ax.yaxis`—as axis (not axes) methods. These are demonstrated below, taking an array of tick locations and then the corresponding labels. I use `LATEX` strings to label the ticks. Here, that allows for a prettier y -axis, using fractions instead of decimals for tick labels. And on the x -axis, we can give a proper label of π at $x = \pi$.

```

1 x = np.linspace(0, np.pi * 2, 100)
2
3 fig, ax = plt.figure(), plt.axes()
4 ax.plot(x, np.sin(x))
5
6 # Y axis
7 ax.set_yticks( [-0.5, 0, 0.5] )
8 ax.set_yticklabels( [r"$-\frac{1}{2}$", 0, r"$\frac{1}{2}$"] )
9
10 # X axis
11 ax.xaxis.set_ticks([np.pi])
12 ax.xaxis.set_ticklabels([r"$\pi$"])

```

`ticks1.py`



To remove the ticks entirely, simply pass an empty array to `set_ticks()`. To customize the appearance of your axis ticks and the labels, use the `set_tick_params` axis method. Parameters include `direction`, `width`, `length`, `color`, `pad`, `rotation`, `labelsize`, `labelcolor`

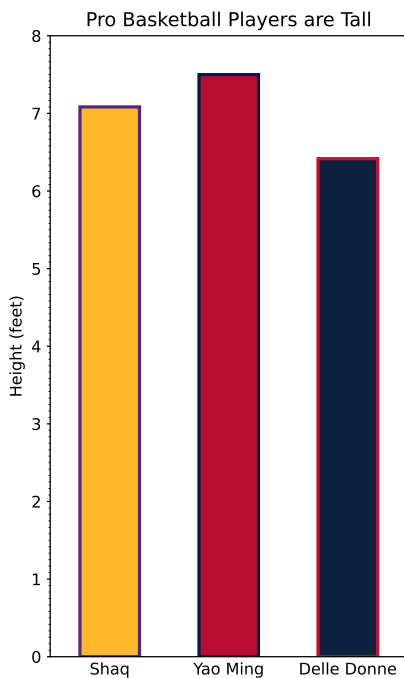
Imagine a measuring ruler, with ticks for every inch and smaller ticks at smaller intervals. So far our ticks have lacked that level of depth, but in fact we can work with two tick levels in matplotlib, major and minor ticks. Minor ticks are not shown by default.

To start exploring these further customizations, you'll need to import additional formatters and or locators. For the below, you must import `MultipleLocator`, running `from matplotlib.ticker import MultipleLocator`.

```

1 heights = pd.Series( {'Shaq': 7 + (1/12),
2                       'Yao Ming': 7.5,
3                       'Delle Donne': 6 + (5/12)})
4
5 fig, ax = plt.figure(figsize = (4,7)), plt.axes()
6
7 heights.plot.bar(ax = ax,
8                 color = ['#FDB927', '#BA0C2F', '#0C2340'],
9                 edgecolor = ['#552583', '#041E42', '#C8102E'],
10                 linewidth = 2)
11 # https://teamcolorcodes.com/
12 # LA Lakers and Houston Rockets and DC Mystics
13
14 # Get rid of ticks on x-axis, rotate text
15 ax.xaxis.set_tick_params(length = 0, which = 'major',
16                           rotation = 0)
17
18 ylim0, ylim1 = 0,8
19 ax.set_ylim([ylim0, ylim1])
20
21 ax.set_yticks(range(ylim0, ylim1+1))
22 #ax.yaxis.set_major_locator(MultipleLocator(1))
23
24 ax.yaxis.set_minor_locator(MultipleLocator(1/12))
25 ax.yaxis.set_tick_params(length = 1, which = 'minor')
26 ax.yaxis.set_tick_params(length = 2, which = 'major')
27
28 ax.set_ylabel("Height (feet)")
29 ax.set_title("Pro Basketball Players are Tall")

```



Major ticks can easily be set with `set_ticks` and its variants. Still, `MultipleLocator` and other locators are useful for setting major ticks without fooling with the details of the axes limits.

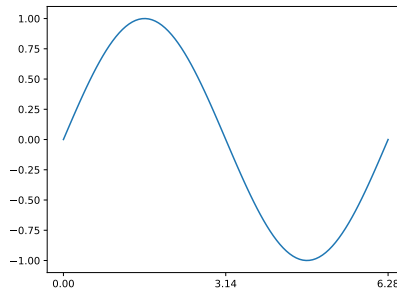
With a function like $\sin x$, ticks might most naturally be placed at multiples of π . This can be accomplished by the below.

```

1 x = np.linspace(0, np.pi * 2, 100)
2
3 fig, ax = plt.figure(), plt.axes()
4 ax.plot(x, np.sin(x))
5
6 ax.xaxis.set_major_locator(MultipleLocator(np.pi))

```

[mult-locator.py](#)

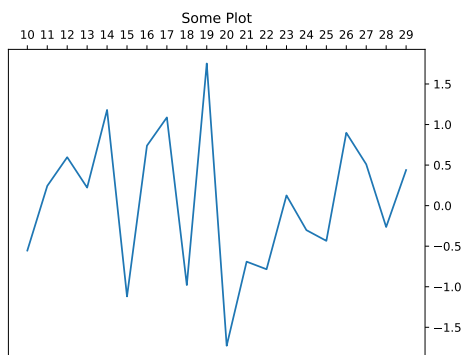


It's true you could avoid the complication of locator classes by just using `ax.set_xticks([0, np.pi, 2*np.pi])`. For a plot this simple, do that. Suppose, you put ticks up to 3π though. Then you've extended the x -axis limit of the plot past your data. So you need to know your data to make the right tick adjustments by hand. If you'll be using the same code with different datasets, it'll be easier to use the details-free `MultipleLocator` and you can still rely on limit defaults or adjust them independently.

Next, you might want to change the positioning of the ticks. By default x -axis ticks are on the bottom and y -axis ticks are on the left. You can modify these positions with axis methods. In time series data, for example, you might prefer to have the y -axis ticks on the right. Time marches on to the right and placing your ticks on the right can help emphasize that movement. This can be done with `set_ticks_position('right')` or the more concise `tick_right()`. The latter also accepts arguments of `'left'`, `'bottom'`, and `'top'`. Each has an abbreviated method like `tick_left()`.

```
1 fig, ax = plt.figure(), plt.axes()
2 x = np.arange(10, 30, 1)
3 y = np.random.normal(size = len(x))
4 ax.plot(x,y)
5
6 # set what ticks are shown
7 ax.xaxis.set_ticks(x)
8
9 # move the ticks
10 ax.yaxis.tick_right()
11 ax.xaxis.set_ticks_position('top')
12
13 ax.set_title("Some Plot")
```

`tick-right.py`

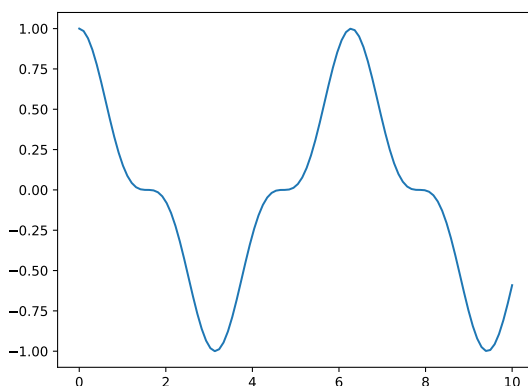


2.4 Grids

Including gridlines in a plot is generally discouraged (Knaflitz 2015, Schwabish 2021). It's clutter that won't spark joy. Perhaps we could stop here, with the instruction to run `ax.grid(False)` as in the code below (or rely on a style, like the default, that does this automatically).

```
1 fig, ax = plt.figure(), plt.axes()
2 x = np.linspace(0,10,100)
3 ax.plot(x, np.cos(x)**3)
4 ax.grid(False)
```

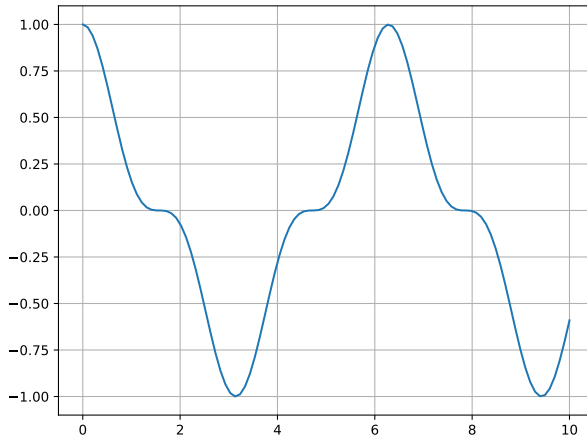
[grid-false.py](#)



This does seem preferable to the following, but it's hardly an abomination.

```
1 fig, ax = plt.figure(), plt.axes()
2 x = np.linspace(0,10,100)
3 ax.plot(x, np.cos(x)**3)
4 ax.grid(True)
```

[grid-true.py](#)



As a compromise, you might include gridlines for a single axis. If you want to emphasize that there is a slight trend in the data, then *y*-axis gridlines can help bring that pattern to the eye. Below we plot plots with and without a line of best fit and gridlines. Axis gridlines can be toggled independently by using `ax.xaxis.grid()` and `ax.yaxis.grid()`.

```
1 fig, ax = plt.figure(), plt.axes()
2
3 x = np.linspace(0, 10, 100)
4 y = 10 + .2*x
5 points = y + np.random.normal(size = len(x))
6 ax.scatter(x, points)
7
8 ax.set_ylim(0,30)
9 ax.set_xticks([])
```

[y-grid-false.py](#)

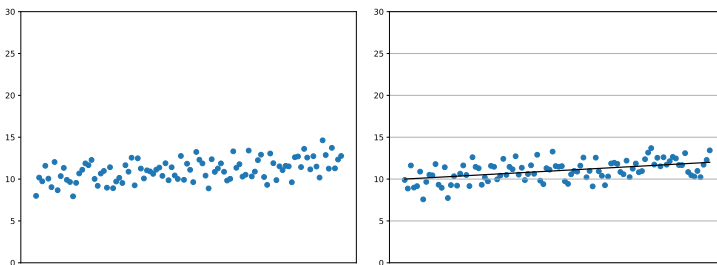
```
1 fig, ax = plt.figure(), plt.axes()
2
```

```

3 x = np.linspace(0,10, 100)
4 y = 10 + .2*x
5 points = y + np.random.normal(size = len(x))
6 ax.scatter(x,points)
7
8 ax.set_ylim(0,30)
9 ax.set_xticks([])
10
11 # Add grid and line of best fit
12 ax.yaxis.grid(True)
13 ax.plot(x, y, color = 'black')

```

y-grid-true.py



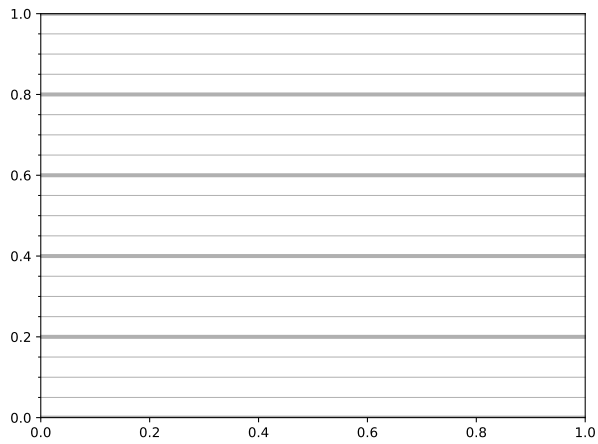
What we learned previously about locating ticks in Section 2.3 can be reapplied here, as seen in the examples further below. The location of gridlines and ticks can be set by the `set_major_locator()` and `set_minor_locator()` methods. `ax.grid()` is used to display the gridlines, but note it features a parameter `which`. The default value of `which` is `'major'`. To include minor gridlines, those minor ticks must be explicitly created (at least in the default style) and then the gridlines must be toggled on with `ax.grid(True, which = 'minor')` or for a single axis with `ax.xaxis.grid(True, which = 'minor')` for example.

```

1 fig, ax = plt.figure(), plt.axes()
2 ax.xaxis.grid(False)
3 ax.yaxis.grid(True, linewidth = 3)
4 ax.yaxis.grid(True, which = 'minor', linewidth = 0.5)
5 ax.yaxis.set_minor_locator(mpl.ticker.AutoMinorLocator()
6                             )

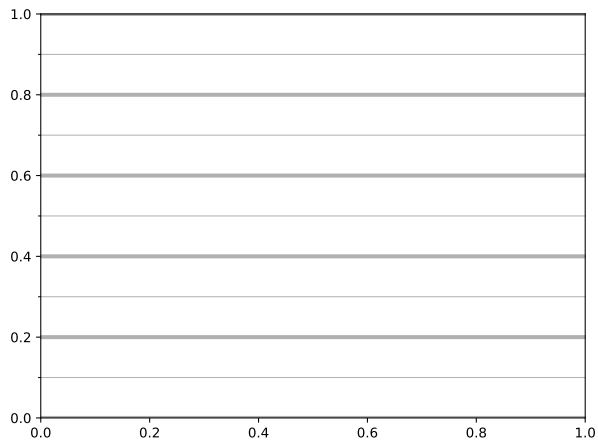
```

grids-auto.py



```
1 fig, ax = plt.figure(), plt.axes()
2 ax.xaxis.grid(False)
3 ax.yaxis.grid(True, linewidth = 3)
4 ax.yaxis.grid(True, which = 'minor', linewidth = 0.5)
5 ax.yaxis.set_minor_locator(mpl.ticker.MultipleLocator
    (.1))
```

[grids-multi.py](#)



Chapter 3

Plot Elements and Coordinate Systems

This chapter can be skipped by the reader in a hurry. I include it to establish some vocabulary about the basic plot elements and then discuss the different coordinate systems that can be used within a single plot—not polar vs. Cartesian coordinates but data coordinates vs. figure coordinates, for example. Coordinate systems do come up repeatedly in future chapters.

3.1 Primitives and Containers

Once you have a your figure and axis objects, you'll want to add actual plot elements to them, lines for a line chart, bars for a bar chart, annotations, etc. We already did that in Chapter 1, creating line plots. In matplotlib, these elements belong to the Artist class, it being a very general base class. Artists objects are basically the water you've been swimming in this whole time—you just might not have noticed it. Artist objects can be either primitives or containers. Containers include background items like the figure and axes objects. Primitives are the meat of the plot, like the line created by a call to `ax.plot()`. Important primitive Artist objects include `Line2D`, `Patches`, and `Text`.

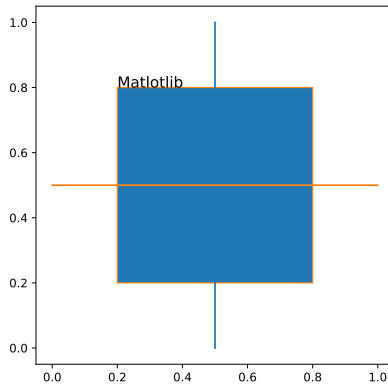
```
1 fig, ax = plt.figure(), plt.axes()
2 ax.set_aspect(1)
3
4 # Patches
5 rect = plt.Rectangle(xy = (0.2, 0.2),
```

```

6         width = 0.6,
7         height = .6,
8         facecolor = 'C0',
9         edgecolor = 'C1')
10 patch = ax.add_artist(rect)
11
12 # Lines
13 x, y = [0.5, 0.5], [0, 1]
14 line, = ax.plot(x, y)
15 lines = ax.plot(y,x)
16
17 # Text
18 text = ax.text(0.2, 0.8, 'Matplotlib', size = 13)

```

artists.py



What might be unusual in the above is that we don't simply run `ax.plot(x, y)`. Instead we actually assign the plot call to a variable, `line, = ax.plot(x,y)`. Usually, this isn't necessary, but this allows us to reference the same object later in the program. The plot method creates a tuple of `Line2D` objects. In this case, that tuple contains only one item and it is assigned to the variable `line`.

Now that we have the object as `line`, we can get properties or make changes. You can obtain the color with the `get_color()` method or change it with `set_color()`. You can even remove the plot element with `line.remove()`. These are all niche uses. However, we will later make use of `remove()` when iteratively centering text. We'll also use the `get_window_extent()` artist method frequently to help space objects in the plot.

3.1.1 Ordering with `zorder`

Default Ordering

By default, text is plotted over lines and lines are plotted over patches, like the fill created by `fill_between()`. Within each of these three categories, objects created later in the program are plotted over previously created objects. The `zorder` parameter can be used to create a different ordering. Objects with a greater `zorder` value are ordered further to the front.

First, we create and plot without specifying the `zorder` for any object to observe default behavior. We also print the `zorder` for each object using `get_zorder()`. Text has a `zorder` of 3, lines have a `zorder` of 2, and each patch object will have `zorder` = 1. Note `patch1` and `patch2` have the same `zorder`, but the red `patch2` is added later in the program so it is plotted over the green `patch1`, being as if `patch1` has a lower `zorder`.

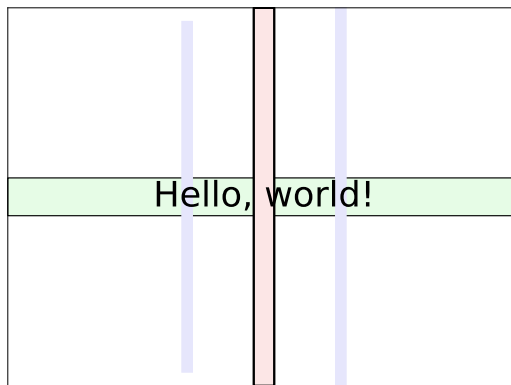
```
1 fig, ax = plt.figure(), plt.axes()
2 ax.set_xlim(0,1)
3 ax.set_ylim(0,1)
4 ax.set_xticks([])
5 ax.set_yticks([])
6
7 # make colors
8 green = (.9, .99, .9)
9 blue = (.9, .9, .99)
10 red = (.99, .9, .9)
11
12 # Text with default zorder of 3
13 text = ax.text(0.5, 0.5, "Hello, world!",
14               size = 30,
15               ha = 'center',
16               va = 'center')
17
18 # Lines with default zorder of 2
19 line1 = ax.axvline(0.65,
20                  linewidth = 10,
21                  color = blue)
22 line2 = ax.plot([0.35, 0.35], [.05, .95],
23                linewidth = 10,
24                color = blue)
25
26 # Patches with default zorder of 1
27 patch1 = ax.fill_between([0,1], 0.45, .55,
28                          facecolor = green,
29                          edgecolor = 'black')
30 patch2 = ax.fill_between([.48,.52], 0, 1,
31                          facecolor = red,
32                          edgecolor = 'black',
```

```

33         linewidth = 2)
34
35 # Check zorders
36 print(text.get_zorder())
37 print(line1.get_zorder())
38 print(line2[0].get_zorder())
39 print(patch1.get_zorder())
40 print(patch2.get_zorder())

```

[default-z.py](#)



Custom Ordering

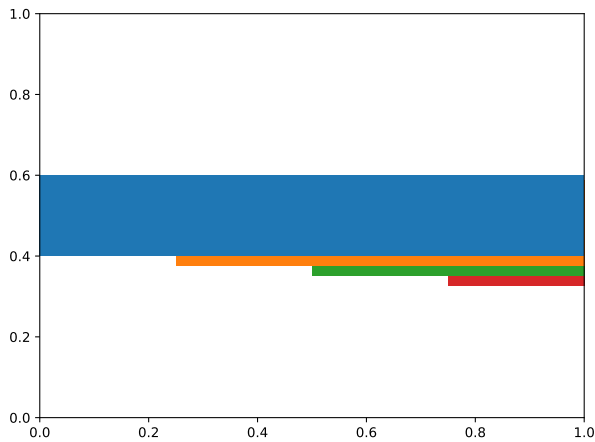
Then, we reverse the ordering.

```

1 fig, ax = plt.figure(), plt.axes()
2
3 print(fig.get_zorder())
4 print(ax.get_zorder())
5
6 for i in [0, 0.25, .5, .75]:
7
8     t = ax.fill_between([i, 1], 0.4 - i/10, .6 - i/20,
9                        zorder = 1 - i)
10    print(t.get_zorder())
11
12 ax.set_xlim(0,1)
13 ax.set_ylim(0,1)

```

[reverse-z.py](#)

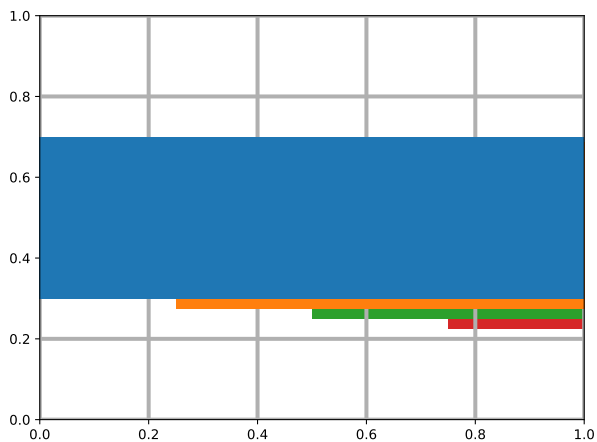


Axes and Tick Ordering

Notice that by default, gridlines are ordered below artists added to a plot regardless of where the call to show the gridlines is placed. This can be changed using `ax.set_axisbelow()`, which also reorders the ticks. The `XAxis` and `YAxis` can be ordered independently using the `set_order()` axis method.

```
1 fig, ax = plt.figure(), plt.axes()
2 for i in [0, 0.25, .5, .75]:
3     ax.fill_between([i,1], 0.3 - i/10, .7 - i/20,
4                     zorder = 2-i)
5 ax.grid(True, linewidth = 3)
6 ax.set_xlim(0,1)
7 ax.set_ylim(0,1)
8 print(ax.get_zorder())
```

`default-axes.py`

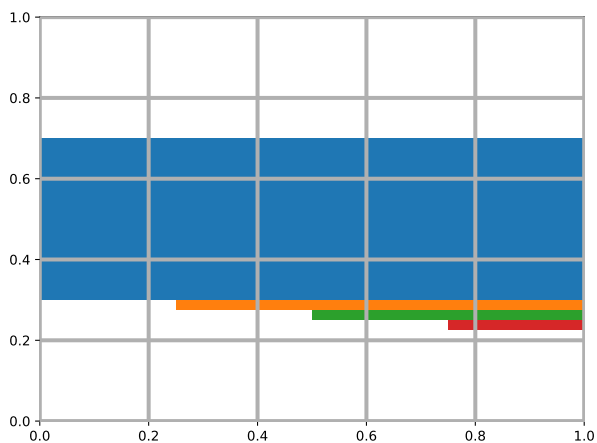


```

1 fig, ax = plt.figure(), plt.axes()
2 for i in [0, 0.25, .5, .75]:
3     ax.fill_between([i,1], 0.3 - i/10, .7 - i/20,
4                     zorder = 2-i)
5 ax.grid(True, linewidth = 3)
6 ax.set_xlim(0,1)
7 ax.set_ylim(0,1)
8 ax.set_axisbelow(False)
9 print(ax.get_zorder())

```

front-axes.py

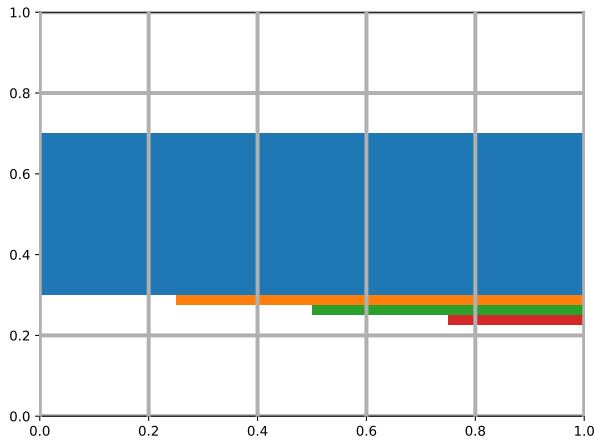


```

1 fig, ax = plt.figure(), plt.axes()
2 for i in [0, 0.25, .5, .75]:
3     ax.fill_between([i,1], 0.3 - i/10, .7 - i/20,
4                     zorder = 2-i)
5 ax.grid(True, linewidth = 3)
6 ax.set_xlim(0,1)
7 ax.set_ylim(0,1)
8 ax.xaxis.set_zorder(3)

```

front-axis.py



3.2 Coordinate Systems and Transformations

So far we have worked with data coordinates and you might not even realize there could be anything else. When we plotted a line between the points (0,0) and (1,1), we meant those as values in the usual xy -plane. But with use of transformations, we might also plot according to axes, figure, and display coordinates. In axes coordinates, (0,0) is the bottom left of the axes and (1,1) is the top right. Similarly, in figure coordinates, (0,0) is the bottom left of the figure and (1,1) is the top right. We won't cover the fourth type, display coordinates, which is the pixel coordinate system (for certain backends). The matplotlib [documentation](#) cautions that you should rarely work with display coordinates. However, display coordinates are a necessary evil when converting from one system to another. Note, it is important not to manipulate the figure or

axes dimensions after referencing the display coordinate system or you might encounter unexpected behavior.

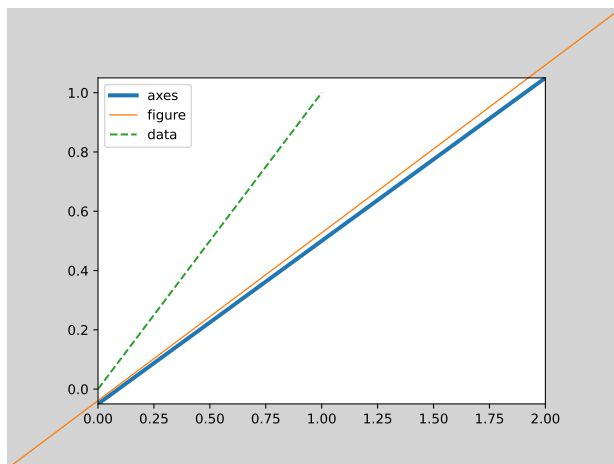
The plot below features a group of plot calls using axes coordinates, then a group using figure coordinates, and then a single call using data coordinates.

```

1 fig, ax = plt.figure(facecolor = 'lightgray'), plt.axes
  ()
2
3 ax.plot([0, 1], [0, 1],
4         linewidth = 3,
5         transform = ax.transAxes,
6         label = 'axes')
7
8 ax.plot([0, 1], [0, 1],
9         color = 'C1',
10        linewidth = 1,
11        transform = fig.transFigure,
12        clip_on = False,
13        label = 'figure')
14
15 ax.plot([0, 1], [0, 1],
16         color = 'C2',
17         linestyle = 'dashed',
18         clip_on = False,
19         label = 'data')
20
21 ax.set_xlim(0,2)
22 ax.legend()

```

coords.py

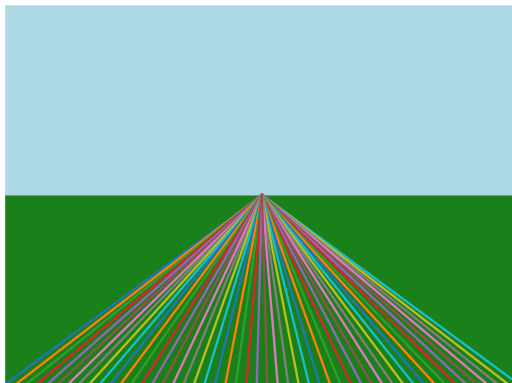


Axes and figure coordinates are often useful when you would like placement to be independent of the data, perhaps to enforce that something remain in the center of the plot by using an axes coordinate of 0.5. Below, we make use of that to set a vanishing point at the vertical halfway point.

```

1 fig, ax = plt.figure(), plt.axes()
2 ax.axis('off')
3 # lines to horizon
4 for i in np.linspace(0,1,50):
5     ax.plot([i,.5], [0.00, .5],
6             transform = ax.transAxes,
7             linewidth = 2,
8             zorder = 10-(i-0.5)**2)
9
10 # fill bottom half
11 green = (.1, .5, .1)
12 ax.fill_between(x = (0,1),
13                y1 = 0,
14                y2 = 0.5,
15                transform = ax.transAxes,
16                color = green)
17
18 # fill top half
19 ax.fill_between(x = (0,1),
20                y1 = 0.5,
21                y2 = 1,
22                transform = ax.transAxes,
23                color = 'lightblue')
```

[coord-horizon.py](#)



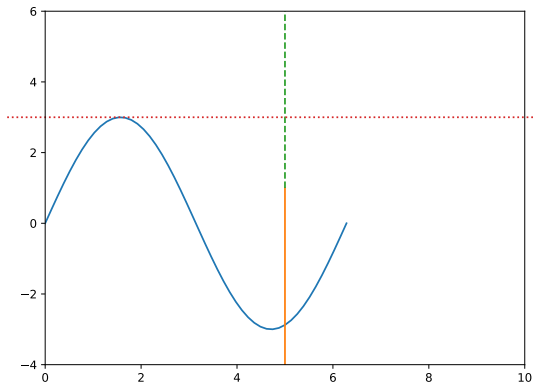
We can convert a point or sequence of points from one coordinate system to another using the appropriate transform ob-

ject. `ax.transData.transform([x,y])` converts `x,y` from data coordinates to display coordinates. Simply replacing `ax.transData` with `ax.transAxes` or `fig.transFigure` converts from the corresponding coordinate system to display coordinates. The opposite direction is achieved by inverting the transformation—`ax.transData.inverted().transform([x,y])`. To go from data coordinates to figure or axes coordinates, you can make a pit stop in display coordinates. For example, `ax.transData.inverted().transform(ax.transAxes.transform([0.5, 0.5]))` returns the middle of the axes window in data coordinates. The example below breaks this up into two steps. Again, take note that all plotting is done after setting a tight layout and after setting the axes limits to avoid resizing the figure and endangering the reliability of our coordinate transformations.

```

1 # Plot setup
2 fig, ax = plt.figure(), plt.axes()
3 x = np.linspace(0, 2*np.pi)
4 sin, = ax.plot(x, 3*np.sin(x))
5 ax.set_xlim(0, 10)
6 ax.set_ylim(-4, 6)
7 fig.tight_layout()
8
9 # Vertical line with axes coordinates
10 middle = [0.5, 0.5]
11 bottom_half = [0, 0.5]
12 ax.plot(middle, bottom_half,
13         transform = ax.transAxes)
14
15 # Continue vertical line with data coordinates
16 mid_in_display = ax.transAxes.transform([0.5, 0.5])
17 mid_in_data = ax.transData.inverted().transform(
18     mid_in_display)
19 top_mid_in_display = ax.transAxes.transform([0.5, 1])
20 top_mid_in_data = ax.transData.inverted()\
21     .transform(top_mid_in_display)
22 x = mid_in_data[0], top_mid_in_data[0]
23 y = mid_in_data[1], top_mid_in_data[1]
24 ax.plot(x, y, linestyle = 'dashed')
25
26 # Horizontal lines in figure coordinates
27 top_wave_display = ax.transData.transform([np.pi/2, 3])
28 top_wave_figure = fig.transFigure.inverted()\
29     .transform(top_wave_display)
30 y = top_wave_figure[1], top_wave_figure[1]
31 ax.plot([0,1], y,
32         transform = fig.transFigure,
33         linestyle = 'dotted',
34         clip_on = False)

```



3.3 Use Window Extents

Another useful method is `get_window_extent()`, which allows you to find the bounding box (the coordinates for the corners of the enclosing rectangle) for something added to a plot. This can be used to find the display coordinates for where an annotation begins or ends, for example. Like in the previous section, note that the results will not update and be inaccurate if changes are made to the figure size, axes limits, or the canvas used. The method also requires a renderer. The technicalities for why can be put aside. Either include `fig.canvas.draw()` first, so the rendered is already cached, or include the argument `renderer = fig.canvas.get_renderer()` in the call to `get_window_extent()`. Below is a simple example. We create a text object with the axes method `ax.text()` in the normal way, but we take the atypical step of assigning the object to a variable. Below, that variable is named `center_text` and then we call `get_window_extent()` as a Text method, or an Artist method more abstractly.

```

1 fig, ax = plt.figure(), plt.axes()
2
3 center_text = ax.text(0.5, 0.5,
4                       'centered text',
5                       ha = 'center')
6
7 fig.canvas.draw()
8 box = center_text.get_window_extent()
9 data_box = ax.transData.inverted().transform(box)
10
11 # left limit
12 ax.axvline(data_box[0][0],

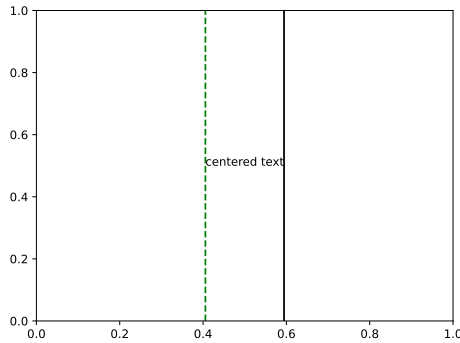
```

```

13         color = 'green',
14         linestyle = 'dashed')
15
16 # right limit
17 ax.axvline(data_box[1][0],
18            color = 'black')

```

[window-extent.py](#)



So what? A formatted title can stand in for a legend, helping reduce clutter. This helps us heed the call from Schwabish 2021 to label data directly and avoid legends when possible. In the line chart below, a legend is unnecessary given the color-coding in the title. We create a title not with the typical `ax.set_title()` but with a series of `ax.text()` calls. There are several because a single Text object can't have multiple colors. The `ha` parameter is for horizontal alignment, and this is covered in more detail in a later chapter. By using `ha = 'left'`, the text will begin at the given x and y coordinates.

```

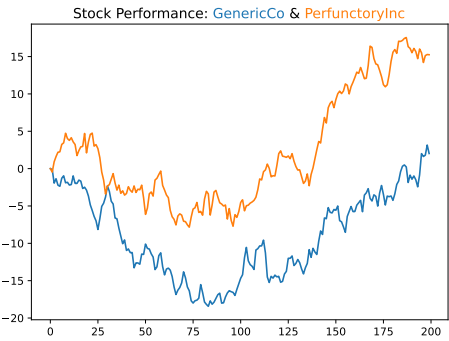
1 x_len = 200
2 x = range(0, x_len)
3
4 # Create a Gaussian random walk starting at 0
5 start = np.zeros(1)
6 y1 = np.concatenate([start, np.random.normal(0, 1, x_len
7 - 1)]).cumsum()
8
9 # Start plot
10 fig, ax = plt.figure(figsize = (7,5)), plt.axes()
11 fig.canvas.draw()

```

```

12
13 # Color arguments added to make defaults explicit
14 ax.plot(x, y1, color = 'C0')
15 ax.plot(x, y2, color = 'C1')
16
17 # Tuned by hand
18 shift = .099814 # Where titling starts on x-axis
19 y_level = 1.02
20 transform = ax.transAxes # use axes coords
21
22 t1 = ax.text(shift, y_level, 'Stock Performance:',
23             transform = transform,
24             ha = 'left',
25             fontsize = 13,
26             color = 'black')
27
28 # Get where text ended
29 x_pos = t1.get_window_extent()\
30         .transformed(transform.inverted()).x1
31
32 t2 = ax.text(x_pos, y_level, ' GenericCo',
33             transform = transform,
34             ha = 'left',
35             fontsize = 13,
36             color = 'C0')
37
38 x_pos = t2.get_window_extent()\
39         .transformed(transform.inverted()).x1
40
41 t3 = ax.text(x_pos, y_level, ' &',
42             transform = transform,
43             ha = 'left',
44             fontsize = 13,
45             color = 'black')
46
47 x_pos = t3.get_window_extent()\
48         .transformed(transform.inverted()).x1
49
50 t4 = ax.text(x_pos, y_level, ' PerfunctoryInc',
51             transform = transform,
52             ha = 'left',
53             fontsize = 13,
54             color = 'C1')
55
56 x_pos = t4.get_window_extent()\
57         .transformed(transform.inverted()).x1
58
59 # compare distances to the edge
60 # equal means perfect centering
61 print(shift, 1-x_pos)

```



Chapter 4

Text and Titles

4.1 Simple Titles

As we learned in Chapter 1, we can add a title with the axes method `set_title()`. Simply pass the string of your choice as the argument. For multi-line titles, recall `\n` can be used in a string to start a new line. Common optional arguments include `color`, `fontsize`, `weight`, and `loc`.

Colors will be addressed in Chapter 6, but to start you can simply use the name of any not-too-exotic color as a string.

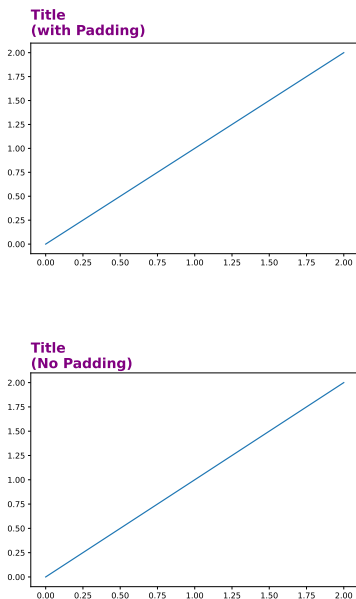
`fontsize` (or `size`) can be a number or chosen from `'small'`, `'medium'`, or `'large'`, and `'small'` and `'large'` may be intensified with a `'x-'` or `'xx-'` prefix. Similarly, `weight` (or `fontweight`) can be a number or chosen from options like `'bold'` or `'light'`.

`loc` determines the location of the title, either `'left'`, `'center'`, or `'right'`. In the default style, the default value will be `'center'`. You might prefer using `'left'` to match the Google Sheets default (thus matching the vast majority of plots I've seen in industry). `pad` controls the space between the title and the top of the axes.

```
1 x = np.linspace(0,2,2)
2 fig, ax = plt.figure(), plt.axes()
3
4 ax.plot(x,x)
5 ax.set_title("Title\n(with Padding)",
6             fontsize = 'xx-large',
7             weight = 'bold',
8             color = 'purple',
9             loc = 'left',
```

```
10 pad = 10)
```

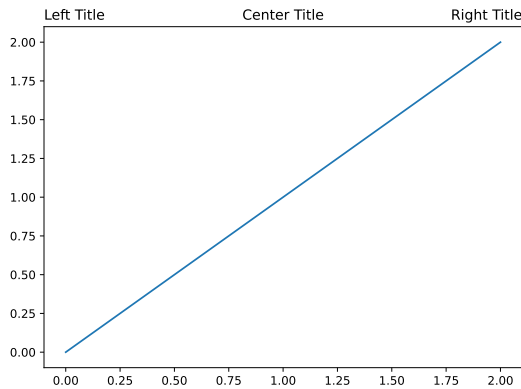
[title-pad.py](#)



A plot can actually have one title for every `loc` value as well.

```
1 x = np.linspace(0,2,2)
2 fig, ax = plt.figure(), plt.axes()
3
4 ax.plot(x,x)
5 ax.set_title("Left Title",
6             loc = 'left')
7 ax.set_title("Right Title",
8             loc = 'right')
9 ax.set_title("I won't be long for this world.",
10            loc = 'center')
11
12 # This only overwrites the center title above
13 ax.set_title("Center Title")
```

[title-loc.py](#)

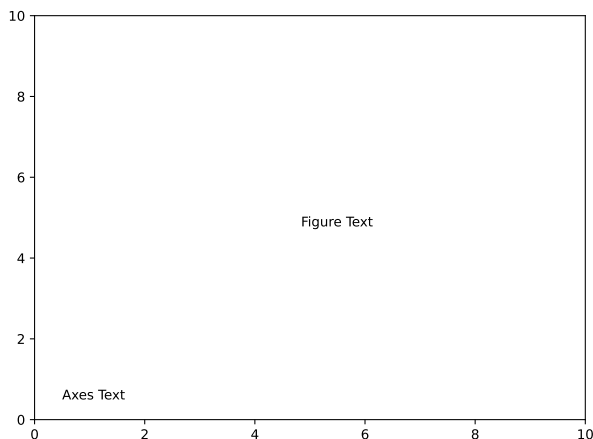


4.2 Text and Placement

Matplotlib offers `text` as both a figure and an axes method. Let's start with some code to understand what they do. Both take x and y positions as the first two arguments and then a string. The figure method method is the same as using the axes method with a transformation to figure coordinates.

```
1 fig, ax = plt.figure(), plt.axes()
2
3 ax.set_xlim([0,10])
4 ax.set_ylim([0,10])
5
6 fig.text(0.5, 0.5, 'Figure Text')
7 ax.text(0.5, 0.5, 'Axes Text')
```

[text-methods.py](#)



Immediately, we see that despite passing the same x and y position values, the figure and axes methods place the text differently. By default, the figure method uses “figure” coordinates, where (0,0) is the bottom left and (1,1) is the top right. The axes method uses x and y data coordinates by default. We will modify this shortly.

A more common concern is the alignment of the text. Both figure and axes text methods include parameters `verticalalignment` and `horizontalalignment`, which can be abbreviated as `va` and `ha`. By default, the text is placed so that the given coordinate is at the bottom-left corner of the text.

```
1 fig, ax = plt.figure(), plt.axes()
2 x, y = 0.5, 0.5
3 ax.scatter([x], [y])
4 ax.text(x,y, 'text', fontsize = 20)
5 ax.axis('off')
```

text-default-align.p

text

For vertical alignment, the options are `'top'`, `'bottom'`, or `'center'`. For horizontal alignment, the options are `'left'`, `'right'`, or `'center'`. The default demonstrated above was `'bottom'` and `'left'`. It does result in the text being above and to the right of the coordinate point, perhaps confusingly, but the interpretation is that the coordinate point is at the bottom-left of the text. The possible alignments are illustrated below.

```
1 fig, ax = plt.figure(), plt.axes()
2
3 x1, x2, y = 0.49, 0.51, 0.5
4 ax.scatter([x1,x2], [y,y])
5
6 va_options = ['top', 'bottom', 'center']
7 ha_options = ['left', 'right', 'center']
8
9 counter = 0 # for color cycling
10 for va in va_options:
11     for ha in ha_options:
12         # first letter of each option
13         label = va[0] + "-" + ha[0]
14
15         # assign label to point
16         x = x1
17         if 'c' in label:
18             x = x2
19
20         ax.text(x, y,
21               label,
22               va = va,
23               ha = ha,
24               fontsize = 20,
25               color = 'C'+str(counter))
26         counter += 1
27
28 ax.axis('off')
```

`text-align.py`

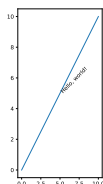
b-rb-l
t-t-l

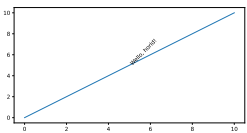
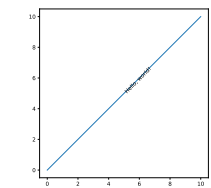
b-c
c-c-l

Text can be rotated with the `rotation` parameter. By default, a plot isn't square—the aspect ratio (the ratio of y -unit to x -unit) is not one. That means that the 45 degree line created by $y = x$ is not actually plotted at 45 degrees. Yet according to the `rotation` parameter, text rotated at 45 degrees is plotted at 45 degrees—that angle is not converted based on the aspect ratio. Later in Section 10.1, I go into further detail in how to use some trigonometry to get the exact angle if you'd like to slope text at some angle, accounting for the aspect ratio.

```
1 x = np.linspace(0,10)
2 fig, ax = plt.figure(), plt.axes()
3 ax.plot(x,x)
4 ax.set_aspect(2)
5 ax.text(5,5,
6         'Hello, world!',
7         rotation = 45)
```

[text-rotation2.py](#)





4.2.1 Text Formatting for Numbers

Here I’ve tucked away a subsection on formatting numbers in Python. This has nothing to do with matplotlib, formally speaking. Still, sometimes you want your text annotations or titles to contain numbers formatted just so and you’ll want Python to figure that out instead of doing it by hand. You might want commas as the thousands separator (the more readable 1,000,000 instead of 1000000), you might want leading zeros (01 instead of 1), or you might want a currency symbol (\$2 instead of 2). The table below demonstrates by example how to do this with `str.format`.

Code	Output
<code>'{:,'}.format(10**6)</code>	<code>'1,000,000'</code>
<code>'\${:,.2f}'.format(10**6)</code>	<code>'\$1,000,000.00'</code>
<code>'{:0>3.0f}'.format(1)</code>	<code>'001'</code>
<code>'{:>3.0f}'.format(1)</code>	<code>' 1'</code>
<code>'\${:0>4.0f}'.format(1)</code>	<code>'\$0001'</code>
<code>'{:+,.1f}'.format(1000)</code>	<code>'+1,000.0'</code>
<code>'{:0<+4,.1f}'.format(-1)</code>	<code>'-1.0'</code>
<code>'{:0<5.0f}'.format(1)</code>	<code>'10000'</code>
<code>'{:0<5,.0f}'.format(1)</code>	<code>'10000'</code>
<code>'{:0<8,.0f}'.format(1000)</code>	<code>'1,000000'</code>
<code>'{: .0e}'.format(10.1**6)</code>	<code>'1e+06'</code>
<code>'{: .1f} and {:.1f}'.format(9, 1)</code>	<code>'9.0 and 1.0'</code>
<code>'{1:.1f} and {0:.1f}'.format(9, 1)</code>	<code>'1.0 and 9.0'</code>
<code>'{0:}' and {0}'.format(1)</code>	<code>'1 and 1'</code>
<code>'{:}' and {:}'.format(1)</code>	<code>IndexError</code>

Understanding everything above requires some knowledge of [format specifications](#). A format specifier is a string that can specify fill, align, sign, width, grouping option, precision, and type (`[[fill] align][sign][#][0][width][grouping_option][.precision][type]`). These must be properly ordered but anything can be omitted to accept the default. These arguments go inside curly braces and to the right of a colon, `{:}`. The curly braces tell Python where to place the argument you pass to the `format()` method. You can also pass multiple arguments inside `format()`. By default, they are placed in order (the first argument replaces the first `{}` and so on), but to the left of the colon, you can also specify the index value for the argument to use.

The *fill* is a character that can be used to pad the number. Used with a *align* and *width*, we can add leading zeros. The default is a space if no fill character is provided. Using `'0>4'`, this will create leading zeros (right-aligned) up to a width of 4. So 1 becomes `'0001'` and 10000 is not padded, being simply `'10000'`.

The *grouping option* would come next, allowing for a thousands separator of a comma or an underscore. `'{: ,}'.format(10000)` produces `'10,000'`. Note that when used with padded numerals on the right, the padding is ignored in finding the thousands separators, so `'{: <08, .0f}'.format(1000)` produces the confusing `'1,000000'`.

Precision is next with a decimal and then how many digits to display past the decimal place or before and after, depending on the lastly specified *type*. Observe `'{: .2}'.format(np.pi)` produces `'3.1'` and `'{: .2f}'.format(np.pi)` produces `'3.14'`. You'll want type `'f'` for a float. Use `'e'` for scientific notation. You may read up on the many other types, including locale aware types, in the Python documentation.¹

Whatever we put outside the curly braces is simply concatenated to the text on the left or right. So `'${}'.format(123)` turns 123 into the dollar figure `'$1231'`. And `'{:} lbs.'.format(123)` would produce `'1231 lbs'`.

Perhaps this will come in handy when you'd like figure text or the filename in a certain format. I often use leading zeros in some filenames so that alphabetically ordering the files will be coherent (your file system will likely maintain `'1' < '10' < '2'`). If you are creating many plots that will be frames in an animation, and you'll have some number ticking up as the frames progress, the padding might help the eye.

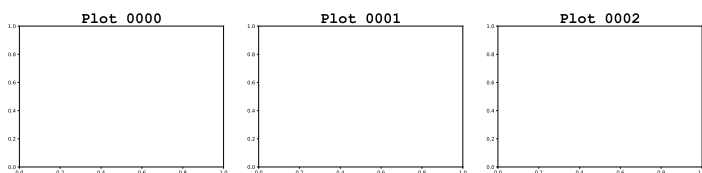
¹<https://docs.python.org/3/library/string.html#grammar-token-type>

```

1 for i in range(3):
2     fig, ax = plt.figure(), plt.axes()
3     label = '{:0>4}'.format(i)
4     ax.set_title("Plot " + label,
5                 fontname = 'Courier New',
6                 weight = 'bold',
7                 fontsize = 30)
8     fig.tight_layout()
9     fig.savefig(label + ".pdf")
10    plt.show()

```

[text-formatting.py](#)



4.3 Legends

As you should know, legends provide a key to the colors and symbols used in a plot. You can create a legend with `legend()`, as either a figure or axes method. Without any extra customization this is done with `ax.legend()` or `fig.legend()`. Here, we will only cover axes legends. We'll return to figure legends when they are more naturally useful in Chapter 7 on multiple axes and multiple plots.

But first, you need labels for your plot elements (called *artist* objects) before you can create a legend. This can be done with the `label` parameter in methods like `plot()`. Or you can use `set_label()` on the plot element object. Using `set_label()` adds some complication to the code, as seen below in an otherwise simple example. Note the legend needs to be added after the labeled plot elements you want included in the legend.

```

1 fig, ax = plt.figure(), plt.axes()
2 x = np.linspace(0, 2*np.pi, 100)
3
4 # Label in one go
5 ax.plot(x, np.sin(x), label = 'sine')
6
7 # Label as Artist method
8 cos, = ax.plot(x, np.cos(x))
9 cos.set_label('cosine')
10

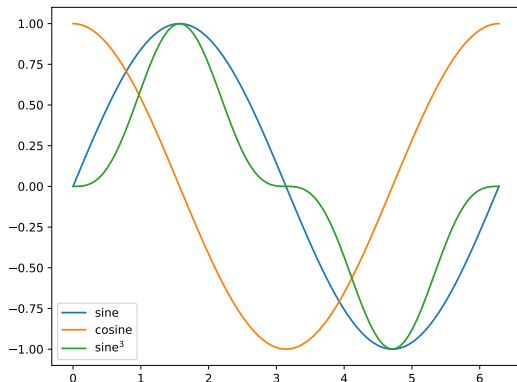
```

```

11 # Label as Artist method
12 sine3 = ax.plot(x, np.sin(x)**3)
13 sine3[0].set_label(r'sine$~3$')
14
15 # Construct legend
16 ax.legend()

```

[legend-labels.py](#)



If you are using a pandas plot method, the labels will be set automatically according to the column or series names. For such instances where an element is automatically included in a legend and you want to exclude it, you can exclude that element by specifying `label = '_nolegend_'` in the plot call.

```

1 # Construct DataFrame
2 n = 100
3 sqrts = np.concatenate([np.zeros(1), np.ones(n).cumsum()
4                          **0.5] )
5 ser1 = pd.Series(data = -sqrts, name = 'Lower Bound')
6 ser2 = pd.Series(data = sqrts, name = 'Upper Bound')
7 df = pd.DataFrame([ser1, ser2]).T
8
9 # Add random walk
10 df['Walk'] = np.concatenate([np.zeros(1), np.random.
11                             normal(size = n).cumsum()])
12
13 # Plot
14 fig, ax = plt.subplots()
15 df['Lower Bound'].plot(color = 'black', label = '
    Boundary')
16 df['Upper Bound'].plot(color = 'black', label = '
    _nolegend_')
17 df['Walk'].plot()

```

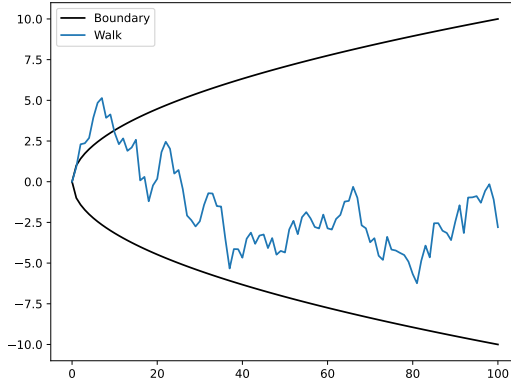


```

16
17 ax.legend()

```

pd-legend.py



A more common concern might be how to customize the placement of the legend and its actual appearance.

To change the placement of the legend, you may use the `loc` parameter. The default value is `'best'`, where best is determined by matplotlib. Other valid values are `'center'` and `'right'` (but not `'left'`) and then modifications like `'upper center'`, `'center right'`, and `'lower left'`.

For further customization of the placement, use the `bbox_to_anchor` parameter. This accepts 2-tuple or 4-tuple, giving the x location, the y location, and the width and height optionally.

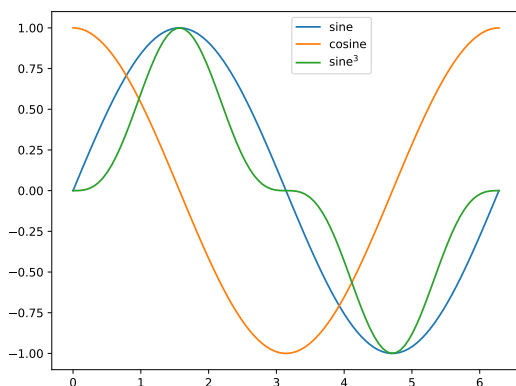
By default, x and y are in axes coordinates. So the program below places a legend in the top and center of the axes. The alignment is done according to `loc`. If, for example, `loc = 'lower right'`, then the lower right corner of the legend is placed at the specified x and y .

```

1 fig, ax = plt.figure(), plt.axes()
2 x = np.linspace(0,2*np.pi,100)
3 ax.plot(x, np.sin(x), label = 'sine')
4 ax.plot(x, np.cos(x), label = 'cosine')
5 ax.plot(x, np.sin(x)**3, label = r'sine$~3$')
6
7 # Construct legend
8 ax.legend(bbox_to_anchor = (0.5,1))

```

legend-bb.py

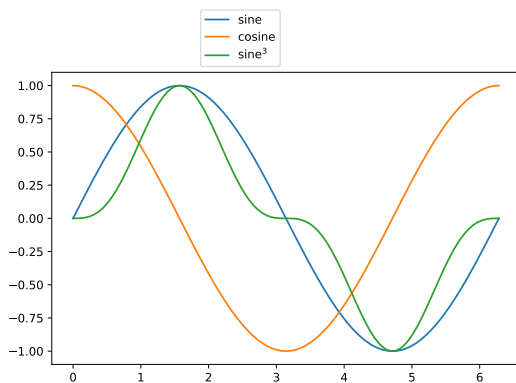


```

1 fig, ax = plt.figure(), plt.axes()
2 x = np.linspace(0,2*np.pi,100)
3 ax.plot(x, np.sin(x), label = 'sine')
4 ax.plot(x, np.cos(x), label = 'cosine')
5 ax.plot(x, np.sin(x)**3, label = r'sine$~3$')
6
7 # Construct legend
8 ax.legend(bbox_to_anchor = (0.5,1),
9           loc = 'lower right')

```

[legend-bb-loc.py](#)



If using a 4-tuple, the tuple is interpreted as the plot region in which to put the legend, according to `loc`.

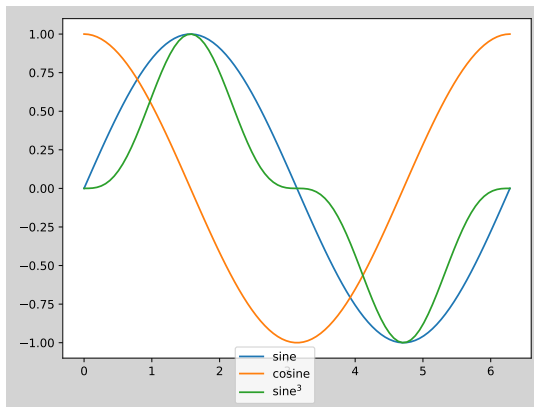
Use `bbox_transform` to use a coordinate system other than the default axes coordinates.

```

1 fig, ax = plt.figure(facecolor = 'lightgray'), plt.axes
  ()
2 x = np.linspace(0,2*np.pi,100)
3 ax.plot(x, np.sin(x), label = 'sine')
4 ax.plot(x, np.cos(x), label = 'cosine')
5 ax.plot(x, np.sin(x)**3, label = r'sine$^3$')
6
7 # Construct legend
8 ax.legend(bbox_to_anchor = (0.5,0),
9           loc = 'lower center',
10          bbox_transform = fig.transFigure)

```

[legend-transform.py](#)



There are many parameters to change the appearance of the legend. We won't cover all of them. Two useful parameters are `facecolor` and `ncol`. The former changes the background color of the legend and the latter sets the number of columns, changing the default shape of the legend. I use these and a few other self-explanatory parameters in the program below.

```

1 fig, ax = plt.figure(), plt.axes()
2
3 x = np.linspace(0,2*np.pi,100)
4
5 ax.plot(x, np.sin(x), label = 'sine')
6 ax.plot(x, np.cos(x), label = 'cosine')
7 ax.plot(x, np.sin(x)**3, label = r'sine$^3$')
8
9 # Construct legend
10 ax.legend(bbox_to_anchor = (0.5,-0.3),
11          loc = 'lower center',
12          ncol = 3,
13          facecolor = 'lightgray',

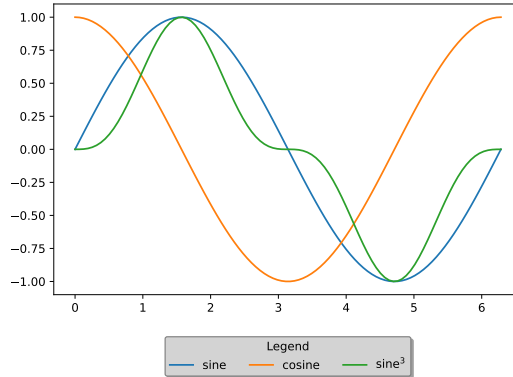
```

```

14     edgecolor = 'gray',
15     shadow = True,
16     title = 'Legend')

```

[legend-shape.py](#)



4.4 Annotations

Knaflitz [2015](#) and Schwabish [2021](#) both advise to label data directly and to annotate graphs with explanatory notes when helpful, as this helps convey the meaning of the graph more simply and directly.

You can annotate a chart with `text()` method calls, or you can use the `annotate()` method, for which you specify the text placement and a line segment to the part of the graph the text references.

4.4.1 Labeling and Arrows

The following graph is nothing special, but we avoid having to create a legend by labeling the data with the text color matching the line color.

```

1 fig, ax = plt.figure(), plt.axes()
2 x = np.arange(-10,6,1)
3 past = x[x<=0]
4 future = x[x>=0]
5
6 y_historical = np.random.normal(0,1,size = len(past))
7 y_projected = np.concatenate([y_historical[-1:],
8                               np.random.normal(0,3, size = len(
9                                   future)-1)])

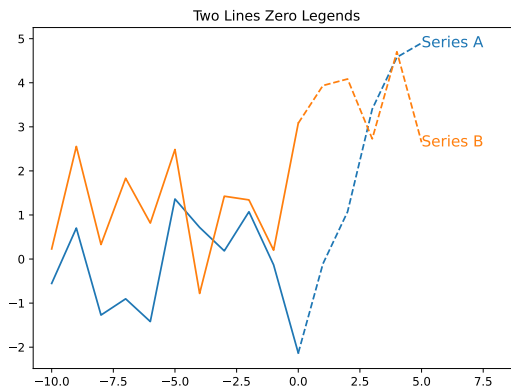
```

```

10 z_historical = np.random.normal(1,1,size = len(past))
11 z_projected = np.concatenate([z_historical[-1:],
12                               np.random.normal(3,1, size = len(
13                                   future)-1)])
14 ax.plot(past, y_historical)
15 ax.plot(future, y_projected, linestyle = 'dashed', color
16         = 'C0')
17 ax.plot(past, z_historical, color = 'C1')
18 ax.plot(future, z_projected, linestyle = 'dashed', color
19         = 'C1')
20 # Label Data
21 ax.text(future[-1], y_projected[-1],
22         s = 'Series A',
23         va = 'center',
24         color = 'C0',
25         size = 13)
26 ax.text(future[-1], z_projected[-1],
27         s = 'Series B',
28         va = 'center',
29         color = 'C1',
30         size = 13)
31
32 ax.set_xlim(ax.get_xlim()[0], 9)
33 ax.set_title("Two Lines Zero Legends")

```

[label-data.py](#)



Next, we use the `annotate()` method. This method comes with the option to include an arrow pointing from `xytext` to the point `xy`.

```

1 fig, ax = plt.figure(), plt.axes()

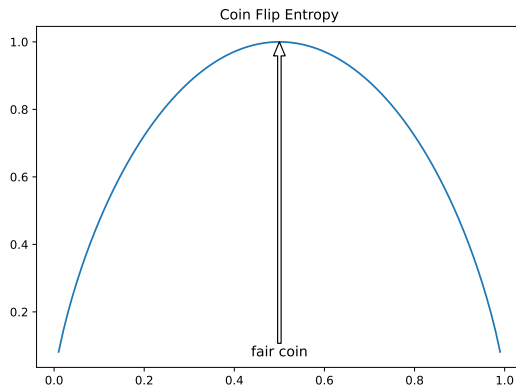
```

```

2
3 x = np.linspace(0,1,100) # Pr(heads)
4 x = x[(x!=0) & (x!=1)]
5 entropy = -x*np.log2(x) - (1-x)*np.log2(1-x)
6 ax.plot(x,entropy)
7 ax.annotate('fair coin',
8             xy = (0.5,1),
9             xytext = (0.5, 0.1),
10            arrowprops=dict(facecolor='white',
11                            edgecolor = 'black',
12                            width = 3,
13                            headwidth = 10,
14                            linewidth = 1),
15            ha = 'center',
16            va= 'top', # text alignment around xytext
17            size = 12)
18
19 ax.set_title("Coin Flip Entropy")

```

[annotate-arrow.py](#)



If you would like an arrow and no text, simply use the empty string ''. It is necessary to pass a dictionary to the `arrowprops` property.

```

1 fig, ax = plt.figure(), plt.axes()
2
3 # no arrow, no text
4 # this does nothing
5 ax.annotate('',
6             xy = (0.1, 0.8),
7             xytext = (0.9, 0.9))
8
9 # arrow
10 ax.annotate('', xy = (0.2, 0.2),

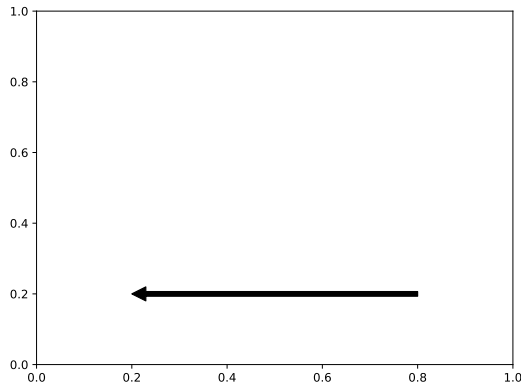
```

```

11     xytext = (0.8, 0.2),
12     arrowprops = dict(color = 'black'))

```

[arrow-only.py](#)



Lastly, one can also reference specific artist objects in the annotation instead of coordinates. In the below we place the annotations at the end of `a_line` and `b_line`.

```

1 fig, ax = plt.figure(), plt.axes()
2 x = np.arange(-10,6,1)
3 past = x[x<=0]
4 future = x[x>=0]
5 y_historical = np.random.normal(0,1,size = len(past))
6 y_projected = np.concatenate([y_historical[-1:],
7                               np.random.normal(0,3, size = len(
8                                   future)-1)])
9 z_historical = np.random.normal(1,1,size = len(past))
10 z_projected = np.concatenate([z_historical[-1:],
11                               np.random.normal(3,1, size = len(
12                                   future)-1)])
13 ax.plot(past, y_historical)
14 a_line, = ax.plot(future, y_projected, linestyle = '
15     dashed', color = 'C0')
16 ax.plot(past, z_historical, color = 'C1')
17 b_line, = ax.plot(future, z_projected, linestyle = '
18     dashed', color = 'C1')
19
20 # Label Data
21 ax.annotate('Series A',
22             xy = (1, y_projected[-1]),
23             xycoords = (a_line, 'data'),
24             color = 'C0',
25             size = 12)

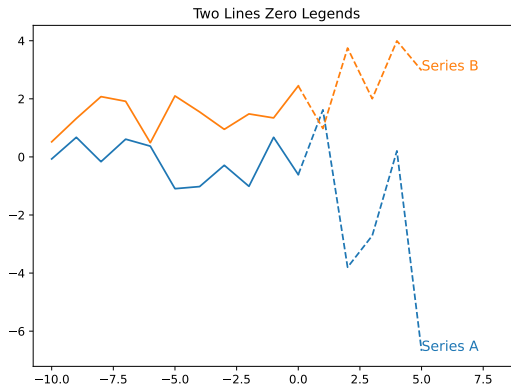
```

```

23 ax.annotate('Series B',
24             xy = (1, z_projected[-1]),
25             xycoords = (b_line, 'data'),
26             color = 'C1',
27             size = 12)
28
29 ax.set_xlim(ax.get_xlim()[0], 9)
30 ax.set_title("Two Lines Zero Legends")

```

[direct-annotation.py](#)



4.5 Fancy Titles

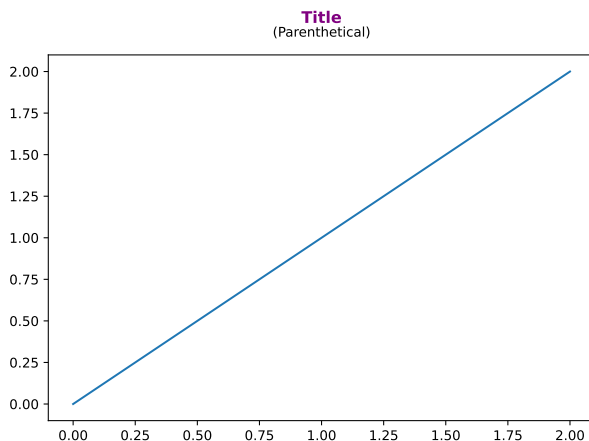
If you'd like to format different parts of the title different, you'll have to move beyond simply using `set_title`. The New York Times, for example, routinely includes a title and a subtitle in a plot. This requires using `text()` and `set_title()` separately, as there can only be one format style applied to a title. A simple example is below.

```

1 x = np.linspace(0,2,2)
2 fig, ax = plt.figure(), plt.axes()
3 ax.plot(x,x)
4
5 ax.set_title("Title",
6             weight = 'bold',
7             color = 'purple',
8             pad = 24)
9
10 ax.text(0.5, 1.05,
11         s = '(Parenthetical)',
12         transform = ax.transAxes,
13         ha = 'center')

```

[subtitle.py](#)



4.5.1 Multi-colored Titles

In Chapter 3, we created a multi-colored title using the Artist method `get_window_extent()`. The advantage of a multi-colored title is that we can do without a legend. For someone who doesn't want to get into the complications of `get_window_extent()`, the x and y placement of the text could be done by sight.

```

1 x = range(101)
2
3 # Create a Gaussian random walk starting at 0
4 start = np.zeros(1)
5 y1 = np.concatenate( [start,np.random.normal(0,1,100)] )
6     .cumsum()
7 y2 = np.concatenate( [start,np.random.normal(0,1,100)] )
8     .cumsum()
9
10 fig, ax = plt.figure(), plt.axes()
11 plt.tight_layout()
12 # Color arguments added to make defaults explicit
13 ax.plot(x,y1, color = 'C0')
14 ax.plot(x,y2, color = 'C1')
15
16 ax.text(0.4, 1.05, 'GenericCo',
17         transform = ax.transAxes,
18         ha = 'left',
19         fontsize = 13,
20         color = 'C0')
21
22 ax.text(0.4, 1.05, 'Stock Performance:',
23         transform = ax.transAxes,
24         ha = 'right',

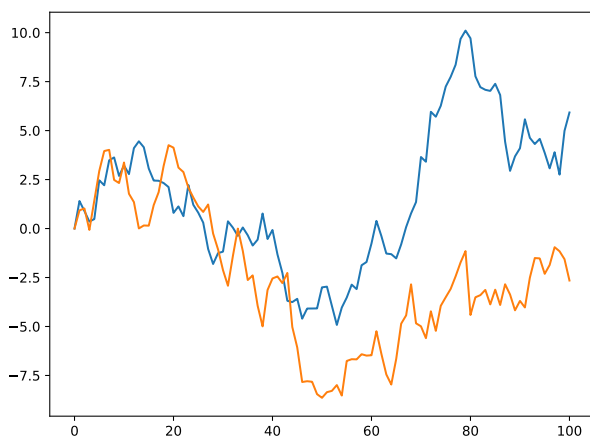
```

```

23         fontsize = 13,
24         color = 'black')
25
26 ax.text(0.64, 1.05, '&',
27         transform = ax.transAxes,
28         ha = 'right',
29         fontsize = 13,
30         color = 'black')
31
32 ax.text(0.64, 1.05, ' PerfunctoryInc',
33         transform = ax.transAxes,
34         ha = 'left',
35         fontsize = 13,
36         color = 'C1')

```

multicolor-inexact.p



Greater elegance requires greater complication. If you are (understandably) dissatisfied with the above, invest in the topics covered in Chapter 3. Below, we build on the solution from Chapter `chapter:elements` by creating a function that creates a multi-colored title. Note we remove text options with the `remove()` method and work all in a single figure. This replaces the work of tuning the centering by hand that was done previously.

```

1 def color_title(labels, colors, textprops ={'size': '
   large'}, ax = None, y = 1.013,
2         precision = 10**-2):
3
4     "Creates a centered title with multiple colors. "
5

```

```

6     if ax == None:
7         ax = plt.gca()
8
9     plt.gcf().canvas.draw()
10    transform = ax.transAxes # use axes coords
11
12    # initial params
13    xT = 0 # where the text ends in x-axis coords
14    shift = 0 # where the text starts
15
16    # for text objects
17    text = dict()
18
19    while (np.abs(shift - (1-xT)) > precision) and (
20    shift <= xT) :
21        x_pos = shift
22
23        for label, col in zip(labels, colors):
24
25            try:
26                text[label].remove()
27            except KeyError:
28                pass
29
30            text[label] = ax.text(x_pos, y, label,
31                                transform = transform,
32                                ha = 'left',
33                                color = col,
34                                **textprops)
35
36            x_pos = text[label].get_window_extent()\
37                    .transformed(transform.inverted()).x1
38
39            xT = x_pos # where all text ends
40
41            shift += precision/2 # increase for next
42            iteration
43
44            if x_pos > 1: # guardrail
45                break

```

color-title.py

```

1 x = range(101)
2 # Create a Gaussian random walk starting at 0
3 start = np.zeros(1)
4 y1 = np.concatenate( [start,np.random.normal(0,1,100)] )
5     .cumsum()
6 y2 = np.concatenate( [start,np.random.normal(0,1,100)] )
7     .cumsum()
8
9 fig, ax = plt.figure(), plt.axes()
10 plt.tight_layout()

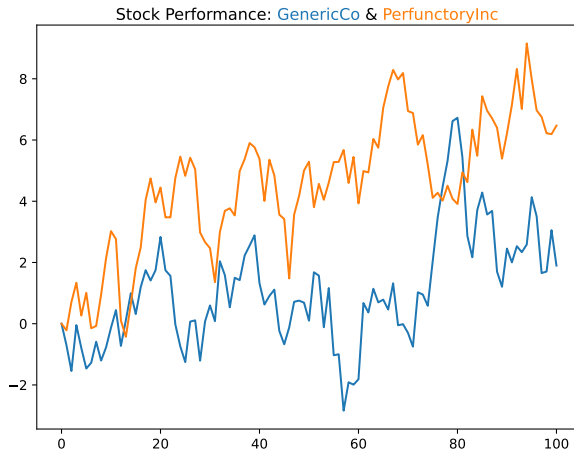
```

```

9 # Color arguments added to make defaults explicit
10 ax.plot(x,y1, color = 'C0')
11 ax.plot(x,y2, color = 'C1')
12
13 labels = ['Stock Performance: ', 'GenericCo', ' & ', '
14           PerfunctoryInc']
15 colors = ['black', 'C0', 'black', 'C1']
16 color_title(labels, colors)

```

[color-title-ex.py](#)



4.6 Fonts

Finally, you might want to customize the fonts. In matplotlib 3.6 and newer, there is a `get_font_names()` method that can be used to display available font names. The code below creates a figure for each font. I get several warnings with messages like “Glyph 105 (i) missing from current font.”

```

1 font_list = font_manager.get_font_names()
2 for f in font_list:
3     plt.plot()
4     plt.gcf().text(0.5, 0.5,
5                     s = f,
6                     size = 20,
7                     ha = 'center',
8                     fontname = f)
9     plt.axis('off')
10    plt.show()

```

[get-fonts.py](#)

4.6.1 Importing Fonts with Font Manager

If you're unsatisfied with the basic fonts available in matplotlib, just add your own. You can find fonts available for download from theleagueofmoveabletype.com or fonts.google.com.

After you've downloaded a font family, you should have folder for that font with otf or ttf files. Matplotlib has a font manager and you just need to tell matplotlib to look for a font in that folder. This is done below using `fontSystemFonts()` and `addfont()`. Once the font files are added, you can simply specify the font in the `text()` call like any other in-built font.

```
1 # font download
2 # https://fonts.google.com/specimen/Pacifico
3 # access font and add to font manager
4 font_dirs = ['Downloads/Pacifico'] # change depending on
   where you downloaded it
5 font_files = font_manager.findSystemFonts(fontpaths=
   font_dirs)
6 for font_file in font_files:
7     font_manager.fontManager.addfont(font_file)
8
9 # Make Figure
10 fig, ax = plt.figure(), plt.axes()
11 t = fig.text(0.5,0.5,
12             'Live Laugh Love',
13             ha = 'center',
14             va = 'center')
15 ax.axis('off')
16 t.set_size(50)
17 t.set_name("Pacifico")
18 t.set_color('yellow')
19 fig.set_facecolor('brown')
```

`font.py`



Live Laugh Love

Chapter 5

Dates

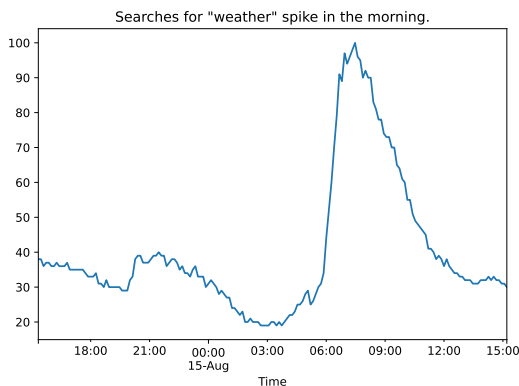
Matplotlib can handle dates, helping you to create better axis ticks and label formatting. Matplotlib's capabilities are built on the `datetime` and `dateutil` modules.

5.1 Plotting

Let's import some time series data. Below we use pandas integration and plot from a DataFrame with an index of pandas Timestamp values. Matplotlib recognizes these as dates and handles this reasonably well automatically, though the exact formatting could be improved.

```
1 url = 'https://github.com/alexanderthclark/MPL-Data/raw/  
    main/WeatherAug1415Trends.csv'  
2 df = pd.read_csv(url, parse_dates = ['Time'])  
3  
4 fig, ax = plt.figure(), plt.axes()  
5 df.set_index("Time").plot(ax = ax)  
6 ax.set_title("Searches for \"weather\" spike in the  
    morning.")  
7 ax.legend().set_visible(False)
```

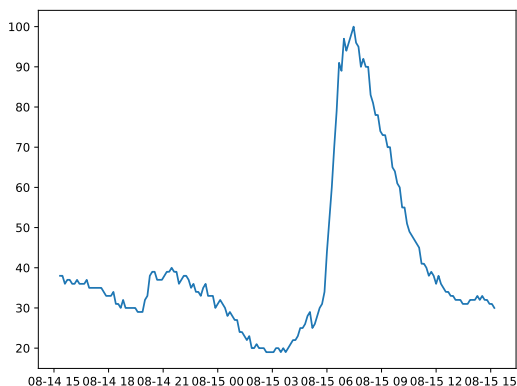
[pd-dates.py](#)



Before we try to improve the formatting, see what happens if we try to use the axes plot method.

```
1 fig, ax = plt.figure(), plt.axes()
2 ax.plot(df.Time, df.weather)
```

[ax-dates.py](#)



You might find older code using `plot_date()`. The method was removed in matplotlib 3.11; pass datetime-like values directly to `plot()` instead.

5.1.1 Time Zone Handling

Matplotlib's date converters, locators, and formatters are time-zone aware. If no time zone is specified, Matplotlib uses `rcParams['timezone']`, which is `'UTC'` by default. You can choose the time

zone used for tick labels by passing an IANA time-zone name to a formatter, as in `mdates.DateFormatter('%H:%M %Z', tz = 'America/New_York')`. The underlying observations do not move; only their displayed clock times change.

It is important to distinguish conversion from localization. A time-zone-aware timestamp identifies an unambiguous instant and can be converted for different audiences while a naive timestamp contains no such information. With `pandas`, `tz_localize()` attaches a time zone to naive dates, while `tz_convert()` converts dates that are already time-zone aware. Localize only when you know which time zone the recorded clock times represent. Otherwise, the same numbers can be assigned to the wrong instant.

Named zones are better than fixed offsets because their rules account for daylight-saving transitions. Python's standard-library `zoneinfo` module supplies `tzinfo` objects for these zones. For plotting, a zone name or `tzinfo` object can be passed to Matplotlib's [date locators and formatters](#). Best practice is to store an unambiguous time, commonly in UTC, and convert it only when choosing how the audience should read the axis.

5.2 Ticks and Formatting

5.2.1 Date Formats

The specific format of the displayed dates and times can be modified with `mdates.DateFormatter()`. This takes a format string and creates a formatter that can be passed to an axis method `set_major_formatter()` or `set_minor_formatter()`.

Here are some common format codes, applied to Sunday January 30, 2000, 11:59PM, local to Louisville, Kentucky. These can all be verified with `pd.Timestamp(year = 2000, month = 1, day = 30, hour = 23, minute = 59, tz = 'America/Kentucky/Louisville').strftime()`.

Code	Output/Example
<code>'%Y'</code>	4-Digit Year
<code>'%m'</code>	Month Number
<code>'%d'</code>	Day of Month
<code>'%B'</code>	Month Name
<code>'%H'</code>	24-Hour Clock Hour
<code>'%M'</code>	Minute
<code>'%h'</code>	12-Hour Clock Hour
<code>'%p'</code>	AM or PM
<code>'%A'</code>	Day of Week
<code>'%Z'</code>	Timezone Name
<code>'%Y-%m'</code>	<code>'2000-01'</code>
<code>'%Y/%m/%d'</code>	<code>'2000/01/30'</code>
<code>'%B %y'</code>	<code>'January 00'</code>
<code>'%H:%M %Z'</code>	<code>'23:59 EST'</code>
<code>'%A %I%p'</code>	<code>'Sunday 11PM'</code>

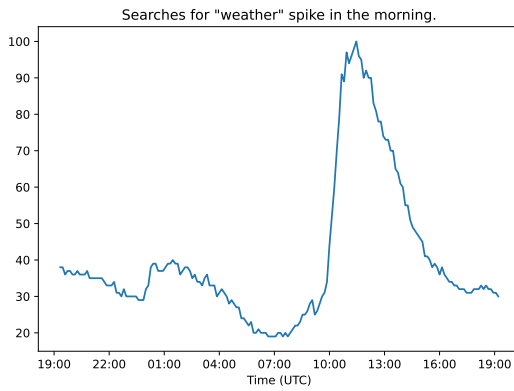
A more complete list of format codes can be found in the [Python documentation](#). Codes that generate actual names, like `'%A'` or `'%B'`, can be made lowercase to produce an abbreviated name. Notice that these formats create zero-padded numbers like `'07'` instead of `'7'`. On Mac or Linux, padding can be eliminated with the `'-'` modifier, using `'%-H'` or `'%-m'` instead of `'%H'` or `'%m'` for example. On Windows, use `'#'`.

```

1 fig, ax = plt.figure(), plt.axes()
2 xformatter = mdates.DateFormatter('%H:%M')
3 ax.plot(df.Time, df.weather)
4 ax.set_title("Searches for \"weather\" spike in the
5               morning.")
6 ax.set_xlabel("Time (UTC)")
6 ax.xaxis.set_major_formatter(xformatter)

```

date-fmt.py

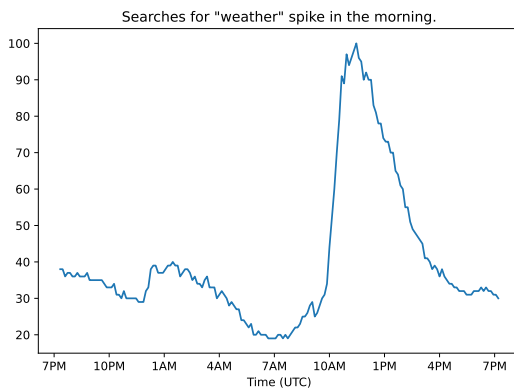


```

1 fig, ax = plt.figure(), plt.axes()
2 xformatter = mdates.DateFormatter('%-I%p')
3 ax.plot(df.Time, df.weather)
4 ax.set_title("Searches for \"weather\" spike in the
5             morning.")
6 ax.set_xlabel("Time (UTC)")
7 ax.xaxis.set_major_formatter(xformatter)

```

[date-fmt2.py](#)



Chapter 6

Colors

Methods like `plot` and `text` include a color parameter, which we've already made use of. While you can get pretty far simply using `color = 'blue'`, you might also make use of colormaps or set your own colors using hex strings or RGB(A) tuples.

6.1 Colormaps

According to the style sheet you are using, there will be some colormap and you will cycle through those colors by default when plotting (but not for text). The colors can be identified by the strings `'C0'`, `'C1'`, If, as in the default, your color map has only 10 distinct colors, then the eleventh color `'C10'` is valid, but simply refers to `'C0'` and the colors cycle from there. You'll notice that with successive plot calls on the same axes, the colors will automatically move through the colormap. This is not the case with text, as is demonstrated in the program below.

```
1 fig, ax = plt.figure(), plt.axes()
2 for i in range(12):
3     # Plot color automatically cycles through color map
4     ax.plot([0,1], np.ones(2)*i)
5
6     # Text with default color on the left
7     ax.text(0, i, 'C' + str(i),
8            va = 'center', ha = 'right')
9
10    # Text with variable color on the right
11    ax.text(1, i, 'C' + str(i),
12           va = 'center', ha = 'left',
13           color = 'C'+str(i))
```

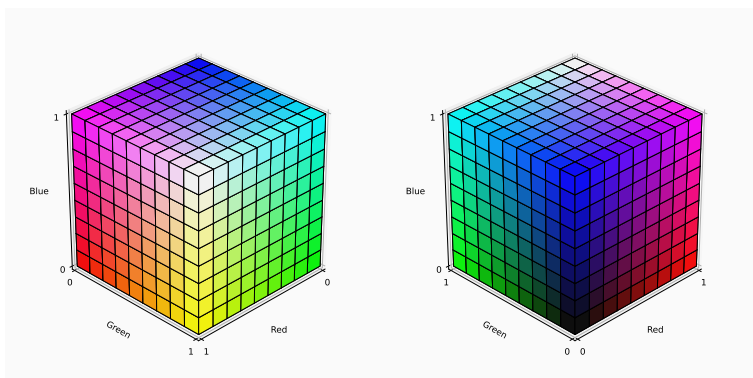
```
14 ax.axis('off')
```

colors.py



6.2 Red, Green, Blue, Alpha

An RGB color is given by three values, specifying the amount of red, green, and blue. In matplotlib, these values are between zero and one (you might also see RGB values between zero and 255 elsewhere). These colors live inside a cube, as a particular color is a triple $(r, g, b) \in [0, 1]^3$.



I like working with RGB tuples because they can be manipulated with mathematical operations. Two colors can easily be averaged or we can create a gradient between two.

```
1 # Set Colors
2 green = 76, 217, 100
3 green = np.array(green)/255
4 blue = 90, 200, 250
5 blue = np.array(blue)/255
6
7 # How many color changes
8 segments = 100
9 interval_starts = np.linspace(0, 1, segments)
10
11 fig, ax = plt.subplots(figsize = (8,8))
12
13 colors = dict()
14 for i in range(3):
15     colors[i] = np.linspace(blue[i], green[i], segments)
16
17 for i in range(segments-1):
18     rgb = colors[0][i], colors[1][i], colors[2][i]
19     x = interval_starts[i], interval_starts[i+1]
20     y = (0.5, 0.5)
21     ax.plot(x, y, color = rgb,
22             linewidth = 20,
23             solid_capstyle = 'round')
24
25 ax.set_aspect('equal')
26 ax.axis('off')
```



Any color can be made lighter by averaging it with white, (1,1,1), or darker by averaging it with black (0,0,0). We can also find the inverse of an RGB color by simply subtracting that triple from (1,1,1). RGBA tuples are very similar, adding a fourth alpha value for the opacity.

With RGB and RGBA colors being so handy, you might want to convert strings like 'C0' into RGB. `ColorConverter()` lets us do this, with the `to_rgb()` and `to_rgba()` methods. Below, we create another color gradient between the default 'C0' blue, to 'C1' orange, and on to light blue 'C9'.

```
1 # Set Colors
2 blue = mpl.colors.ColorConverter().to_rgb('C0')
3 orange = mpl.colors.ColorConverter().to_rgb('C1')
4
5 n_colors = 10
6 color_strings = dict()
7 for i in range(n_colors):
8     color_strings[i] = 'C'+str(i)
9 segments = 1000 # How many color changes
10
11 fig, ax = plt.subplots(figsize = (14,8))
12
13 for c in range(n_colors - 1):
14     color1 = mpl.colors.ColorConverter().to_rgb(
        color_strings[c])
```



```

15     color2 = mpl.colors.ColorConverter().to_rgb(
16         color_strings[c+1])
17
18     interval_starts = np.linspace(c, c+1, segments)
19     colors = dict()
20     for i in range(3):
21         colors[i] = np.linspace(color1[i], color2[i],
22             segments)
23
24     for i in range(segments-1):
25
26         rgb = colors[0][i], colors[1][i], colors[2][i]
27
28         x = interval_starts[i], interval_starts[i+1]
29         y = [0.3, 0.5]
30
31         ax.plot(x, y,
32             color = rgb,
33             linewidth = 20,
34             solid_capstyle = 'round')
35
36     ax.text(c, .51,
37         s = 'C'+str(c),
38         va = 'bottom',
39         size = 12,
40         ha = 'center')
41
42 ax.text(9, .51,
43     s = 'C9',
44     va = 'bottom',
45     size = 12,
46     ha = 'center')
47
48 ax.set_aspect('equal')
49 ax.axis('off')

```

color-map.py



Color Cube Code

Here is the code for one of the RGB color cubes.

```

1 light_gray = [.98]*3
2 fig = plt.figure(figsize = (6,6),
3                     facecolor = light_gray)
4 ax = plt.axes(projection='3d',
5               facecolor = light_gray)
6
7 # control how many cubes/color changes
8 pieces = 10
9 grid = np.linspace(0, 1, pieces)[: -1]
10 width = grid[1] - grid[0]
11
12 # Make smaller cube units
13 for x in grid:
14     for y in grid:
15         for z in grid:
16             vertices = list()
17             for prod in product([x,x+width],[y,y+width],
18                               [z,z+width]):
19                 vertices.append(list(prod))
20
21             faces = list()
22             for key, face in enumerate([x,y,z]):
23                 # face is 0
24                 helper0 = [x for x in vertices if x[key]
25 == face]
26                 helper1 = [x for x in vertices if x[key]
27 == face + width]
28                 helper0.sort()
29                 helper0 = helper0[0:2] + helper0
30                 helper0[0:2]
31                 helper1.sort()
32                 helper1 = helper1[0:2] + helper1
33                 helper1[0:2]
34                 faces.append((helper0))
35                 faces.append(helper1)
36
37             facecolor = (x + width / 2,
38                       y + width / 2,
39                       z + width / 2)
40             pc = Poly3DCollection(faces,
41                                 facecolor = facecolor,
42                                 edgecolor = 'black')
43             ax.add_collection3d(pc)
44
45 # Label Axes
46 ax.set_xlabel("Red")
47 ax.set_ylabel('Green')
48 ax.set_zlabel("Blue")

```

```
45 # Set Ticks
46 ax.set_xticks([0,1])
47 ax.set_yticks([0,1])
48 ax.set_zticks([0,1])
49 # Change padding
50 ax.xaxis.set_tick_params(pad = 0.1)
51 ax.yaxis.set_tick_params(pad = 0.1)
52 ax.zaxis.set_tick_params(pad = 0.1)
53 # Change azimuth
54 angle = 45 # + 180 # for second cube
55 ax.view_init(elev = None, azimuth = angle)
56 # Zoom out so labels are not cut off
57 ax.set_box_aspect([1,1,1], zoom = 0.86)
```

[color-cube.py](#)

Chapter 7

Multiple Axes and Plots

7.1 Multiple Axes

Let's start with a concrete goal to help illustrate possible uses of multiple axes. We want to plot a standard normal distribution. This is the familiar bell curve with a range of possible draws from the normal distribution on the x -axis and y values are the value of the probability density function (PDF) evaluated at each x value. Furthermore, we have a z -score and we want the visual to help us see how often we should get smaller z -scores if we are sampling from this distribution. In particular, let's say our z -score is 0.674.

To answer the question narrowly, the following plot does the job well and without reaching for multiple x - or y -axes.

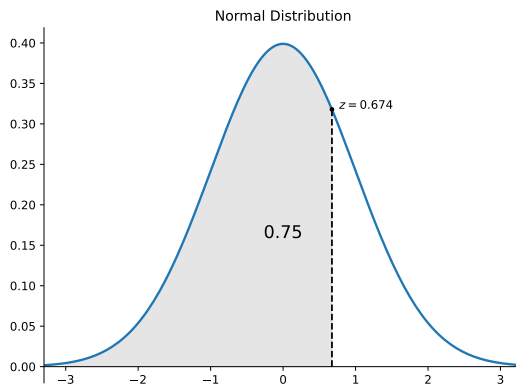
```
1 fig, ax = plt.figure(), plt.axes()
2 ax.set_xlim(-3.3, 3.3)
3 x = np.linspace(-4, 4, 200)
4 ax.plot(x, stats.norm.pdf(x),
5         linewidth = 2)
6 ax.set_title("Normal Distribution")
7
8 # Choose a point
9 z = 0.674
10 pdfz = stats.norm.pdf(z)
11 cdfz = stats.norm.cdf(z)
12
13 # Indicate point on plot
14 ax.plot([z, z],
15         [0, pdfz],
16         color = 'black',
17         linestyle = 'dashed')
18 ax.plot([z], [pdfz],
```

```

19         color = 'black',
20         marker = '.')
21 ax.text(z + .1, pdfz,
22         s = '$z = {:.2f}'.format(z) )
23
24 # Create fill to left
25 x_vals = np.linspace(-3,z,100)
26 ax.fill_between(x_vals,
27                 np.zeros(100),
28                 stats.norm.pdf(x_vals),
29                 color = 'gray',
30                 alpha = .2)
31
32 # Area/cumulative density text
33 ax.text(0, pdfz/2,
34         s = "{:.2f}".format(cdfz),
35         size = 15,
36         ha = 'center')
37
38 # Change axes styling
39 ax.spines['bottom'].set_position('zero')
40 ax.spines['top'].set_visible(False)
41 ax.spines['right'].set_visible(False)

```

[norm-pdf.py](#)

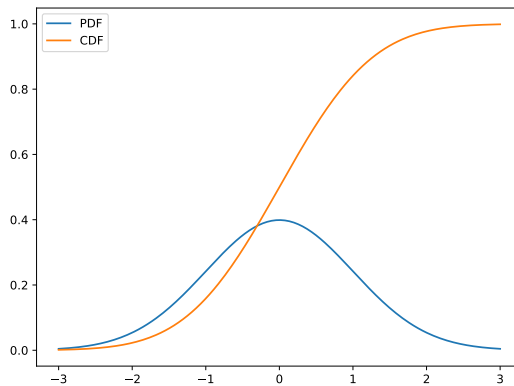


Still, it leaves the reader to rely on their eyeballing abilities to imagine how that area might change if the z -score changed. The graph itself lacks information from the cumulative density function (CDF), used to calculate that our z -score at the 75%ile of values drawn from the standard normal distribution. If your reader might be interested in this kind of thought exercise, you should include more of this information in the plot. First, we might add this

information by simply plotting both the PDF and CDF together. Eyeballing is still necessary to imagine how much rarer a z -score of 0.7 is, but at least with the CDF included, we can be a little more precise.

```
1 fig, ax = plt.figure(), plt.axes()
2 x = np.linspace(-3,3,200)
3 pdf_y = stats.norm.pdf(x)
4 cdf_y = stats.norm.cdf(x)
5 ax.plot(x,pdf_y, label = 'PDF')
6 ax.plot(x,cdf_y, label = 'CDF')
7 ax.legend()
```

[cdf-pdf.py](#)



Still, the plot above isn't very good. Here, more ticks or a grid would be helpful for tracing out what the CDF value is for a particular z -score. But apart from that, you might also see that the orange CDF dwarfs the blue PDF. While not terribly extreme, these functions cover different enough y values that having a shared y -axis is questionable, because the point isn't to draw attention to this difference. One fix for this is to create a second y -axis on the right. Knaflitz [2015](#) advises against a secondary y -axis. Dual axis charts aren't as immediately readable, so do be judicious and take extra care to make it clear which plot corresponds to which y -axis.

7.1.1 Using `twinx()` and `twiny()`

If we want a second y -axis, or a dual y -axis chart, we can start by creating a plot as usual, creating figure and axes objects `fig`, `ax`, and then create one more axes object with `ax.twinx()`. Give that a

name, `ax2` is what I use below, and the basics are all the same from there. A dual y -axis chart is created with `twinx()` because it is the x -axis that is shared and the y -axes are independent.

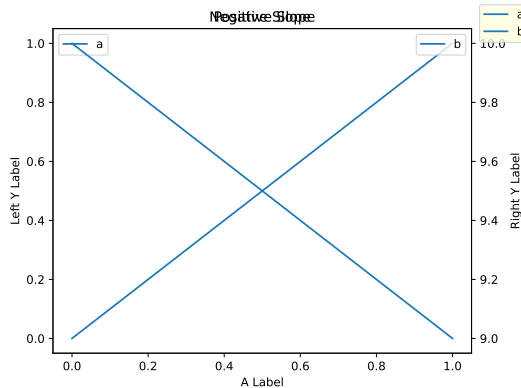
Let's take a brief detour from our normal distribution plots to illustrate some of the basics. You'll notice a few problems with the following plot.

```

1 fig, ax = plt.figure(), plt.axes()
2 ax2 = ax.twinx()
3
4 x = np.linspace(0,1,2)
5 ax.plot(x, x, label = 'a')
6 ax2.plot(x, 10-x, label = 'b')
7
8 ax.set_xlabel("A Label")
9
10 # This does nothing
11 ax2.set_xlabel("Label Attempt")
12
13 ax.set_ylabel("Left Y Label")
14 ax2.set_ylabel("Right Y Label")
15
16 ax.set_title("Positive Slope")
17 ax2.set_title("Negative Slope")
18
19 ax.legend()
20 ax2.legend()
21 fig.legend(facecolor = 'lightyellow')

```

`dual-bad.py`



From the second `plot()` call, everything starts to go downhill.

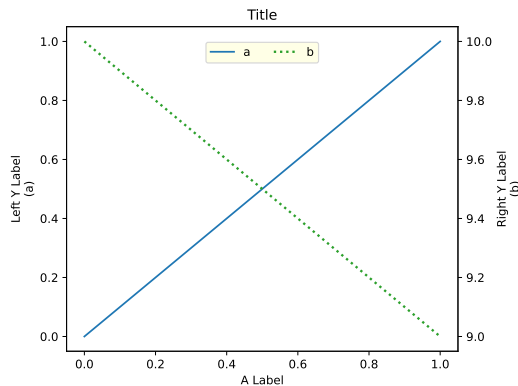
1. The plotted lines are the same color.

2. `set_xlabel()` does nothing for the x -axis-sharing twin axes.
3. The titles overlap.
4. `legend()` fails as an *axes* method. The figure legend isn't placed well.
5. It's not clear what line plot corresponds to what axis.

To fix the color issue, we must explicitly pass color values. The fixes for the second and third items are simple. Just use the original axes object for titling and labeling the shared axis. For the fourth, legend issue, we must use `legend()` as a figure method and explicitly pass a `loc` value. To clarify what line plot corresponds to what y -axis, we can tell the reader with our y -axis labels. This isn't a great solution, but it's where we'll start for the most basic fix. To match a line to its axis, we have too many steps to follow: match the plot to its label with the legend and then match the label to its axis.

```

1 fig, ax = plt.figure(), plt.axes()
2 ax2 = ax.twinx()
3
4 x = np.linspace(0,1,2)
5 ax.plot(x, x, label = 'a')
6 ax2.plot(x, 10-x,
7           label = 'b',
8           color = 'C2',
9           linestyle = 'dotted',
10          linewidth = 2)
11
12 ax.set_title("Title")
13
14 ax.set_xlabel("A Label")
15 ax.set_ylabel("Left Y Label\n(a)")
16 ax2.set_ylabel("Right Y Label\n(b)")
17
18 fig.legend(facecolor = 'lightyellow',
19           bbox_to_anchor = (.5,.92),
20           ncol = 2,
21           loc = 'center',
22           bbox_transform = ax.transAxes)
```

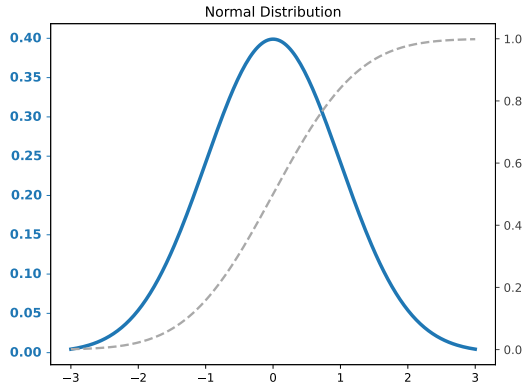


Returning to the normal distribution, we'll try to do a better job of making it more visually apparent what pieces of the plot belong to what y -axis.

```

1 fig, ax = plt.figure(), plt.axes()
2 ax2 = ax.twinx()
3 x = np.linspace(-3,3,200)
4 pdf_y = stats.norm.pdf(x)
5 cdf_y = stats.norm.cdf(x)
6
7 # Plot Curves
8 ax2.plot(x, cdf_y,
9         color = 'darkgray',
10        linestyle = 'dashed',
11        linewidth = 2)
12 ax.plot(x, pdf_y,
13        linewidth = 3)
14
15 ax.set_title("Normal Distribution")
16
17 # Change Ticks
18 # Use LaTeX \mathbf to make ticks bold
19 bolded_ticks = [r'$\mathbf{\text{"{:}.2f}}".format(x)+r"$}$"
20                 for x in ax.get_yticks()]
21 ax.set_yticklabels(bolded_ticks)
22 ax.tick_params(axis = 'y',
23               colors = 'C0',
24               labelsiz = 11)
25 ax2.tick_params(axis = 'y',
26               colors = (.25,.25,.25)) # darker gray

```



Here, the CDF plot and the secondary axis serve as a kind of footnote to the main point in the CDF.

Adding the cumulative distribution function helps, but that S-curve adds visual noise someone familiar with PDFs and CDFs might be better off without. One solution might be to add a second x -axis which annotates the chart with the CDF value at each point on the first x -axis.

```

1 fig, ax = plt.figure(), plt.axes()
2 ax2 = ax.twinx()
3
4 # Plot PDF
5 x = np.linspace(-3,3,200)
6 y = stats.norm.pdf(x)
7 ax.plot(x,y)
8
9 # Set x ticks for bottom x-axis
10 xticks = np.linspace(-2,2,6)
11 ax.set_xticks(xticks)
12
13 # Get corresponding CDF values for each tick
14 labels2 = list()
15 for tick in xticks:
16     cumulative = stats.norm.cdf(tick)
17     labels2.append(round(cumulative,2))
18
19 # Add ticks to top x-axis
20 ax2.set_xticks(xticks)
21 ax2.set_xticklabels(labels2, color = 'red')
22
23 # Clear y ticks
24 ax.set_yticks([])
25
26 # Set Limits
27 xlims = -3,3

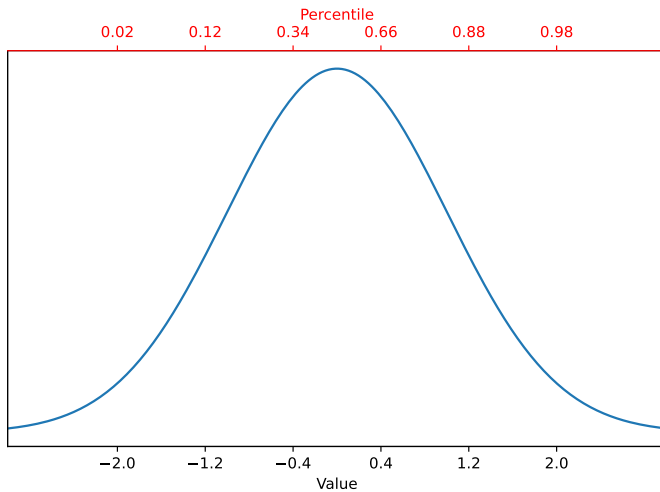
```

```

28 ax2.set_xlim(xlims)
29 ax.set_xlim(xlims)
30
31 # Label and change color
32 ax.set_xlabel("Value")
33 ax2.set_xlabel("Percentile", color = 'red')
34 ax2.spines['top'].set_color('red')
35 ax2.tick_params(axis='x', colors = 'red')

```

dual-norm-b.py



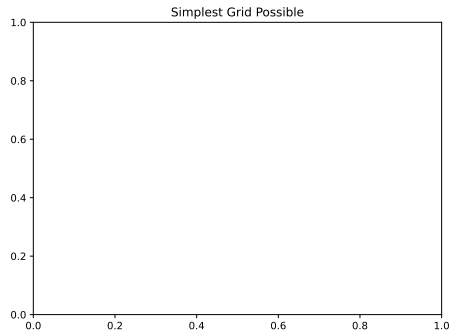
7.2 Multiple Plots

We can add several subplots to a figure in several different ways. We'll go over using `plt.subplots` and `fig.add_subplot`. `plt.subplots` is also useful as a shortcut, as `fig, ax = plt.figure(), plt.axes()` can be replaced with `fig, ax = plt.subplots()` for any figure with just one subplot (i.e. in every previous instance of `fig, ax` in this book.) as the default is a 1×1 grid of a single plot.

7.2.1 Using `subplots`

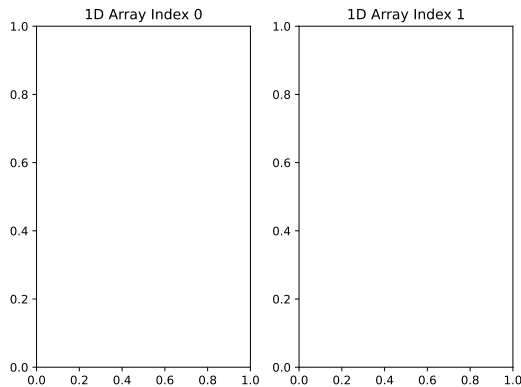
`plt.subplots` creates a figure *and* axes object(s). The first two arguments are `rows` and `ncols` for the number of rows and columns in the resulting plot grid. If the grid is not 1×1 , then you will have multiple axes objects in an array. Let's have a look.

```
1 fig, ax = plt.subplots()
2 ax.set_title("Simplest Grid Possible")
```

[trivial-sub.py](#)

Now, let's make non-trivial grids. Here, `ax` is a 1D array.

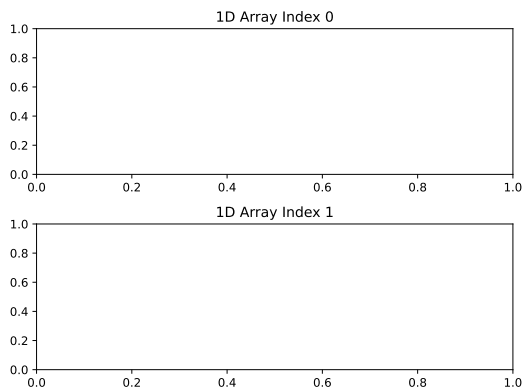
```
1 fig, ax = plt.subplots(1,2)
2 ax[0].set_title("1D Array Index 0")
3 ax[1].set_title("1D Array Index 1")
4 plt.tight_layout()
```

[subplots-1d.py](#)

Below, `ax` is again a 1D array.

```
1 fig, ax = plt.subplots(2,1)
2 ax[0].set_title("1D Array Index 0")
3 ax[1].set_title("1D Array Index 1")
4 plt.tight_layout()
```

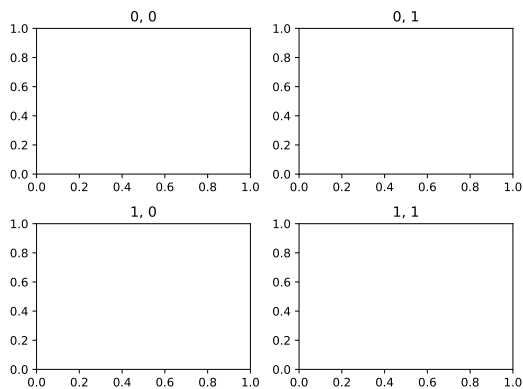
[subplots-1d-vert.py](#)



Next, with multiple rows and columns, `ax` is a 2D array.

```
1 fig, ax = plt.subplots(2,2)
2 ax[0][0].set_title("0, 0")
3 ax[0][1].set_title("0, 1")
4 ax[1][0].set_title("1, 0")
5 ax[1][1].set_title("1, 1")
6 plt.tight_layout()
```

[subplots-2d.py](#)



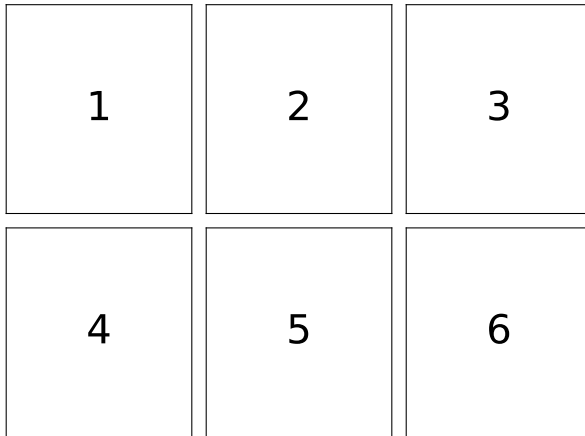
The `ax` object is made as simple as possible based on the `squeeze` parameter, where the default behavior is `squeeze = True` so that unnecessary dimensions are squeezed out of the array. By toggling `squeeze = False`, `ax` will always be made a 2D array. Setting this parameter to be false can be useful when you need to write more flexible code that can accommodate subplots of different dimensions.

7.2.2 Using `add_subplot`

You can avoid indexing an axes array by using the figure method `add_subplot`. The method creates an axes instance and requires specifying the subplot grid's dimensions and then the index or order within that grid. Subplots are not ordered by their row and column numbers, but by a single number. The numbering starts at 1 and increases moving to the right across the first row of graphs, and then proceeds to continue to the next row, again increases from left to right, and on and on. This is demonstrated below.

```
1 fig = plt.figure()
2 for i in range(1,7):
3     ax = fig.add_subplot(2,3,i)
4     ax.text(0.5, 0.5,
5             s = str(i),
6             ha = 'center',
7             va = 'center',
8             fontsize = 30)
9     ax.set_yticks([])
10    ax.set_xticks([])
11 fig.tight_layout()
```

[add-subplot.py](#)



The index value can also be a tuple.

7.2.3 Figure Annotations and Legends

In this subsection, we concern ourselves with customizing the entire figure. Each subplot can be customized just as you might usually

customize a single plot. For a figure object `fig`, the axes objects can be accessed by iterating over `fig.axes`, so that all axis limits can be changed in one loop. Figure customizations might include the spacing between plots, standardization of axes, and titling.

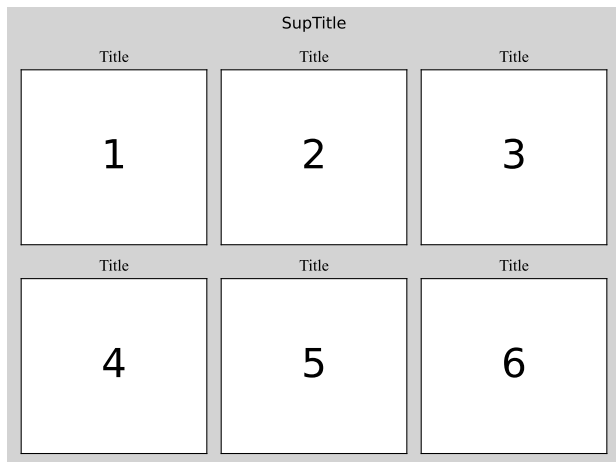
First, the figure method `suptitle()` is useful in creating a title that applies to the entire figure.

```

1 fig = plt.figure(facecolor = 'lightgray')
2
3 for i in range(1,7):
4     ax = fig.add_subplot(2,3,i)
5     ax.text(0.5, 0.5,
6             s = str(i),
7             ha = 'center',
8             va = 'center',
9             fontsize = 30)
10    ax.set_yticks([])
11    ax.set_xticks([])
12    ax.set_title("Title",
13                fontsize = 12,
14                fontname = 'Times New Roman')
15
16 fig.suptitle('SupTitle')
17 fig.tight_layout(rect = (0,0,1,1)) # no change

```

[suptitle.py](#)



Sometimes a `suptitle` is cut off when saving the figure. This can be solved by changing the dimensions in `tight_layout()`. Set the `rect` argument to a 4-tuple, like `(0,0,1,.95)`. This modifies the space

dedicated to the subplots, and the last value adjusts the vertical upper limit.

You can also draw lines between two different subplots with `ConnectionPatch`, a kind of *patch*. Patches will arise again in Part III, but for now `ConnectionPatch` is simply a tool for drawing a line between points on two different axes. These points are specified by parameters `xyA` and `xyB`. We specify the coordinate systems using `coordsA` and `coordsB`, making use of what we learned about transforms in Section 4.2 to specify our given coordinates are data coordinates. Then we use the `arrowstyle` parameter to create a line with arrows on both ends and the `shrinkA` and `shrinkB` parameters control how much the line will fall short of, or shrink away from the referenced point.

This code also makes use of the `transform` parameter to specify that the passed coordinates are data coordinates. See Section 4.2 for a review of transformations and other coordinate systems.

```

1 fig = plt.figure(figsize = (7,6))
2
3 # Generate random data
4 n = 100
5 x = np.random.normal(size = n)
6 y = np.random.normal(size = n)
7 # z is determined by x except for one outlier
8 z = np.concatenate([np.array([4]), 1- x[1:**2])
9
10 # Add x,y scatter plot
11 ax12 = fig.add_subplot(2,2,(1,2))
12 ax12.scatter(x,y, alpha = 0.5)
13
14 # Add x,z scatter plot
15 ax3 = fig.add_subplot(2,2,3)
16 ax3.scatter(x,z, alpha = 0.5)
17
18 # Add y,z scatter plot
19 ax4 = fig.add_subplot(2,2,4)
20 ax4.scatter(y,z, alpha = 0.5)
21
22 # Draw lines connecting the outlier as it appears in
   each scatter plot
23 con = ConnectionPatch(
24     xyA = (x[0], y[0]),
25     coordsA = ax12.transData,
26     xyB = (x[0], z[0]),
27     coordsB = ax3.transData,
28     arrowstyle = "<->",
29     shrinkA = 2,
30     shrinkB = 0)
31 fig.add_artist(con)

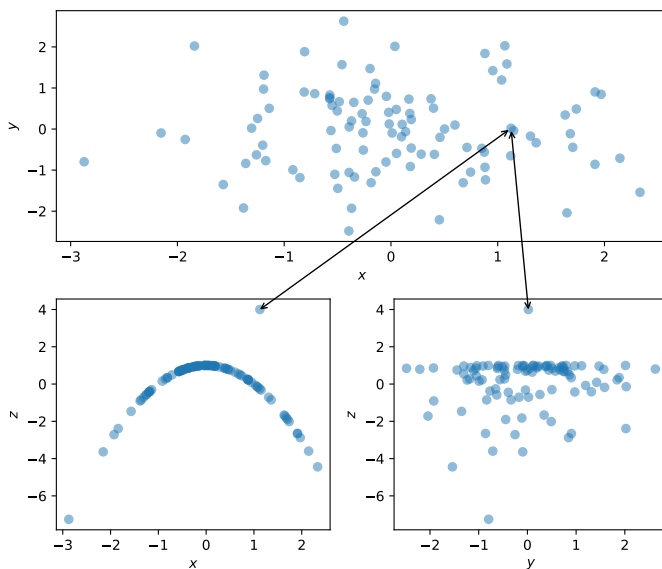
```

```

32
33 con = ConnectionPatch(
34     xyA = (x[0],y[0]),
35     coordsA = ax12.transData,
36     xyB = (y[0], z[0] ),
37     coordsB = ax4.transData,
38     arrowstyle = "<->",
39     shrinkA = 2,
40     shrinkB = 0)
41 fig.add_artist(con)
42
43 ax12.set_xlabel("$x$")
44 ax12.set_ylabel("$y$")
45 ax3.set_ylabel("$z$")
46 ax3.set_xlabel("$x$")
47 ax4.set_ylabel("$z$")
48 ax4.set_xlabel("$y$")
49 plt.tight_layout()

```

[connect-path.py](#)



7.3 GridSpec

For irregular plot grids, `GridSpec` is your friend. You can specify a grid with some number of rows and columns and spacing between

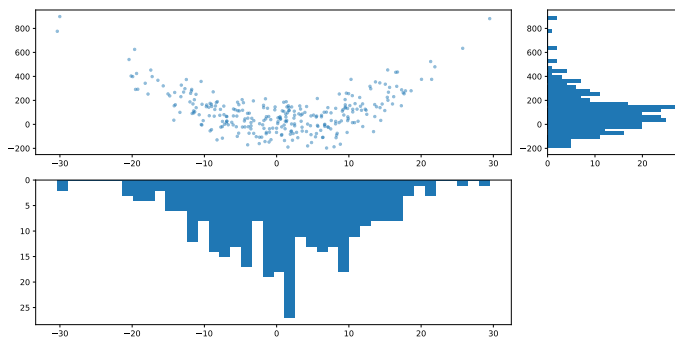
them. For example, `grid = plt.GridSpec(2, 3, wspace = 1, hspace = 0.3)`. Then, you can specify subplot locations using the typical slicing syntax. For example, `plt.subplot(grid[0,0])`. Or you can create an axis object for a subplot with `ax = fig.add_subplot(grid[0,0])`.

```

1 fig = plt.figure(figsize=(12,6))
2 spec = gridspec.GridSpec(ncols=4,
3                           nrows=2,
4                           figure=fig)
5 x = np.random.normal(0, 10, size = 300)
6 y = x**2 + np.random.normal(0, 100, size = 300)
7
8 ax1 = fig.add_subplot(spec[0, 0:3])
9 ax1.plot(x, y,
10          linestyle='None',
11          marker='.',
12          alpha=0.5)
13
14 ax2 = fig.add_subplot(spec[0, 3:4], sharey = ax1)
15 ax2.hist(y, orientation='horizontal', bins=40)
16
17 ax3 = fig.add_subplot(spec[1, 0:3], sharex = ax1)
18 ax3.hist(x, bins = 40)
19 ax3.invert_yaxis()
20 plt.tight_layout()

```

`gridspec.py`



Chapter 8

Style Configuration

This is a brief chapter that might provide a sigh of relief. So many of the parameters we have tweaked so far, sometimes laboriously, can be altered in one go with `plt.style.use`. Try out the [many style sheets already available](#).¹ You may also define your own or simply change certain parameters directly in your code to apply that styling for your entire session.

8.1 rcParams

Change the matplotlib parameters directly by updating the dictionary-like variable `mpl.rcParams`. A full list of the available parameters can be found in the [documentation](#) or you may simply print `mpl.rcParams` to inspect it directly. Note the above line is `mpl.rcParams` and not `plt.rcParams`, because we are not working within the pyplot submodule. Accordingly, you'll have to run `import matplotlib as mpl` first. Because this is like a Python dictionary, you can adjust the settings by simply updating the dictionary value, `mpl.rcParams['axes.grid'] = True` for example.

Working with `rcParams`, directly or through a custom style sheet, provides value that compounds as you add more and more plots to your code. Consider the two programs below. Without `rcParams`, we update each plot.

```
1 x = np.linspace(0,1,2)
2
```

¹https://matplotlib.org/stable/gallery/style_sheets/style_sheets_reference.html#sphx-glr-gallery-style-sheets-style-sheets-reference-py

```

3 fig1, ax = plt.figure(), plt.axes()
4 ax.plot(x, x)
5 ax.grid(True)
6 ax.spines['top'].set_visible(False)
7 ax.spines['right'].set_visible(False)
8 plt.show()
9
10 fig2, ax = plt.figure(), plt.axes()
11 ax.plot(x, 1 - x)
12 ax.grid(True)
13 ax.spines['top'].set_visible(False)
14 ax.spines['right'].set_visible(False)
15 plt.show()

```

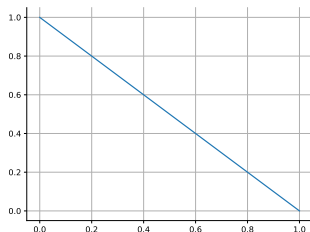
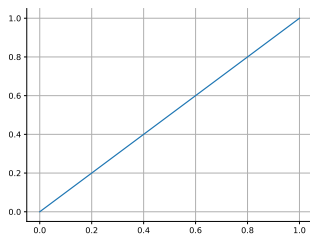
[style-manual.py](#)

Now let's update rcParams just once for a standard style.

```

1 # Use rcParams
2 mpl.rcParams['axes.grid'] = True
3 mpl.rcParams['axes.spines.top'] = False
4 mpl.rcParams['axes.spines.right'] = False
5
6 x = np.linspace(0,1,2)
7 plt.plot(x,x)
8 plt.savefig("style1.pdf")
9 plt.show()
10
11 plt.plot(x, 1-x)
12 plt.savefig("style2.pdf")
13 plt.show()

```

[style-rc.py](#)

This is a significant step, nearing us to a dramatic close of Part I. In the second program, we not only save on redundant code, we also revert back to pyplot functions. The object-oriented approach was useful in offering greater customization. But, at least in this case, that's a ladder we can kick away now that we've climbed it to the top.

8.2 Defining Your Own Style

Once you understand rcParams, you can define your own style for repeated use. Schwabish 2021 counsels organizations to adopt a data visualization style guide. Practically, that also means matplotlib-using organizations should choose or create a standard matplotlib style.

To define your own style, create a file with the `.mplstyle` extension, specifying a value for the various rcParams you wish to customize. Note that a colon separates the key and the default like in a dictionary, but that we do not separate key-value pairs by commas and each pair is on a separate line. Further, none of these values are formatted as strings, even though you would, for example, use a string `'Times'` when updating the font from rcParams dictionary. You only need to specify values you wish to change relative to the default.

```
1 axes.spines.left : True
2 axes.spines.right : False
3 axes.spines.bottom : True
4 axes.spines.top : False
5 xtick.labelsize : large
6 font.family : Times
```

`tiny-style.mplstyle`

After saving the above as a file `tiny_style.mplstyle` and placing it in the working directory, we can use our custom style with the program below. Note the colormap, among many other things, has not changed relative to the default because we did not alter that in the style file.

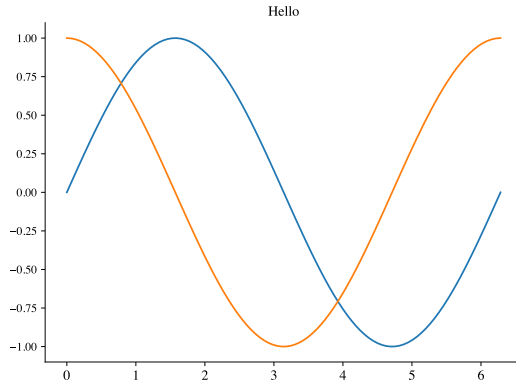
```
1 plt.style.use('../..//stylelib/tiny-style.mplstyle')
2
3 fig, ax = plt.figure(), plt.axes()
4 x = np.linspace(0, 2*np.pi, 100)
5 plt.plot(x, np.sin(x))
6 plt.plot(x, np.cos(x))
7 plt.title('Hello')
```

```

8
9 # Inspect the updated rcParams
10 #print(mpl.rcParams)

```

[tiny-style-ex.py](#)



You can also just save direct modifications to the rcParams dictionary and run that before plotting.

```

1 import matplotlib as mpl
2 mpl.rcParams['axes.spines.left'] = True
3 mpl.rcParams['axes.spines.right'] = False
4 mpl.rcParams['axes.spines.bottom'] = True
5 mpl.rcParams['axes.spines.top'] = False
6 mpl.rcParams['axes.titlesize'] = 25
7 mpl.rcParams['xtick.labelsize'] = 'large'
8 mpl.rcParams['font.family'] = 'Times'

```

[style-changes.py](#)

Then add this code to ahead of creating your plot. I saved the above as `style_changes.py` and below I use the Jupyter `%run` magic command to run `style_changes.py` without having to copy and paste.

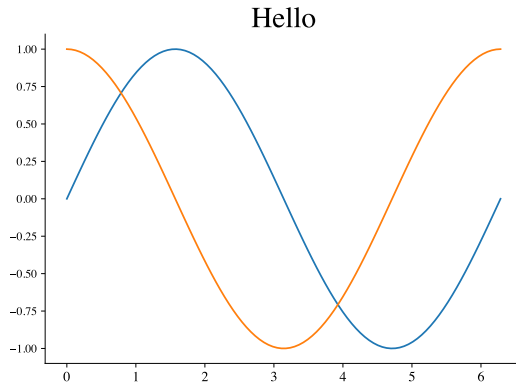
```

1 plt.style.use('default')
2 %run ../../python/style-changes.py
3
4 x = np.linspace(0,2*np.pi,100)
5 plt.plot(x, np.sin(x))
6 plt.plot(x, np.cos(x))
7 plt.title('Hello')

```

[py-styled.py](#)

The result is



8.2.1 Temporary Configurations

With some creativity, you can also avoid modifying rcParams by defining a function or writing a Python file that makes certain standardized plot modifications and then run that after you've created your figure and axes objects, using the IPython `%run` magic command.

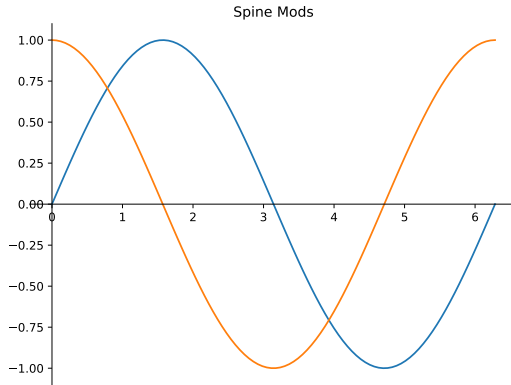
We save the following as `spine-mod.py` and then use it below to modify a plot.

```
1 import matplotlib.pyplot as plt
2
3 plt.gca().spines['left'].set_position('zero')
4 plt.gca().spines['bottom'].set_position('zero')
5
6 plt.gca().spines['top'].set_visible(False)
7 plt.gca().spines['right'].set_visible(False)
```

`spine-mod.py`

```
1 x = np.linspace(0,2*np.pi,100)
2 plt.plot(x, np.sin(x))
3 plt.plot(x, np.cos(x))
4 plt.title('Spine Mods')
5
6 %run ../../python/spine-mod.py
```

`spine-mod-ex.py`



In the plot program, we use pyplot functions instead of using the OOP approach. Note that even if we created an `ax` variable, we must still use `plt.gca()` instead of `ax` in the file, because there is no `ax` to reference in `spine_mod.py`². We can instead use `%run -i` to let the file access our global variables. Also, `%run -i` eliminates the need to import matplotlib again—we could delete the import statement from `spine-mod.py` and use `%run -i spine-mod.py`.

We save the following file as `spine-mod2.py` and can create the same plot with the program further below.

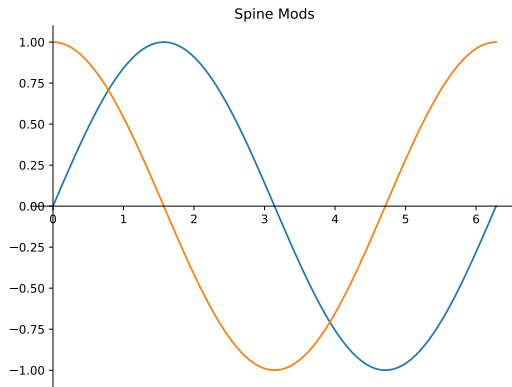
```
1 ax.spines['left'].set_position('zero')
2 ax.spines['bottom'].set_position('zero')
3 ax.spines['top'].set_visible(False)
4 ax.spines['right'].set_visible(False)
```

`spine-mod2.py`

```
1 x = np.linspace(0, 2*np.pi, 100)
2 fig, ax = plt.figure(), plt.axes()
3 ax.plot(x, np.sin(x))
4 ax.plot(x, np.cos(x))
5 ax.set_title('Spine Mods')
6 %run -i ../python/spine-mod2.py
```

`spine-mod2-ex.py`

²Variables outside the file are not in a shared *scope* because `%run` creates a new *namespace* for variables inside the file)



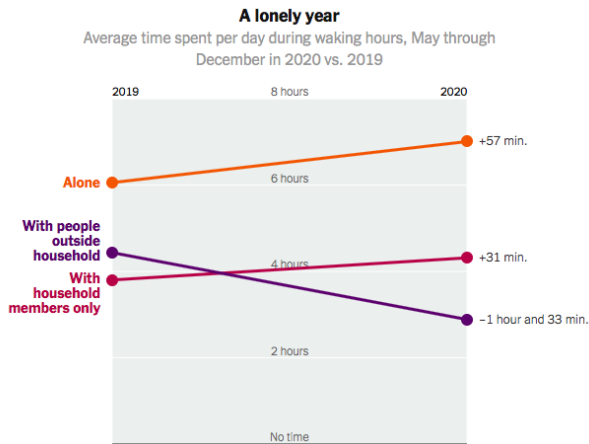
A further complication arises if our figure contains multiple subplots. In this case, we can access all the axes objects as an attribute of the figure object.

```
1 for ax in fig.axes:
2     ax.spines['left'].set_position('zero')
3     ax.spines['bottom'].set_position('zero')
4     ax.spines['top'].set_visible(False)
5     ax.spines['right'].set_visible(False)
```

`spine-mod3.py`

8.3 A Final Prose Example

In this section, we'll integrate much of what we've learned so far to create a line chart that, though simple, is far from the default. We'll imitate a chart from the New York Times article, *The Pandemic Changed How We Spent Our Time*, by Ben Casselman and Ella Koeze. Below is the original.



Imitating this graphic will take a lot of code, as demonstrated in Section 8.3.1. In Section 8.3.2, we invest in reconfiguring the style and creating functions that help reduce the tedium that would otherwise be required to make several plots of this style.

8.3.1 A First Go

The program below would be even longer if not for the use of dictionaries like `plot_style`, which pairs keyword arguments and specific values for the `plot()` method. This can be passed to `plot()` after being unpacked with the `**` operator.

```

1  fig = plt.figure(figsize = (7,6.5))
2  ax = plt.axes(facecolor = (.94, .94, .96))
3
4  ## Add data and annotations
5  plot_style = dict(marker = 'o',
6                    clip_on = False, # don't clip markers at axes
7                    linewidth = 3,
8                    markersize = 8)
9  right_text_style = dict(ha = 'left', fontsize = 11,
10                         fontname = 'Helvetica',
11                         color = (.3,.3,.3),
12                         va = 'center')
13  left_text_style = dict(ha = 'right',
14                        fontsize = 12,
15                        fontweight = 'bold',
16                        fontname = 'Helvetica')
17  x_vals = [2019,2020]
18  # alone
19  col = 'orange'

```

```

20 ax.plot(x_vals, [6,7],
21         color = col,
22         **plot_style)
23 ax.text(2019 - .02, 6, 'Alone',
24         color = col,
25         va = 'center',
26         **left_text_style)
27 ax.text(2020.03, 7, '+57 min.',
28         **right_text_style)
29
30 # within household
31 col = 'purple'
32 ax.plot(x_vals, [4.3,2.7],
33         color = col,
34         **plot_style)
35 ax.text(2019 - .02, 4.2, 'With people\noutside\
    nhousehold',
36         color = col,
37         va = 'bottom',
38         **left_text_style)
39 ax.text(2020.03, 2.7, '-1 hour and 33 min.',
40         **right_text_style)
41
42 # outside household
43 col = 'darkred'
44 ax.plot(x_vals, [3.9,4.4], color = col,
45         **plot_style)
46 ax.text(2019 - .02, 4, 'With\nhousehold\nmembers only',
47         color = col,
48         va = 'top',
49         **left_text_style)
50 ax.text(2020.03, 4.4, '+31 min.',
51         **right_text_style)
52
53
54 # Label Gridlines
55 text_style = dict(color = (.3,.3,.3),
56                   va = 'bottom',
57                   ha = 'center',
58                   fontname = 'Helvetica',
59                   fontsize = 11)
60
61 ax.text(2019.5, 0.01, "No Time", **text_style)
62 for y in [2,4,6,8]:
63     ax.text(2019.5, y+.04, "{} hours".format(y),
64             **text_style)
65
66 # Label 2020/2021 for x-axis
67 year_text_style = dict(color = (.3,.3,.3),
68                        va = 'bottom',
69                        fontname = 'Helvetica',
70                        fontsize = 12)

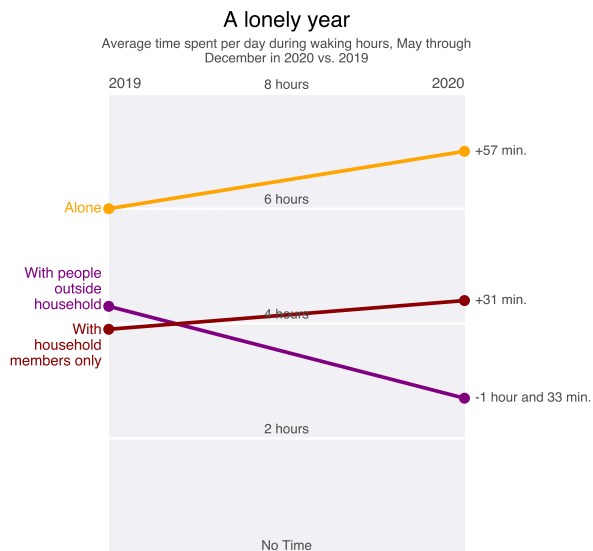
```

```

71 ax.text(2019, 8.05, '2019',
72         ha = 'left', **year_text_style)
73 ax.text(2020, 8.05, '2020',
74         ha = 'right', **year_text_style)
75
76 # set main title
77 ax.set_title("A lonely year", pad = 55, fontweight = '
    bold',
78              fontname = 'Helvetica',
79              fontsize = 18)
80
81 # set subtitle
82 s = 'Average time spent per day during waking hours, May
    through\nDecember in 2020 vs. 2019'
83 ax.text(2019.5, 8.5, s, **text_style)
84
85 # x axis
86 ax.set_xticks([])
87 ax.set_xlim(2019,2020)
88
89 # y axis
90 ax.yaxis.grid(True, color = 'white')
91 ax.set_yticks([0,2,4,6,8])
92 ax.set_yticklabels([])
93 ax.set_ylim([-0.02,8])
94 ax.yaxis.set_tick_params(length = 0, grid_linewidth = 2)
95
96 # spines
97 for i in ['top','left','right']:
98     ax.spines[i].set_visible(False)
99 ax.spines['bottom'].set_color('darkgray')
100 ax.spines['bottom'].set_linewidth(2)

```

nyt-rep1.py



8.3.2 Reconfigured, Refactored, and Reusable

```

1 axes.linewidth: 2
2 axes.facecolor: (.94, .94, .96)
3 axes.grid.axis: y
4 axes.grid: True
5 grid.color: white
6 grid.linewidth: 2
7 axes.spines.bottom: True
8 axes.spines.top: False
9 axes.spines.left: False
10 axes.spines.right: False
11 axes.edgecolor: darkgray
12 xtick.bottom: False
13 xtick.top: False
14 ytick.left: False
15 xtick.labeltop: True
16 xtick.labelbottom: False
17 ytick.labelleft: False
18 xtick.color: (.3,.3,.3)
19 text.color: (.3,.3,.3)
20 font.size: 12
21 lines.marker: o
22 lines.markersize: 8
23 lines.linewidth: 3
24 axes.titlesize: 18
25 axes.titleweight: bold

```

```
26 axes.formatter.useoffset: False
```

[nyt-helper.mplstyle](#)

```
1 def title_and_subtitle(title, subtitle = '', pad = 0.01,
2     fig = None, ax = None):
3     """Add a centered title and subtitle to a plot."""
4     if ax == None:
5         ax = plt.gca()
6     if fig == None:
7         fig = plt.gcf()
8     fig.canvas.draw()
9
10    top_of_figure = 1 # axes coords
11    # update if there are xticks on the top
12    tick0 = ax.get_xticklabels()[0]
13    top_of_ticklabels = tick0.get_window_extent().
14    transformed(ax.transAxes.inverted()).y1
15    top_of_figure = max([top_of_ticklabels, top_of_figure
16    ])
17
18    # Add subtitle
19    if subtitle:
20        subt = ax.text(0.5, top_of_figure + pad,
21            s = subtitle,
22            ha = 'center',
23            va = 'bottom',
24            size = '11',
25            fontname = 'Helvetica',
26            transform = ax.transAxes)
27        # update top of figure to top of the subtitle
28        top_of_figure = subt.get_window_extent().
29        transformed(ax.transAxes.inverted()).y1
30
31    # add title
32    ax.text(0.5, top_of_figure + pad,
33        s = title,
34        ha = 'center',
35        va = 'bottom',
36        size = '18',
37        fontname = 'Helvetica',
38        fontweight = 'bold',
39        transform = ax.transAxes,
40        color = 'black')
```

[nyt-helper-function.](#)

```
1 data = {2019: [6, 4.3, 3.9],
2         2020: [7, 2.7, 4.4]}
3 df = pd.DataFrame(data = data,
4     index = ['Alone',
5             'With people outside household']
6     ,
```



```
6 'With household members only']])
```

nyt-helper-data.py

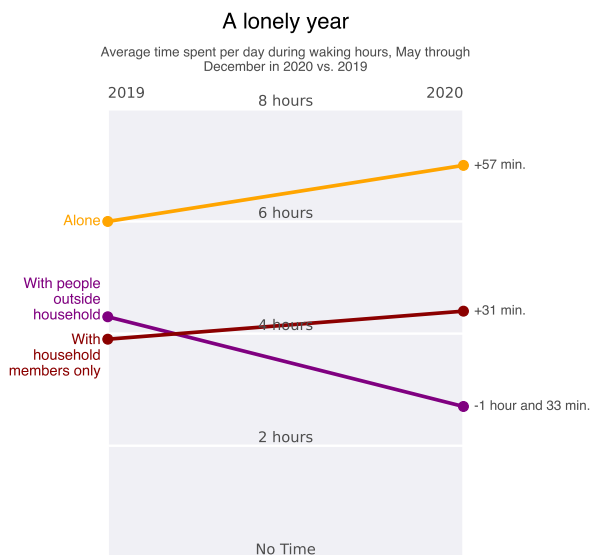
```
1 with plt.style.context("../stylelib/nyt-helper.
   mplstyle"):
2
3     fig = plt.figure(figsize = (7,6.5))
4     ax = plt.axes(facecolor = (.94, .94, .96))
5
6     # plot the data
7     colors = ['orange', 'purple', 'darkred']
8     df.T.plot(ax = ax, clip_on = False, color = colors)
9
10    # annotate lines
11    right_text_style = dict(ha = 'left',
12                           fontsize = 11,
13                           fontname = 'Helvetica',
14                           va = 'center')
15    left_text_style = dict(ha = 'right',
16                          fontsize = 12,
17                          fontweight = 'bold',
18                          fontname = 'Helvetica')
19
20    # alone
21    ax.text(2019 - .02, 6, 'Alone',
22           color = colors[0],
23           va = 'center',
24           **left_text_style)
25    ax.text(2020.03, 7, '+57 min.',
26           **right_text_style)
27
28    # Within household
29    ax.text(2019 - .02, 4.2, 'With people\noutside\
   nhousehold',
30           color = colors[1],
31           va = 'bottom',
32           **left_text_style)
33    ax.text(2020.03, 2.7, '-1 hour and 33 min.',
34           **right_text_style)
35
36    # Outside household
37    ax.text(2019 - .02, 4, 'With\nhousehold\nmembers
   only',
38           color = colors[2],
39           va = 'top',
40           **left_text_style)
41    ax.text(2020.03, 4.4, '+31 min.',
42           **right_text_style)
43
44
45    # Title and sub
46    t = 'A lonely year'
```

```

47 s = 'Average time spent per day during waking hours,
48     May through\nDecember in 2020 vs. 2019'
49
50 # Clean ticks etc
51 yticks = range(0,9,2)
52 ax.set_yticks(yticks)
53 ax.set_ylim(0,8)
54 # add custom labels for y ticks
55 for tick in yticks:
56     label = "{} hours".format(tick)
57     if tick == 0:
58         label = 'No Time' # custom label
59     ax.text(2019.5, tick + .01, label,
60            va = 'bottom',
61            ha = 'center')
62
63 # move x-tick labels to horizontally align
64 ax.legend().set_visible(False)
65 x_vals = [2019,2020]
66 ax.set_xticks(x_vals)
67 ax.set_xlim(x_vals)
68 xticks = ax.get_xticklabels()
69 xticks[0].set_ha('left')
70 xticks[-1].set_ha('right')

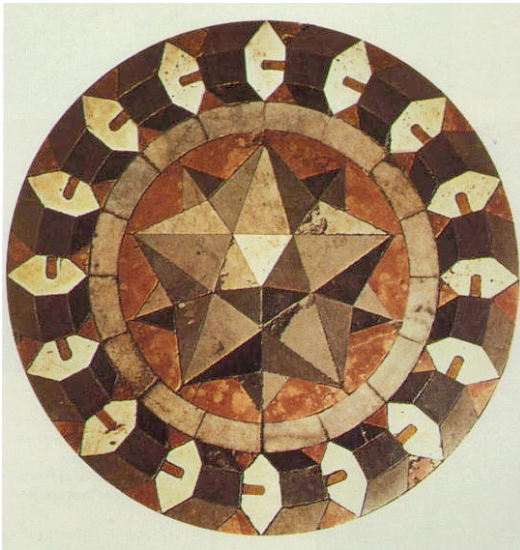
```

[nyt-refactor.py](#)



Part II

Mathematical Interlude



St Mark's Basilica, Floor mosaic by Paolo Uccello (Public Domain)

Chapter 9

Math

In Part III, we'll begin to treat Matplotlib more like a blank canvas. The complexity can evolve any number of ways, and one key complexity is the placement of items in a plot. Doing that well means understanding angles. So this math interlude guides us through trigonometry and some light linear algebra. Some pieces of this chapter are unnecessary. `plt.Circle()` can be used to create a circle without any knowledge of trigonometry. Instead, we plot circles the old-fashioned way. We create a lot of points that, when connected, form a circle.

Why bother? Indeed, your Python interpreter won't be impressed if you know trigonometry. We shouldn't bother in every case, but math can compensate for a lack of matplotlib knowledge. I'd rather know a lot of math and a little matplotlib than a little math and a lot of matplotlib. Math is durable knowledge, useful in non-plotting contexts. A deeper understanding is also what allows us to create the color gradient in Section 10.2, which can't be fashioned with a simple call to `plt.Circle()`.

9.1 Circles

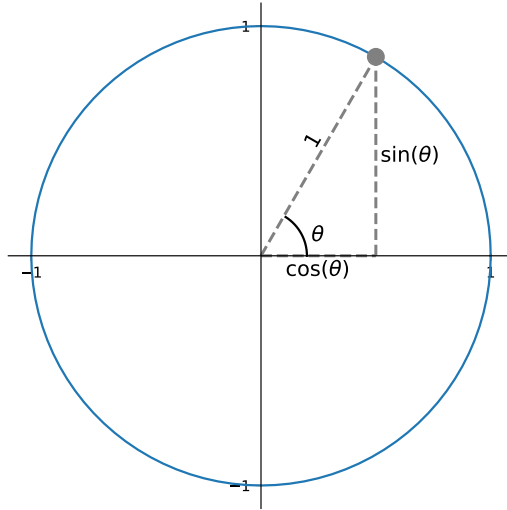
9.1.1 The Unit Circle

The unit circle is the gem of pre-calculus. Understanding it is useful for plotting circles or arcs by hand (though one can also use `Circle` or `Arc` objects). It tells us how to relate angles to a particular point in the xy -plane. For a point on the unit circle at angle θ from the

origin, we can find its coordinates as

$$(x, y) = (\cos(\theta), \sin(\theta)).$$

Tracing a line from the origin to the point on the circle, we can create a right triangle as shown below.



```

1 angles = np.linspace(0, 2*np.pi, 101)
2 x = np.cos(angles)
3 y = np.sin(angles)
4
5 fig, ax = plt.figure(figsize = (5,5)), plt.axes()
6 ax.set_aspect('equal')
7
8 # Make circle
9 ax.plot(x,y)
10
11 # Plot example right triangle
12 angle = np.pi/3
13
14 # make hypotenuse
15 ax.plot([0,np.cos(angle)], [0,np.sin(angle)],
16         linestyle = 'dashed', color = 'gray', linewidth =
17         2)#
18
19 # mark point on circle
20 ax.plot([np.cos(angle)], [np.sin(angle)],
21         marker = 'o', color = 'gray', markersize = 11)

```

```

22 # dashed lines for opposite and adjacent
23 ax.plot([0,np.cos(angle)], [0,0],
24         linestyle = 'dashed', color = 'gray', linewidth =
25         2)
26 ax.plot([np.cos(angle),np.cos(angle)], [0,np.sin(angle)
27         ],
28         linestyle = 'dashed', color = 'gray', linewidth =
29         2)
30 # Triangle side lengths
31 fontsize = 14
32 ax.text(0.5*np.cos(angle) - .02, 0.5*np.sin(angle)+.02,
33         '1', rotation = math.degrees(angle), ha = '
34         center', va = 'bottom', size = fontsize)
35 ax.text(0.5*np.cos(angle), -.02, r"$\cos(\theta)$",
36         rotation = 0, ha = 'center', va = 'top', size =
37         fontsize)
38 ax.text(np.cos(angle) + .02, 0.5*np.sin(angle), r"$\sin
39         (\theta)$",
40         rotation = 0, ha = 'left', va = 'center', size =
41         fontsize)
42 # make small arc and mark angle
43 x = np.cos(angles[angles<= angle])
44 y = np.sin(angles[angles<= angle])
45 ax.plot(0.2*x,0.2*y, color = 'black')
46 ax.text(0.2*np.cos(np.pi/10), 0.2*np.sin(np.pi/10),
47         r" $\theta$", size = 14)
48 # clean appearance
49 %run ../../python/spine-mod.py
50 ax.set_xticks([-1, 1])
51 ax.set_yticks([-1, 1])

```

unit-circle.py

We can plot a circle or an arc from θ_1 to θ_2 , by connecting the points $(\cos(\theta_1), \sin(\theta_1)), \dots, (\cos(\theta_2), \sin(\theta_2))$, where enough intermediate angles between θ_1 and θ_2 are included so the piecewise-linearity is smoothed out to give the appearance of a curve. In Subsection 9.1.2, we consider how to do the same, but for non-unit circles.

9.1.2 Non-unit Circles

The unit circle has a radius of one and it's centered at the origin. How do we obtain coordinates for other circles? There are two steps to change the radius and shift a circle off the origin.

1. **Change the radius.** Multiply the coordinates by the desired radius r .
2. **Shift the circle.** Add the desired horizontal and vertical shifts to the x and y coordinates, respectively.

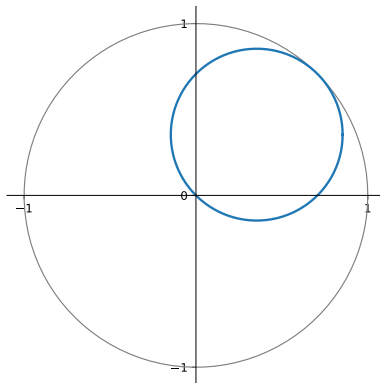
These are ordered because the radius multiplier should not be applied to the added shift term. Below, we shrink the unit circle and move it up and along the 45-degree line.

```

1 angles = np.linspace(0, 2*np.pi, 100)
2
3 fig, ax = plt.figure(), plt.axes()
4 ax.set_aspect('equal')
5
6 # Unit Circle
7 x = np.cos(angles)
8 y = np.sin(angles)
9 ax.plot(x, y, color = 'gray', linewidth = 1)
10
11 # Shifted
12 new_radius = 0.5
13 new_center = np.cos(np.pi/4)/2, np.sin(np.pi/4)/2
14 shift_x = new_radius*x + new_center[0]
15 shift_y = new_radius*y + new_center[1]
16 ax.plot(shift_x, shift_y, linewidth = 2)
17
18 %run ../../python/spine-mod.py
19
20 ax.set_xticks([-1, 1])
21 ax.set_yticks([-1, 0, 1])

```

[unit-circle-shift.py](#)



9.1.3 Rotations and Ellipses

Now we jump from trigonometry to linear algebra. Matrices can represent transformations, like rotations or stretching. Applied to each point in a circle, a rotation that stretches x and y coordinates differently creates an ellipse.

A rotation of angle θ can be represented as

$$\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

Stretching the x -dimension by a scalar r can be represented with

$$\begin{bmatrix} r & 0 \\ 0 & 1 \end{bmatrix},$$

and the y -dimension is stretched by

$$\begin{bmatrix} 1 & 0 \\ 0 & r \end{bmatrix}.$$

Each of these matrices is applied point by point by left multiplying that point (as a 2×1 column vector) by the transformation matrix,

$$\begin{pmatrix} \tilde{x} \\ \tilde{y} \end{pmatrix} = T \begin{pmatrix} x \\ y \end{pmatrix}.$$

Below we take a circle and shrink it horizontally, stretch it vertically, and then rotate it. The x values are multiplied by $\frac{1}{2}$, the y values are multiplied by 2, and the angle of rotation is 45 degrees ($\frac{\pi}{4}$ radians). The transformation is constructed below.

```

1 theta = np.pi / 4
2 rotation_matrix = np.matrix([[np.cos(theta), -np.sin(
   theta)], [np.sin(theta), np.cos(theta)]])
3
4 x_scale = 0.5
5 x_stretch = np.matrix([[x_scale, 0], [0, 1]])
6
7 y_scale = 2
8 y_stretch = np.matrix([[1, 0], [0, y_scale]])
9
10 transformation = rotation_matrix * y_stretch * x_stretch

```

[tform-matrix.py](#)

Below we plot a unit circle and then apply the transformation to create an ellipse.

```

1 # Create a circle of points
2 angles = np.linspace(0, 2*np.pi, 100)
3 x_vals = np.cos(angles)
4 y_vals = np.sin(angles)
5
6 # Begin plot
7 fig, ax = plt.subplots(1,2)
8
9 # simplify axes names
10 ax0, ax1 = ax[0], ax[1]
11
12 # Plot a circle
13 ax0.plot(x_vals, y_vals)
14
15 # Mark the y and x directions/axes
16 # vertical axis
17 height = 1.2
18 p1 = np.array([0,-height])
19 p2 = np.array([0,height])
20 points = [p1,p2]
21 x_vertical = [p[0] for p in points]
22 y_vertical = [p[1] for p in points]
23 ax0.plot(x_vertical, y_vertical)
24
25 # horizontal axis
26 width = height
27 p1 = np.array([height,0])
28 p2 = np.array([-height,0])
29 points = [p1,p2]
30 x_horiz = [p[0] for p in points]
31 y_horiz = [p[1] for p in points]
32 ax0.plot(x_horiz, y_horiz)
33
34 # Make Ellipse
35 new_points = [transformation * np.matrix(p).T for p in
36               zip(x_vals,y_vals)]
37
38 new_x = [np.array(x).flatten()[0] for x in new_points]
39 new_y = [np.array(x).flatten()[1] for x in new_points]
40
41 # new vertical axis
42 new_vertical = [transformation * np.matrix(p).T for p in
43                zip(x_vertical, y_vertical)]
44
45 new_x_vertical = [np.array(x).flatten()[0] for x in
46                  new_vertical]
47 new_y_vertical = [np.array(x).flatten()[1] for x in
48                  new_vertical]
49
50 # new horizontal axis
51 new_horiz = [transformation * np.matrix(p).T for p in
52              zip(x_horiz, y_horiz)]
53 new_x_horiz = [np.array(x).flatten()[0] for x in
54               new_horiz]

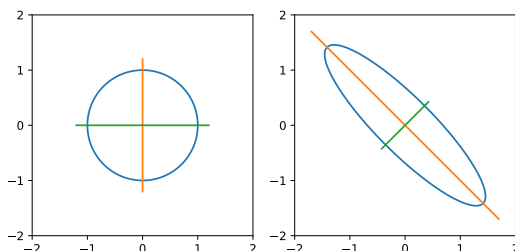
```

```

48 new_y_horiz = [np.array(x).flatten()[1] for x in
    new_horiz]
49
50 # Plot ellipse etc
51 ax1.plot(new_x, new_y)
52 ax1.plot(new_x_vertical, new_y_vertical)
53 ax1.plot(new_x_horiz, new_y_horiz)
54
55 # Change axes appearance
56 args = -2,2
57 for ax_ in ax0, ax1:
58     ax_.set_xlim(args)
59     ax_.set_ylim(args)
60     ax_.set_xticks(np.linspace(*args,5))
61     ax_.set_yticks(np.linspace(*args,5))
62 ax0.set_aspect('equal')
63 ax1.set_aspect('equal')

```

[ellipse-tform.py](#)



9.2 Right Triangles

Right triangles are important to understand not for plotting right triangles necessarily, but for understanding the angle between any two points. The line segment connecting two points forms the hypotenuse of a right triangle, just as was seen in the unit circle.

For any angle θ in a right triangle that is not the right angle itself, we can speak of the sides opposite or adjacent to the angle. The side opposite is the side directly across from the angle. The side opposite the right angle is the hypotenuse (of length c in Pythagoras' Theorem). The SOHCAHTOA mnemonic helps us understand

how side lengths are related to the angles. More clearly written as SOH-CAH-TOA, as it stands for Sine Opposite Hypotenuse Cosine Adjacent Hypotenuse Tangent Opposite Adjacent and means

$$\begin{aligned}\sin \theta &= \frac{\text{opposite}}{\text{hypotenuse}} \\ \cos \theta &= \frac{\text{adjacent}}{\text{hypotenuse}} \\ \tan \theta &= \frac{\text{opposite}}{\text{adjacent}},\end{aligned}$$

where θ is some angle of a triangle in radians and opposite, adjacent, and hypotenuse refer to the lengths of these sides.

By understanding these functions and their inverses, we can recover the angles in a plot. These functions are available from the `math` module as `sin()`, `cos()`, and `tan()`. Their inverses are `asin()`, `acos()`, and `atan()` for arcsin, arccos, and arctan.

`math.atan()` is the most useful. Take two points and the slope m of the line connecting them. Then $\arctan(m) = \theta$ is angle between those points, in radians.

```

1 fig, ax = plt.figure(), plt.axes()
2
3 a = (1,2)
4 b = (7,6)
5
6 # rise over run
7 slope = (a[1] - b[1]) / (a[0] - b[0])
8 angle = math.atan(slope) # radians
9 degrees = math.degrees(angle)
10
11 top_angle = math
12
13 ## add angle semi-circle
14 x = np.linspace(0, angle, 100)
15 ax.plot(0.5 * np.cos(x) + a[0],
16         0.5 * np.sin(x) + a[1],
17         color = 'black')
18 ax.text(0.5*np.cos(angle/2) + 1.1, 0.5*np.sin(angle/2) +
19         2,
20         s = r"${:.1f}".format(degrees) + r"^\circ$")
21
22 # top slope measured relative to a 90-deg rotation
23 top_slope = (b[0]-a[0])/(b[1]-a[1])
24 top_angle = math.atan(top_slope)
25 x = np.linspace(1.5*np.pi, 1.5*np.pi - top_angle, 100)
26 ax.plot(0.5*np.cos(x) + b[0],
27         0.5*np.sin(x) + b[1],

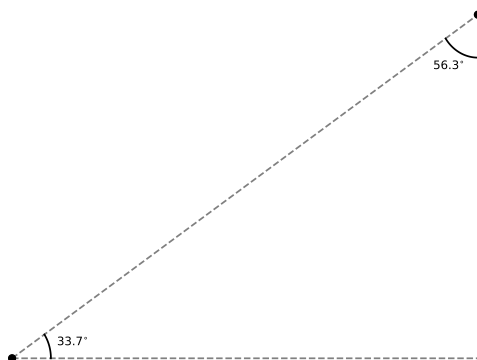
```

```

27     color = 'black')
28 label_angle = 1.5*np.pi - top_angle/2
29 ax.text(0.5*np.cos(label_angle) + b[0] - 0.13, 0.5*np.
        sin(label_angle) + b[1] - 0.2,
30         s = r"${:.1f}".format(math.degrees(top_angle)) +
        r"~{\circ}$",
31         ha = 'center')
32
33
34 # points on left and top
35 ax.plot([a[0], b[0]], [a[1], b[1]], linestyle = '',
        marker = 'o', color = 'black')
36
37 # make a right triangle
38 ax.plot([a[0], b[0]], [a[1], b[1]], linestyle = 'dashed'
        , marker = 'o', color = 'gray', zorder = -1)
39 ax.plot([a[0], b[0]], [a[1], a[1]], linestyle = 'dashed'
        , color = 'gray', zorder = -1)
40 ax.plot([b[0], b[0]], [a[1], b[1]], linestyle = 'dashed'
        , color = 'gray', zorder = -1)
41 ax.axis('off')

```

r-triangle.py



Chapter 10

Applications

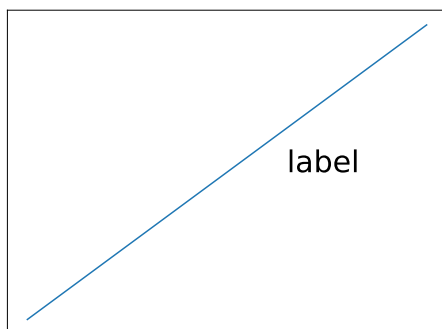
10.1 Sloping Text

Suppose you'd like to label a line.

For a sloped line, you might rather the text sit parallel to the line instead of suffering the below.

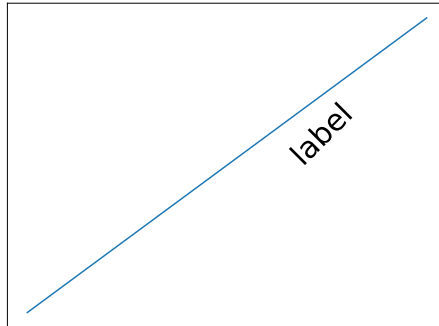
```
1 plt.plot([0,1], [0,1])
2 plt.text(0.65, 0.5,
3         s = 'label',
4         size = 30)
5
6 ax = plt.gca()
7 # Cosmetics
8 ax.grid(False)
9 ax.set_xticks([])
10 ax.set_yticks([])
```

[no-slope.py](#)



The `rotation` argument can help if you know the right angle in degrees. Here the angle is 45 degrees or $\frac{\pi}{4}$ radians. So we modify the second line to be `plt.text(0.65, 0.5, 'label', size = 30, rotation = 45)`.

But this doesn't do what we want! The plot coordinate system is stretched, because we didn't call `ax.set_aspect('equal')` and `text` doesn't recalculate the text angle to make it align.



Now let's solve it for good in the general case, using trigonometry and then `transform_angles`. This is a method that we'll use with the transformation `ax.transData`. Try experimenting by replacing the `x2,y2` values to see this works for any angle.

```

1 x1, y1 = 0, 0
2 x2, y2 = 1, 1
3 x = (x1, x2)
4 y = (y1, y2)
5
6 # plot
7 fig, ax = plt.figure(), plt.axes()
8 ax.plot(x,y)
9
10 # Find angles and then insert text
11 slope = (y2 - y1) / (x2 - x1)
12 true_angle = math.degrees(math.atan(slope))
13
14 # dummy_array is the point where the angles are anchored
15 dummy_array = np.array([[0,0]]) # doesn't matter what
    pair you use
16 # matplotlib.org/stable/api/transformations.html#
    matplotlib.transforms.Transform.transform_angles
17
18 plot_angle = ax.transData.transform_angles(
19     np.array((true_angle,)),
20     dummy_array)[0]
21

```

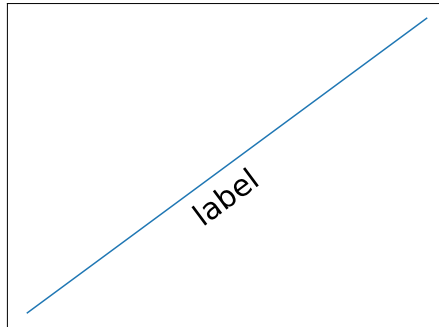


```

22 ax.text(np.mean(x), np.mean(y),
23         s = 'label',
24         rotation = plot_angle,
25         fontsize = 30,
26         va = 'top',
27         ha = 'center')
28
29 ax.grid(False)
30 ax.set_xticks([])
31 ax.set_yticks([])
32 print(true_angle, plot_angle)

```

[slope-label.py](#)



10.2 Circular Arrangements

With our knowledge of the unit circle, we can arrange some points in a circle with no additional help beyond the math package. This might be useful if you want to avoid mixing polar and Cartesian axes.

```

1 n_points = 10
2 pie_angle = 360/n_points # angle of each slice
3 starting_angle = 90
4
5 fig, ax = plt.subplots()
6
7 for i in range(n_points):
8
9     angle = starting_angle + i*pie_angle
10    angle = math.radians(angle)
11    x = math.cos(angle)
12    y = math.sin(angle)
13
14    ax.plot([x],[y], 'o', markersize = 17 - i)

```

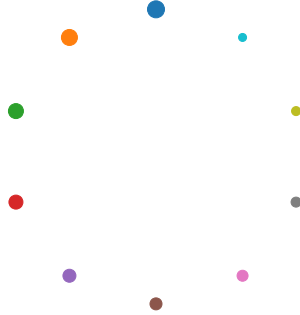
```

15
16 ax.set_aspect('equal')
17 ax.axis('off')

```

circle.py

This code produces the following.



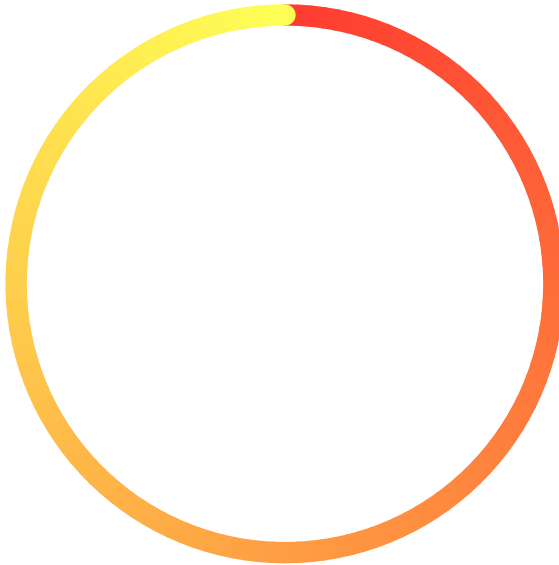
The below makes similar use of trigonometry to create a circle colored according to a gradient, like in Chapter 6. I make use of `solid_capstyle = 'round'` to round the endpoints of the plotted line, creating a cleaner look compared to the default.

```

1 # make a circle gradient
2 start_color = 255/256, 59/256, 48/256 # red
3 end_color = 255/256, 255/256, 85/256 # yellow
4
5 # How many color changes
6 segments = 130
7
8 # Create figure
9 fig, ax = plt.figure(figsize = (8,8)), plt.axes()
10
11 # Start at 90 degrees and return clockwise
12 angles = np.linspace(2.5*np.pi, np.pi/2, segments + 1)
13
14 # Create the intermediate colors
15 colors = dict()
16 for i in range(3):
17     colors[i] = np.linspace(start_color[i], end_color[i],
18                             segments)
19
20 # plot each arc
21 for i in range(segments):
22     start_angle = angles[i]
23     end_angle = angles[i+1]

```

```
24     angle_slice = np.linspace(start_angle, end_angle,
25                               100)
26     x_values = np.cos(angle_slice)
27     y_values = np.sin(angle_slice)
28
29     rgb = colors[0][i], colors[1][i], colors[2][i]
30
31     ax.plot(x_values, y_values,
32            color = rgb,
33            linewidth = 20,
34            solid_capstyle = 'round')
35
36 ax.set_aspect('equal')
37 ax.axis('off')
```

[circle-grad.py](#)

10.3 Network Graphs

Networks are represented mathematically as graphs—a set of vertices and edges between them. In drawing a graph, there are many drawing algorithms available. For large networks or sophisticated algorithms, you should use something off the shelf in a package

like `nxviz`. For a small network, you might avoid dealing with `NetworkX` and `nxviz` and do the drawing yourself. We will work through two simple layouts: arc diagrams and a circular layout for an undirected graph.

An arc diagram places all points on a straight line. The links are drawn as arcs from one point to another.

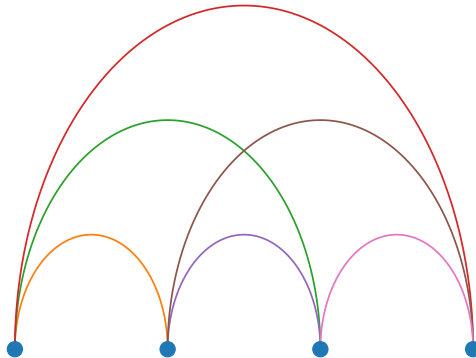
Let's consider the complete graph with four vertices, where every pair is connected.

```

1 fig, ax = plt.figure(), plt.axes()
2 x = np.linspace(0,1,4)
3 ax.plot(x, np.zeros(4),
4         marker = 'o',
5         linestyle = '',
6         markersize = 13)
7
8 angles = np.linspace(0,np.pi,100)
9 for point in x:
10     # connect other points
11     other_x = x[x > point]
12     # construct a half circle
13     unit_x, unit_y = np.cos(angles), np.sin(angles)
14     for other in other_x:
15         # arc is centered between the two points
16         shift = np.mean([point,other])
17         r = (other - point)/2
18         new_x = r*unit_x + shift
19         new_y = r*unit_y
20         ax.plot(new_x, new_y, zorder = -1)
21
22 ax.axis('off')
23 ax.set_aspect(1.5)

```

`arc-graph.py`



Next we move on to a circular layout. This layout places each vertex along a circle. Spaced evenly and with just four vertices in our graph, this will in fact produce a square. We also label each edge.

```

1 fig, ax = plt.figure(), plt.axes()
2
3 n_points = 4
4
5 # Draw vertices
6 angles = np.linspace(0, 2*np.pi, n_points + 1)[0:
    n_points]
7 x = np.cos(angles)
8 y = np.sin(angles)
9 ax.plot(x, y,
10         marker = 'o',
11         linestyle = '',
12         markersize = 13)
13
14 # Draw Edges
15 points = [p for p in zip(x,y)]
16 counter = 1
17 for point, other in combinations(points,2):
18
19     x = [p[0] for p in (point, other)]
20     y = [p[1] for p in (point, other)]
21     ax.plot(x, y, zorder = -1)
22
23     # add a label
24     label_point = .65*np.array(point) + .35*np.array(
        other)
25
26     run = x[1]-x[0]
27     rotation = 90
28     ha = 'left'
29     if run != 0:
30         line_slope = (y[1]-y[0])/(x[1]-x[0])
31         rotation = math.atan(line_slope)
32         rotation = math.degrees(rotation)
33         ha = 'center'
34     else:
35         print(point, other, rotation)
36
37     # get rgb then blend with white
38     line_color = mpl.colors.to_rgb("C"+str(counter))
39     lighter = .8*np.ones(3) + .2*np.array(line_color)
40     ax.text(label_point[0], label_point[1],
41            'label', rotation = rotation,
42            bbox = dict(facecolor = lighter),
43            va = 'center',
44            ha = 'center'
45            )

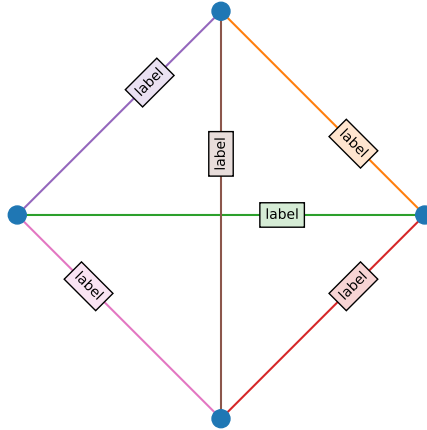
```

```

46     counter += 1
47
48     ax.axis('off')
49     ax.set_aspect('equal')

```

circle-graph.py



10.4 Tony Hawk's Vertical Loop

Tony Hawk became the first skateboarder to skate a vertical loop in 1998. We honor that accomplishment in two dimensions with the help of a rotation matrix. The unit circle is our vertical loop and we add two smaller circles to represent a skateboard. This is trigonometry. The small circles are placed along a ray from the origin of the unit circle to ensure they will lie tangent inside in the loop. In the first subplot, we place the skateboard at the bottom of the ramp. Though the same figure could be produced without using a rotation matrix, we use one so that the first subplot is essentially reused over and over by rotating the skateboard wheels up and around the loop.

```

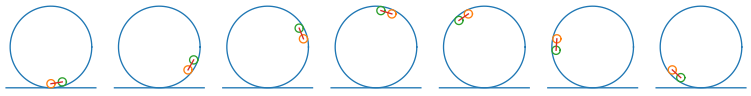
1  thetas = np.linspace(0,2*np.pi,8)[0:-1]
2  fig = plt.figure(figsize = (12,3))
3
4  # Set radius for skateboard wheels
5  radius = 0.1
6
7  # Make individual subplots

```

```

8  for key, theta in enumerate(thetas):
9      rotation_matrix = np.matrix([[np.cos(theta), -np.sin
      (theta)], [np.sin(theta), np.cos(theta)]])
10
11     # Create panel for one frame
12     ax = fig.add_subplot(1, len(thetas), key+1)
13     ax.set_aspect('equal')
14
15     # Plot the loop itself
16     angles = np.linspace(0, 2*np.pi, 100)
17     x = np.cos(angles)
18     y = np.sin(angles)
19     ax.plot(x,y)
20
21     # Make skateboard wheels at bottom of the ramp
22     # and then rotate them counter-clockwise according
    to theta
23     centers = list()
24     for ang in 1.5*np.pi, 1.6*np.pi:
25         center = (1-radius)*np.cos(ang), (1-radius)*np.
        sin(ang)
26
27         # rotate
28         point = np.matrix(center).T
29         rotated_point = rotation_matrix*point
30         rotated_point = np.array(rotated_point).flatten
    ()
31         centers.append(rotated_point)
32
33         # make wheel around new center
34         wheel_x = radius*x + rotated_point[0]
35         wheel_y = radius*y + rotated_point[1]
36
37         ax.plot(wheel_x, wheel_y)
38
39     # connect the two wheel centers
40     c1, c2 = centers
41     ax.plot([c1[0],c2[0]], [c1[1],c2[1]])
42
43     ax.axis('off')
44
45     xlim = ax.get_xlim()
46     ax.plot(xlim, [-1,-1],
47             color = 'CO',
48             zorder = -1)

```



Part III

Poetry



Newton by William Blake (Public Domain)

Chapter 11

Poetry

President George W. Bush is reported to have been fascinated by satellite pictures showing North and South Korea at night. South Korea is covered in bright lights and its communist neighbor, dark, could be mistaken for a collection of dead pixels. The pictures, like any data visualization, communicate and distill a lot of information. Karen Hughes, Bush's former counselor, said that to Bush, the pictures showed "the light and opportunity that comes with freedom, and the dark that comes with a regime that is oppressive." The pictures, like any data visualization, communicate and distill a lot of information. And like an effective novel data visualization, the pictures invite more active interest.

With prose behind us, we approach that end of the spectrum. Scruton [2015](#) argues that "poetry is concerned with the truth as a kind of revelation," standing apart from the "aboutness" of prose. Scruton adds, "When Keats writes his 'Ode to the Nightingale,' he does not describe the bird and its song only: he endows it with value." So here we are, trying to endow some data with value through its presentation.

This is a different kind of task you might take up once you've understood the important insights from your data and you have an editorial perspective. When the Bureau of Labor Statistics reports unemployment numbers, that should not be editorialized—prose line charts and tables will do. But sometimes your audience will benefit from receiving the data pre-chewed or more artfully presented.

To that end, this part aims to help you construct interesting charts, mostly with more considered use of Artist objects. This

will also help you construct prosaic plots, as you might also use these objects to build plots from scratch if the easier way escapes you. This will be more laborious. Poets do write fewer words than prose writers over their careers.

Chapter 12

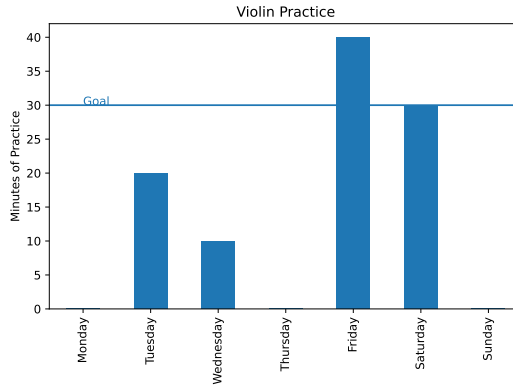
Applications

12.1 Activity Calendar

Let's start with a prose bar chart and turn it into poetry. Suppose you have a goal to practice the violin for at least 30 minutes every day and you've tracked your practice time for a whole week.

```
1 violin_practice = {'Monday': 0,  
2                     'Tuesday': 20,  
3                     'Wednesday': 10,  
4                     'Thursday': 0,  
5                     'Friday': 40,  
6                     'Saturday': 30,  
7                     'Sunday': 0}  
8  
9 pd.Series(violin_practice).plot.bar()  
10 plt.axhline(30)  
11 plt.ylabel('Minutes of Practice')  
12 plt.text(0, 30, 'Goal', color = 'C0')  
13 plt.title('Violin Practice')
```

[violin-bar.py](#)



This creates a chart that is perfectly fine. But a poet might say, why futz with all the numbers and obfuscate the essential meaning? We only need to know if we hit the goal of 30 minutes or not. This kind of simplification might be more motivating. There’s probably some psychology research that supports this, but we’ll proceed without deferring to any such authority because those studies so often fail to replicate. Let’s create a simple activity calendar of sorts.

```

1 fig, ax = plt.figure(figsize = (8,5)), plt.axes()
2 spacing_scale = 2.5
3
4 for idx, day in enumerate(violin_practice):
5     hit_target = violin_practice[day] >= 30
6
7     facecolor = 'black' if hit_target else 'white'
8     c = plt.Circle((idx*spacing_scale, 1),
9                   radius = .4,
10                  facecolor = facecolor,
11                  edgecolor = 'black')
12     ax.add_artist(c)
13     ax.text(idx*spacing_scale, 0,
14           s = day,
15           ha = 'center',
16           va = 'center')
17
18 # Eliminate Clutter
19 ax.set_aspect('equal')
20 ax.axis('off')
21
22 # Set plot window
23 ax.set_xlim([-1, 6*spacing_scale+1])
24 ax.set_ylim([0, 2])

```



You might have in mind ways to spice up this plot even further. Maybe there should be partial credit given to days some with violin practice, but not at least 30 minutes. Perhaps on those days, the circles can be filled with a light gray. The key here is that we're creating a plot that we might think will be more motivating.

Apps like Duolingo or Peloton gamify the user experience, and part of that is rewarding activity streaks over consecutive days. Let's try to enhance the plot so that streaks stand out. We'll accomplish this by adding a yellow edge color and increasing its weight as a streak continues. This adds some visual reward for streaks that was not evident in the previous plot.

```

1 violin_practice = {'Monday': 30,
2                     'Tuesday': 0,
3                     'Wednesday': 30,
4                     'Thursday': 30,
5                     'Friday': 40,
6                     'Saturday': 0,
7                     'Sunday': 60}
8
9 fig, ax = plt.figure(figsize = (8,5)), plt.axes()
10 spacing_scale = 2.5
11
12 streak = 0
13 for idx, day in enumerate(violin_practice):
14
15     hit_target = violin_practice[day] >= 30
16     streak += 1*hit_target
17     streak = streak if hit_target else 0
18
19     facecolor = 'black' if hit_target else 'white'
20     edgcolor = 'yellow' if hit_target else 'black'
21     c = plt.Circle((idx*spacing_scale,1),

```

```

22         radius = .4,
23         facecolor = facecolor,
24         edgecolor = edgecolor,
25         linewidth = 0.5 + streak)
26     ax.add_artist(c)
27     ax.text(idx*spacing_scale, 0,
28           s = day,
29           ha = 'center',
30           va = 'center')
31
32 # Eliminate Clutter
33 ax.set_aspect('equal')
34 ax.axis('off')
35
36 # Set plot window
37 ax.set_xlim([-1, 6*spacing_scale+1])
38 ax.set_ylim([0, 2])

```

[violin-streak.py](#)



12.2 Heatmaps

12.2.1 Google Trends

We have some data in a csv giving Google search trend popularity by month for computer scientist [Grace Hopper](#) and Nobel Prize-winning economist [Elinor Ostrom](#). We'll try visualizing the trends with a few heatmaps.

First, let's import the data. We can even look at the data in a table with matplotlib if for some sick reason you want to use matplotlib for such a task.


```

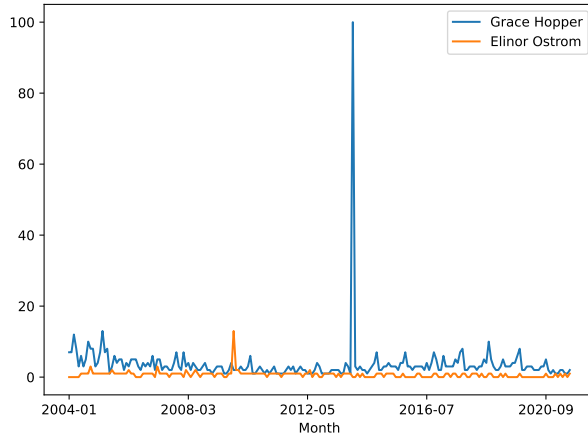
1 url = 'https://github.com/alexanderthclark/MPL-Data/raw/
    main/HopperOstromTrends.csv'
2 df = pd.read_csv(url, index_col = 'Month')
3 fig, ax = plt.subplots()
4 ax.axis('off')
5 n = 10 # how many rows
6 ax.table(cellText = df.head(n).values,
7         rowLabels = df.head(n).index,
8         colLabels = list(df),
9         loc = 'center')

```

mpl-table.py

	Grace Hopper	Elinor Ostrom
2004-01	7	0
2004-02	7	0
2004-03	12	0
2004-04	8	0
2004-05	3	0
2004-06	6	1
2004-07	3	1
2004-08	5	1
2004-09	10	1
2004-10	8	3

A table is capital-B Boring. A `df.plot()` line chart would display these trends very well, but let's move on to making a heatmap. Here we'd have just two lines, but imagine we had several more columns in our dataset. The line chart would become a spaghetti chart. Heatmaps can be useful in avoiding this by giving each data series its own lane. Take this as a toy example of an application that helps when you have that additional clutter.



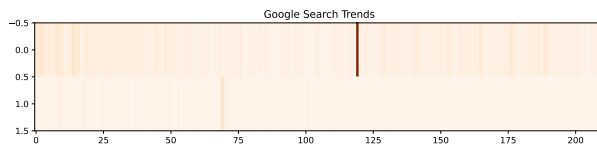
Matplotlib offers the `imshow()` axes method especially for this purpose. Later, we'll create our own heatmap simply by adding our own artist objects to the plot.

Using `imshow()`

First can call `imshow()` with our transposed dataframe to get a sense of what the method does. The transpose is done so that the time (the rows of the DataFrame) will be on the *x*-axis. The `aspect` argument essentially makes the chart taller. With `aspect` set to 20, each cell is 20 times taller than it is wide. Finally, I change the colormap with `cmap = 'Oranges'` to create a monochromatic map where a darker orange is a higher search volume.

```
1 fig, ax = plt.figure(figsize = (10,3)), plt.axes()
2 ax.imshow(df.T,
3           aspect = 20,
4           cmap = 'Oranges')
5 ax.set_title("Google Search Trends")
```

[heat-basic.py](#)



One obvious problem with this is we have no useful labels or ticks, unlike what we'd get for free with a simple `df.plot()` call. This will be remedied with the ticks customizations we learned about in Section 2.3. Another problem is that our data is not well behaved. The trends are washed out because of an outlier. In December 2013, Grace Hopper was featured in the Google Doodle, driving a lot of searches. All other months pale in comparison. By default, `imshow` uses a linear scaling mapping where the lowest value is at the bottom of the colormap and the highest value is at the top of the colormap. As a result almost, all observations are toward the bottom of the colormap. One could transform the data directly or one could change the behavior of `imshow()`. We can transform the data with the `norm` argument or manually set the top and bottom values of the colormap with `vmin` and `vmax`.

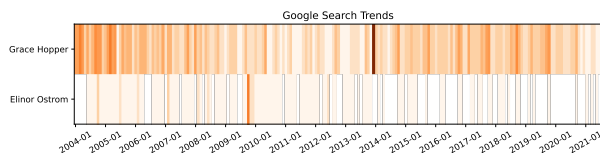
First we use `mpl.colors.LogNorm()` to take a log transform of the data before normalizing. We also thin the *x*-axis ticks manually (instead of using a Formatter) and then relabel the ticks manually.

```

1 fig, ax = plt.subplots(figsize = (10,3))
2 ax.imshow(df.T,
3           aspect = 20,
4           cmap = 'Oranges',
5           norm = mpl.colors.LogNorm())
6 ax.set_title("Google Search Trends")
7
8 thinning = 12 # label every 12th month
9 ax.set_xticks(range(0,len(df),thinning))
10 ax.set_xticklabels(list(df.T[::thinning]),
11                   rotation = 30)
12 ax.set_yticks(range(2))
13 ax.set_yticklabels(df.T.index)

```

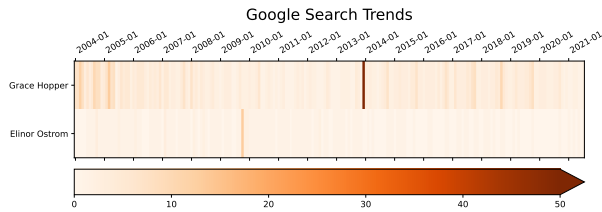
[heat-log.py](#)



The data transformation helps the trends seen in the original line chart pop more prominently. Elinor Ostrom's search interest has remained steady but for a spike in 2009 when she was awarded the Nobel Prize in Economics, along with Oliver Williamson. The surge in search interest for Grace Hopper because of the 2013

Google Doodle remains noticeable, but the yearly spikes in interest around the Grace Hopper conference (usually held around the beginning of October) are more noticeable. Perhaps you find the transformation makes the heatmap too noisy. We can keep the quiet of the original linearly-scaled heatmap, but make but make spikes in interest more visible by lowering the point at which the color gradient maxes out. Below we do this by lowering `vmax`. By default, `vmax` uses the maximum data value (100 in our case). Setting `vmax = 50` means values from 50 to 100 are not differentiated by color in the heatmap. We'll also add a colorbar.

```
1 fig, ax = plt.subplots(figsize = (10,5))
2 s = ax.imshow(df.T,
3               aspect = 20,
4               cmap = 'Oranges',
5               vmax = 50)
6
7 thinning = 12
8 ax.set_xticks(range(0,len(df),thinning))
9 ax.set_xticklabels(list(df.T[::thinning],
10                        rotation = 30,
11                        ha = 'left'))
12
13 ax.set_yticks(range(2))
14 ax.set_yticklabels(df.T.index)
15 ax.xaxis.set_tick_params(labeltop = True,
16                          labelbottom = False,
17                          labelsz = 10,
18                          tick20n = True,
19                          tick10n = True)
20 cbar = fig.colorbar(mappable = s,
21                    ax = ax,
22                    orientation = 'horizontal',
23                    pad = 0.04,
24                    extend = 'max')
25
26 ax.set_title("Google Search Trends",
27             size = 18,
28             pad = 12,
29             loc = 'center')
```



12.2.2 NHL Regular Season Records

The previous examples used `imshow()` to turn a rectangular array into a heatmap. Now we will keep the grid, but draw each observation ourselves. Doing so gives us control over the treatment of missing data, the labels inside the cells, and the story-telling annotations.

The data are final regular-season standings for six NHL teams from 1999–00 through 2008–09, retrieved from the official [NHL standings season manifest](#) and the dated standings endpoints. The accompanying [fetch-nhl-standings.py](#) script retrieves the same season-end rows used to create the committed CSV. Points percentage is calculated as points divided by twice the number of games played. This puts every team on the same zero-to-one scale. The 2004–05 season was never played because of a league-wide lockout.

The teams are selected to show contrasting paths. Detroit shows a consistently strong benchmark; Tampa Bay rises and falls; Nashville shows gradual growth; and Washington, Pittsburgh, and Chicago trace three different rebuilds.

First, here is the conventional version. We pivot the data into a team-by-season matrix and pass it to `imshow()`. A masked value and a modified “bad” color make the lockout visibly different from a low-performing season.

The data reshaping and `imshow()` call follow the same short figure-module style used throughout the book.

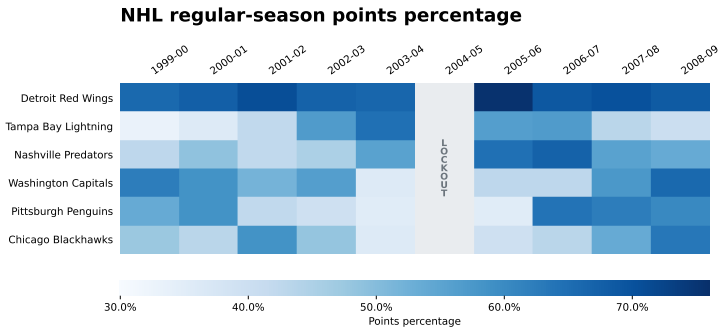
```
1 standings = pd.read_csv('../Data/nhl_regular_season.
   csv')
2
3 seasons = ['1999-00', '2000-01', '2001-02', '2002-03', '
   2003-04',
4           '2004-05', '2005-06', '2006-07', '2007-08', '
   2008-09']
```

```

5 teams = ['Detroit Red Wings', 'Tampa Bay Lightning',
6          'Nashville Predators', 'Washington Capitals',
7          'Pittsburgh Penguins', 'Chicago Blackhawks']
8
9 records = (standings.pivot(index = 'team',
10                             columns = 'season',
11                             values = 'point_pct')
12           .reindex(index = teams,
13                    columns = seasons))
14
15 cmap = mpl.colormaps['Blues'].with_extremes(bad = '#
16         e9ecef')
17
18 fig, ax = plt.subplots(figsize = (10, 4.6))
19 image = ax.imshow(np.ma.masked_invalid(records),
20                  aspect = 'auto',
21                  cmap = cmap,
22                  vmin = .30,
23                  vmax = .76)

```

[hockey-heat-basic.py](#)



This chart gives a quick overview, but it asks the colorbar to do almost all of the work. Exact values require estimation, the missing season competes visually with the data, and the post-lockout rebuilds are easy to overlook.

For the final version, each data value becomes a `Circle` artist. Here we deliberately encode points percentage twice. The radius follows the original expanding-bubble rule, $0.25 + \text{value} / 2$, so stronger seasons swell into their neighbors (an artistic choice). Every circle uses Matplotlib's first default color, `co`, while its opacity follows the same value; direct labels provide exact values. The redundant encodings create a rhythm across each row rather than asking a legend to do all of the work. Compact horizontal season labels keep the top of the grid quiet.

Gold edges mark seasons when one of the six won the Stanley Cup: Detroit in 2001–02 and 2007–08, Tampa Bay in 2003–04, and Pittsburgh in 2008–09.

The text labels also become artists. Each team’s mark is loaded with Pillow, placed in an `OffsetImage`, and anchored beside its row with an `AnnotationBbox`. A shaded band reserves space for the cancelled season, where the letters in “LOCKOUT” are stacked downward but kept upright.

Before placement, the code crops transparent padding and fits every mark inside the same 170-by-170-pixel box. Applying one shared zoom then makes the six labels roughly uniform, but some by-hand modifications are required. Washington’s wide wordmark may deserve a slightly larger team-specific zoom so the small “capitals” lettering remains legible, whereas the other five logos contain no text.

The artist-based version is kept in one figure module, just like the other examples in this chapter.

```

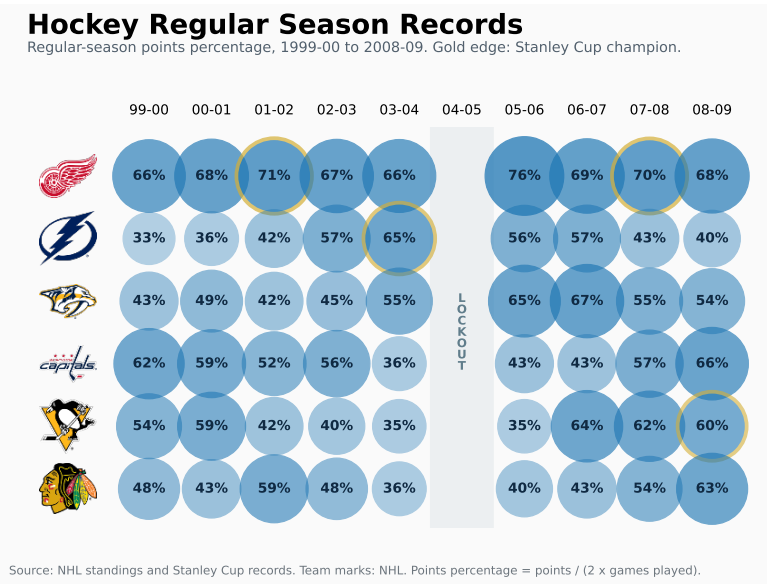
1 from PIL import Image
2
3 fig, ax = plt.subplots(figsize = (11.5, 5.8),
4                           facecolor = (.98, .98, .98))
5 ax.set_facecolor((.98, .98, .98))
6
7 cup_winners = {('Detroit Red Wings', '2001-02'),
8                ('Tampa Bay Lightning', '2003-04'),
9                ('Detroit Red Wings', '2007-08'),
10               ('Pittsburgh Penguins', '2008-09')}
11
12 lockout = seasons.index('2004-05')
13 ax.axvspan(lockout-.5, lockout+.5,
14            color = '#eceff1',
15            zorder = 0)
16
17 for team_key, team in enumerate(teams):
18     for season_key, season in enumerate(seasons):
19         record = records.loc[team, season]
20         if pd.isna(record):
21             continue
22
23         won_cup = (team, season) in cup_winners
24         if won_cup:
25             edgecolor, linewidth = '#D4AF37', 3
26         else:
27             edgecolor, linewidth = 'C0', 1
28
29         circ = plt.Circle((season_key, team_key),
30                           radius = .25 + record/2,
31                           facecolor = 'C0',

```

```

32         edgecolor = edgecolor,
33         linewidth = linewidth,
34         alpha = record,
35         zorder = 2)
36     ax.add_artist(circ)
37     ax.text(season_key, team_key,
38             s = f'{record:.0%}',
39             ha = 'center',
40             va = 'center',
41             color = '#102a43',
42             size = 11,
43             weight = 'bold',
44             zorder = 3)
45
46     ax.text(lockout, (len(teams)-1)/2,
47             s = '\n'.join('LOCKOUT'),
48             ha = 'center',
49             va = 'center',
50             color = '#607d8b',
51             size = 9.5,
52             weight = 'bold',
53             linespacing = .92,
54             zorder = 4)
55
56     team_marks = ['DET', 'TBL', 'NSH',
57                  'WSH', 'PIT', 'CHI']
58     for team_key, mark in enumerate(team_marks):
59         file = '../images/nhl/' + mark + '.png'
60         img = Image.open(file).convert('RGBA')
61
62         # Crop transparent padding before setting a common
63         # maximum size.
64         alpha_box = img.getchannel('A').getbbox()
65         if alpha_box:
66             img = img.crop(alpha_box)
67             img.thumbnail((170, 170))
68
69         off_img = mpl.offsetbox.OffsetImage(np.asarray(img),
70                                             zoom = .27)
71         label = mpl.offsetbox.AnnotationBbox(
72             off_img, (-1.3, team_key),
73             xycoords = 'data',
74             frameon = False)
75     ax.add_artist(label)

```

The finished figure now makes a broader claim. Washington, Pittsburgh, and Chicago all sit near the bottom around the lockout and finish 2008–09 at 66, 60, and 63 percent, respectively. Tampa Bay moves in the opposite direction, peaking at 65 percent before the lockout and ending at 40 percent. Detroit’s consistently large bubbles keep both patterns in perspective, while Nashville supplies a steadier middle path.

12.3 Directed Graphs

Directed graphs arise in many settings. For plotting a large graph, like follower-following relationships in a social network, you might be best served making use of packages like `networkx` and `nxviz`. In other cases, you might do better working by hand. The `graphviz` library is one possible solution. Here, we’ll work directly with `matplotlib`. One directed graph use case might be in illustrating the directed acyclic graph (or DAG) representing a causal theory. In the directed acyclic graph framework (mostly associated with Judea Pearl’s work in causal inference), a directed edge from X to Y means X causes Y , at least in part. This lends itself well to plotting. Below we’ll make a plot, using `plt.Circle()` to draw nodes and creating edges with `ax.annotate()`.

Here, we create the nodes as circle objects and then draw the edges using `ax.annotate()`. Below, the circular nodes are created with the function `make_node()`. Then, the directed edge is drawn with `directed_edge()`. This doesn't allow for an edge from one node back to itself.

```

1 def make_node(center, radius = 'auto', label = '', ax =
  None):
2     """Plot labeled circle object to represent a node.
3     Run after any changes to axes limits, aspect
4     changes, etc if using radius = 'auto'."""
5     if ax is None:
6         ax = plt.gca()
7     t = ax.text(center[0], center[1], label, ha = '
  center', va = 'center')
8     if radius == 'auto':
9         plt.gcf().canvas.draw()
10        box = t.get_window_extent().transformed(ax.
  transData.inverted())
11        width = box.x1 - box.x0
12        radius = (width/2)*1.2
13    c = plt.Circle(center, radius,
14                   facecolor = (.9,.99,.9),
15                   edgecolor = 'black')
16    ax.add_artist(c)
17    return c

```

`dag-node.py`

```

1 def directed_edge(c1, c2, ax = None):
2     """Draw an arrow from c1 to c2."""
3
4     if ax is None:
5         ax = plt.gca()
6
7     center1 = c1.center
8     center2 = c2.center
9
10    length = np.linalg.norm(np.array(center1) - np.array
  (center2))
11
12    r1 = c1.get_radius()
13    r2 = c2.get_radius()
14
15    x1, x2 = center1[0], center2[0]
16    y1, y2 = center1[1], center2[1]
17
18    # Find start and end of arrow based on circle radii
19    # based on linear weights from convex combos
20    tail_weight = r1 / length
21    tail_x = (1-tail_weight)*x1 + tail_weight*x2
22    tail_y = (1-tail_weight)*y1 + tail_weight*y2

```

```

23
24     head_weight = r2/ length
25     head_x = (1-head_weight)*x2 + head_weight*x1
26     head_y = (1-head_weight)*y2 + head_weight*y1
27
28     ax.annotate(' ', xy = (head_x, head_y),
29                 xytext = (tail_x, tail_y),
30                 arrowprops = dict(headwidth = 14,
31                                   linewidth = .1,
32                                   width = 2))

```

dag-edge.py

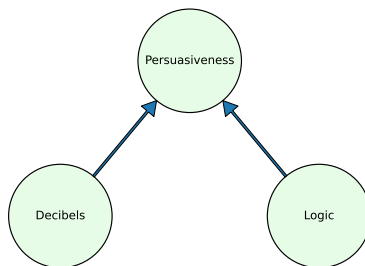
The DAG plotted below describes the theory that the persuasiveness of an argument is caused by its logical soundness and the decibel level at which it is communicated.

```

1 fig, ax = plt.figure(), plt.axes()
2 ax.set_aspect(1)
3 ax.set_xlim(-4,4)
4 ax.set_ylim(-3,3)
5 ax.axis('off')
6
7 persuasive = make_node((0,1.5), 1, 'Persuasiveness')
8 logic = make_node((2.5,-1.5), 1, 'Logic')
9 decibels = make_node((-2.5,-1.5), 1, 'Decibels')
10
11 directed_edge(logic, persuasive)
12 directed_edge(decibels, persuasive)

```

dag-argue.py



12.4 Speedometer

There's an allure to control rooms or all the gauges on the dashboard of a vehicle. If you can make your dashboards alluring, your stakeholders will visit them more often. We'll create a simple speedometer-like gauge to add some visual interest to reporting a single percentile value. We'll use what we learned about rotating points in Section 9.1.3. The gauge will have values running from 0% to 100%, and we'll place these along a half circle. The gauge's hand will point to a particular realized percentile value. We use a rotation matrix to find the correct angle at which to place the hand.

```

1 def rotation(theta):
2     """Construct rotation matrix for angle theta in
3     radians."""
4     rotation_matrix = np.matrix([[np.cos(theta), -np.sin
5     (theta)], [np.sin(theta), np.cos(theta)]])
6     return rotation_matrix
7
8 def speedometer(percentile, ax = None, fsize = 20):
9     """Constructor speedometer plot along the half
10    circle from 0 to pi."""
11    # Make half circle from 0 to pi
12    angles = np.linspace(0, np.pi, 100)
13    x = np.cos(angles)
14    y = np.sin(angles)
15    if ax is None:
16        ax = plt.gca()
17    ax.set_aspect('equal')
18    ax.axis('off')
19    ax.plot(x,y, linewidth = 3, color = 'black')
20
21    # Calculate angle for percentile
22    theta = -np.pi * (percentile/100)
23
24    # Draw hand initialized at 180 degrees
25    length = 0.8
26    base = np.matrix([0,0])
27    tip = np.matrix([-length,0])
28    points = [base,tip]
29    new_points = [rotation(theta)*p.T for p in points]
30    x = [np.array(p).flatten()[0] for p in new_points]
31    y = [np.array(p).flatten()[1] for p in new_points]
32
33    ax.plot(x,y, color = 'darkred',
34            linewidth = 4, solid_capstyle = 'round')
35    ax.plot(0,0, marker = 'o',
36            color = 'darkred', markersize = 10)

```

```

35
36     # Mark every 10pp
37     ticks = np.linspace(0,180, 11)
38     for angle in ticks: #np.arange(0,181, step = step):
39         radians = math.radians(angle)
40
41         # tick line
42         x1, y1 = np.cos(radians), np.sin(radians)
43         x2, y2 = .95*x1, .95*y1
44         ax.plot([x1,x2], [y1,y2],
45                 color = 'black', zorder = -1)
46
47         # place text label by tick
48         raw = 180 - angle
49         ptile = raw/180 * 100
50         s = "{:.0f}%".format(ptile)
51         ax.text(.9*x2, .9*y2, s,
52                 ha = 'center', zorder = -1)
53
54     # ghost in the value bc why not
55     # edit to include raw value or whatever desired
56     ax.text(0.04, .15, '{:.0f}%'.format(percentile),
57            va = 'bottom',
58            ha = 'center',
59            size = fsize,
60            alpha = 0.2)

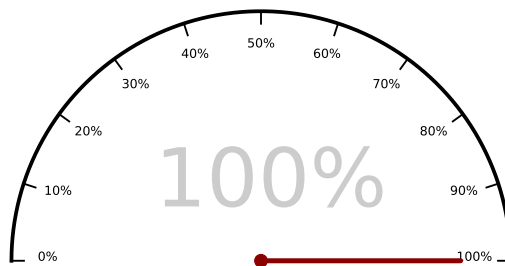
```

[speedo-functions.py](#)

```

1 speedometer(100, fsize = 65)

```

[speedometer.py](#)

The helper function accepts an Axes object rather than creating a new Figure, allowing the same drawing to be reused across subplots. Below, one loop places three gauges on three Axes. You might prefer to make further modifications to the sizing and spacing: the tick labels are crowded in these smaller panels.

```
1 fig, axx = plt.subplots(1,3, figsize = (10,3))
2
3 for ax, p in zip(axx, [20, 50, 80]):
4     speedometer(p, ax = ax, fsize = 33)
```

[speedometers.py](#)



If the gauges must remain this small, we could label every 20 percentage points instead of every 10, reduce the text size, or increase the space between subplots. Which adjustment is best depends on what the audience needs to compare. The hand and the large value already communicate the approximate percentile, so every tick label may not be earning its space.

The reusable pattern matters more than the speedometer itself. The `speedometer()` function handles the geometry and styling of one gauge, while the calling code decides where it belongs. Functions that accept `ax` can be used alone, repeated in a grid, or inserted into a larger composition without rewriting the drawing instructions.

Part IV

Special Topics



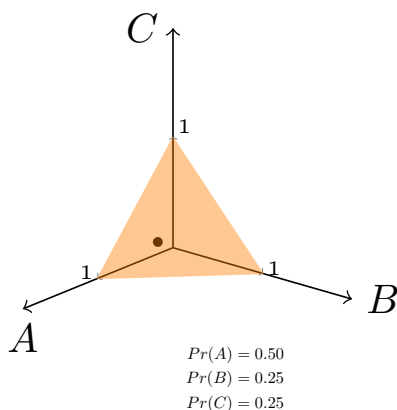
A Virgin with a Unicorn by Domenichino (Public Domain)

Chapter 13

Ternary Plots

13.1 Ternary

This section introduces the [python-ternary package](#). You'll need to install this with pip or conda to follow along. You can use this to make various plots in the two-dimensional simplex. That is, you can make triangle plots where a point in the triangle represents a particular multinomial distribution over three possible outcomes. That is, the triangle is a two-dimensional projection of the space $\{(p_1, p_2, p_3) \in \mathbb{R}^3 : p_1 + p_2 + p_3 = 1 \text{ and } p_i \in [0, 1] \text{ for } i = 1, 2, 3\}$.



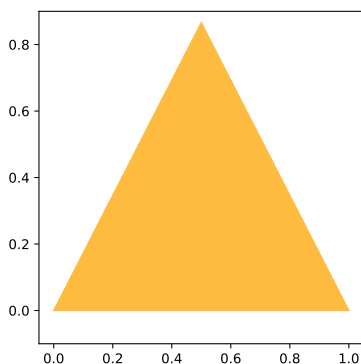
I encountered these diagrams in a few economics courses. Professor [Bill Sandholm](#) made particularly memorable use of these diagrams in his courses and research in evolutionary game theory.

These plots aren't the most natural selection for inclusion in this text. It's a personal indulgence and a favor to other game theorists.

After running `import ternary`, the construction of a plot isn't much different than what we covered in Chapter 1. Start with `figure` and `ternary` axes objects. However, `ternary.figure()` will create both objects. There is no analogue to `plt.axes()` as this more closely imitates `plt.subplots(1,1)` than `plt.figure()`. It's not a perfect replica though. For example, there is no `figsize` parameter, but this can be adjusted with the `figure` method `set_size_inches`.

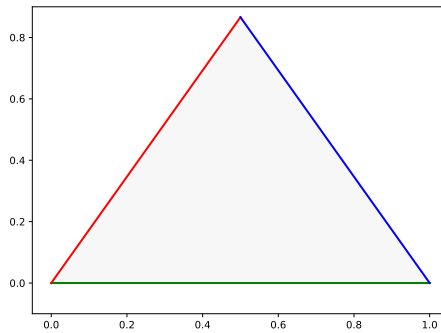
```
1 figure, tax = ternary.figure() # A tuple of plot objects
2 figure.set_size_inches(4,4)
3 tax.set_background_color('orange')
```

[basic-ternary.py](#)



```
1 # Create the Plot
2 scale = 1 # length of the sides
3 figure, tax = ternary.figure(scale=scale)
4
5 # Draw Boundary
6 tax.boundary(linewidth= 2.0,
7             axes_colors = {'l':'red', 'r': 'blue', 'b':
8                           'green'})
```

[color-ternary.py](#)



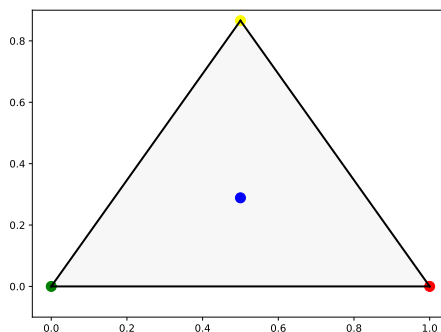
Next, let's add some points with the `scatter` method, which works as you might expect.

```

1 scale = 1
2 figure, tax = ternary.figure(scale=scale)
3
4 # Draw Boundary
5 tax.boundary(linewidth= 2.0)
6
7 # Scatter Points
8 points = [(1,0,0), (0,1,0), (0,0,1), (1/3, 1/3, 1/3)]
9 tax.scatter(points,
10             marker = 'o',
11             color = ['red', 'yellow', 'green', 'blue'],
12             s = 100)

```

[scatter-ternary.py](#)



The x - and y -axis ticks above correspond to the bottom and right axes, but this can be confusing since the x, y point of $(0,0)$ is the point $(0,0,1)$ in the ternary plot. Accordingly, you might prettify the plot by removing those ticks and these axes entirely.

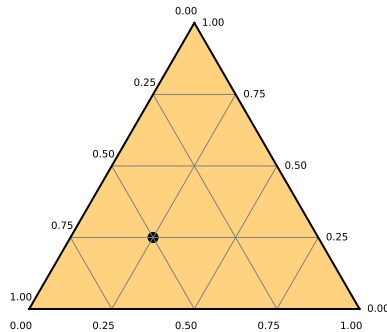
This can be done with `tax.get_axes().axis('off')`. This works like `ax.axis` so we can also pass `'equal'` to equalize the axes. Adding gridlines with the `gridlines` method can also help the eye. This is demonstrated below. The horizontal gridlines correspond to the value along the right axis. The negatively sloped gridlines correspond to the left axis. The positively sloped gridlines correspond to the bottom axis. These gridlines are perpendicular to the direction of ascent along each axis and, just as for a typical plot, they show where the particular axis value is constant.

```

1 scale = 1
2 figure, tax = ternary.figure(scale=scale)
3 tax.set_background_color('orange',
4                           alpha = 0.5)
5
6 # Add ticks along the triangle edges
7 tax.ticks(axis = 'lbr',
8           multiple = .25,
9           tick_formats = '%.2f',
10          offset= 0.02,
11          linewidth = 0)
12
13 tax.gridlines(multiple = .25,
14              color = 'gray',
15              linewidth = 0.5,
16              linestyle = 'solid')
17
18 points = [(0.25, 0.25, 0.5)]
19 tax.scatter(points,
20            marker = 'o',
21            color = ['black'],
22            s = 100)
23
24 tax.boundary(linewidth= 2.0)
25 tax.get_axes().axis('equal')
26 tax.get_axes().axis('off')

```

`grid-ternary.py`



13.2 Application: Rock, Paper, Scissors

Throughout this chapter, we'll analyze the game rock paper scissors. If you need a reminder, there are two players who simultaneously choose an action of either rock, paper, or scissors. Rock beats scissors beats paper beats rock. Choosing the same action results in a tie. Your job is to choose an action based on your expectation of what your opponent will choose.

First, we'll construct a heatmap to show the net winning percentage from choosing a particular action depending on the opponent's probability distribution over the three actions, so that a point in the simplex is that opponent's strategy and the color represents how often you win. The following is a function we'll use in making a heatmap, calculating the net winning percentage of an action against a particular distribution.

```

1 def winning_pct(p, action = 'rock'):
2
3     """What is the net winning percentage given a choice
4       of action and an opponent's strategy p, where p is
5       a probability distribution over rock, paper, and
6       scissors."""
7
8     if action.lower() == 'rock':
9
10        winning_pct = p[2] # pr win
11        net = p[2] - p[1] # pr win - pr lose
12
13    elif action.lower() == 'paper':
14
15        winning_pct = p[0]
16        net = p[0] - p[2]
17
18    elif action.lower() == 'scissors':

```

```

16         winning_pct = p[1]
17         net = p[1] - p[0]
18     else:
19         raise ValueError("Input is not a valid action")
20
21     return net
22

```

`rps-helper.py`

Now, we can create a heatmap using the function `winning_pct` and the `heatmapf` method.

```

1 figure, tax = ternary.figure(scale = 20)
2 # vary scale above for higher resolution
3 figure.set_size_inches(9, 8)
4
5 # Add Heatmap
6 tax.heatmapf(winning_pct, boundary=True,
7              style="triangular",
8              cmap = 'Purples',
9              colorbar = False)
10
11 tax.boundary(linewidth=2.0)
12
13 title = 'Net Winning % If I Choose Rock'
14 tax.set_title(title + "\n")
15
16 tax.right_corner_label('Rock',
17                        position = (.88,0.05,.09), fontsize=10)
18 tax.top_corner_label('Paper',
19                     position = (.01,1.11,.005),fontsize=10)
20 tax.left_corner_label('Scissors',
21                      position = (.07,0.05,1), fontsize=10)
22
23 tax.get_axes().axis('off')
24 # Workaround for a bug with labels
25 tax._redraw_labels()

```

`heat-rps.py`



The natural extension of the above might be to repeat the above, which supposes you choose rock, for the actions paper and scissors. This can be done straightforwardly, by changing the default action in the `winning_pct` function. For the game theorist, the more interesting question is, given my opponent's distribution over actions, what is my best response? The code below plots the regions of pairwise indifference between two actions. Then, we divide up the simplex into three best response regions.

```

1 scale = 1
2 figure, tax = ternary.figure(scale = scale)
3 # vary scale above for higher resolution
4 figure.set_size_inches(9, 8)
5
6 # Rock > Paper
7 # s - p > r - s
8 # 2s > r + p
9 p1 = (2/3, 0, 1/3) # rps ordering
10 p1 = scale * np.array(p1)
11 p2 = (0, 2/3, 1/3)
12 p2 = scale * np.array(p2)
13
14 tax.line(p1, p2,
15         linewidth=3.,
16         marker='s',
17         color='green',
18         linestyle=":")
19 tax.annotate(' R < P', (.75*p1 + .25*p2),

```

```

20         color = 'green',
21         ha = 'left',
22         va = 'top')
23 tax.annotate('R > P ', (.75*p1 + .25*p2),
24             color = 'green',
25             ha = 'right',
26             va= 'bottom')
27
28 # Rock > Scissors
29 # s - p > p - r
30 # s + r > 2p
31 p1 = (2/3, 1/3, 0) # rps ordering
32 p1 = scale * np.array(p1)
33 p2 = (0, 1/3, 2/3)
34 p2 = scale * np.array(p2)
35
36 tax.line(p1, p2,
37         linewidth = 3.,
38         marker = 's',
39         color = 'blue',
40         linestyle = ":",
41         label = 'RockScissors')
42 tax.annotate('R > S', (.75*p1 + .25*p2),
43             color = 'blue',
44             ha = 'left',
45             va = 'top')
46 tax.annotate('R < S', (.75*p1 + .25*p2),
47             color = 'blue',
48             ha = 'right',
49             va= 'bottom')
50
51 # Paper > Scissors
52 # r - s > p - r
53 # 2r > p + s
54 p1 = (1/3, 2/3, 0) # rps ordering
55 p1 = scale * np.array(p1)
56 p2 = (1/3, 0, 2/3)
57 p2 = scale * np.array(p2)
58
59 tax.line(p1, p2,
60         linewidth = 3,
61         marker = 's',
62         color = 'red',
63         linestyle = ":")
64
65
66 tax.annotate('S > P ', (.75*p1 + .25*p2),
67             color = 'red',
68             ha = 'left',
69             va = 'top')
70 tax.annotate('S < P', (.75*p1 + .25*p2),
71             color = 'red',
72             ha = 'right',

```

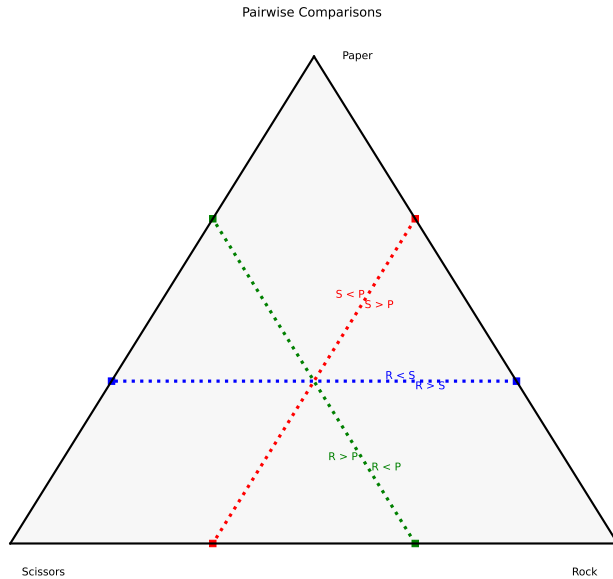


```

73         va= 'bottom')
74
75 tax.boundary(linewidth=2.0)
76
77 # Make pretty as desired
78 title = 'Pairwise Comparisons'
79 tax.set_title(title + " \n")
80
81 tax.right_corner_label('Rock',
82                        position = (.88,0.05,.09),
83                        fontsize=10)
84 tax.top_corner_label('Paper',
85                      position = (.01,1.11,.005),
86                      fontsize=10)
87 tax.left_corner_label('Scissors',
88                       position = (.07,0.05,1),
89                       fontsize=10)
90
91 tax.get_axes().axis('off')
92 tax._redraw_labels()

```

[rps-br-lines.py](#)



We can color best responses zones by using the `heatmap` method, though it will take some work. First, we have some pure Python coding to do. Recall that `heatmap` requires a correspondence of

points and the colors you want. We won't use a colormap, but we'll pass the colors in explicitly as RGBA values. This is a bit of a hack since the resulting plot and its colors won't have the ordered interpretation typical of heatmaps.

```

1 def color_point(x, y, z):
2     """Given an opponent plays rock at chance x, paper
3     at y, and scissors at z, what is the best response?
4     Best responses are mapped to RGB colors."""
5
6     # winning pct's for possible responses
7     rock_net = z - y
8     paper_net = x - z
9     scissors_net = y - x
10
11    # get best response as highest net winning pct
12    list_ = [rock_net, paper_net, scissors_net]
13    best = list_.index(max(list_))
14
15    # map into RGB color weights
16    colors = [0, 0, 0]
17    colors[best] = 1
18
19    # return RGB tuple with fourth value for opacity (
20    alpha)
21    return (*tuple(colors), 1.)

```

rps-br-helper.py

```

1 # Adapted from https://github.com/marcharper/python-
2   ternary/blob/master/README.md RGBA section
3 def generate_heatmap_data(scale=10):
4     from ternary.helpers import simplex_iterator
5     d = dict()
6     for (i, j, k) in simplex_iterator(scale):
7         d[(i, j, k)] = color_point(i, j, k)
8     return d
9
10 # Scale should be chosen high enough for sharp
11   resolution
12 scale = 200
13 data = generate_heatmap_data(scale)
14 figure, tax = ternary.figure(scale=scale)
15 figure.set_size_inches(9, 8)
16
17 tax.heatmap(data, style="hexagonal",
18             use_rgba=True, colorbar = False)
19 tax.boundary()
20 tax.set_title("Best Response Zones")
21
22 # Label the corners

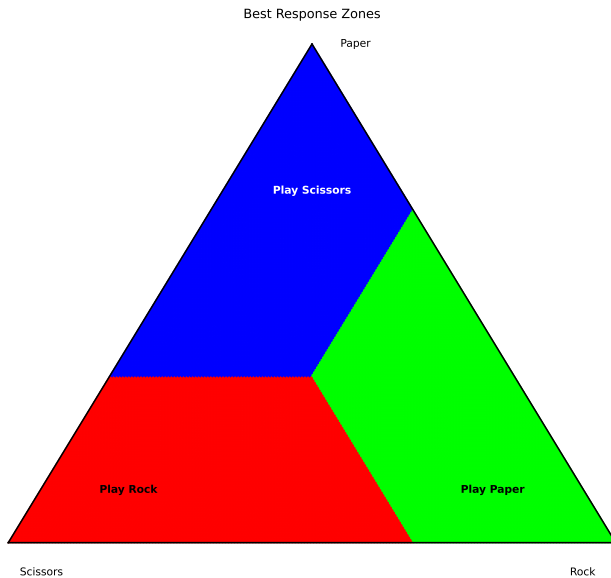
```

```

22 labels = 'Rock', 'Paper', 'Scissors'
23 tax.right_corner_label(labels[0],
24     position = (.88,0.05,.09), fontsize=10)
25 tax.top_corner_label(labels[1],
26     position = (.01,1.11,.005),fontsize=10)
27 tax.left_corner_label(labels[2],
28     position = (.07,0.05,1), fontsize=10)
29
30 # Label best response zones
31 tax.annotate("Play Rock",
32     (.1* scale,.1 * scale,.8 * scale), weight = 'bold')
33 tax.annotate("Play Paper",
34     (.8*scale, .1*scale, .1*scale), ha = 'right',
35     weight = 'bold')
36 tax.annotate("Play Scissors",
37     (.15*scale, .7*scale, .15*scale), ha = 'center',
38     color = 'white', weight = 'bold')
39
40 # Clear background and axes
41 tax.clear_matplotlib_ticks()
42 tax.get_axes().axis('off')
43 tax._redraw_labels()

```

[rps-br-zones.py](#)



Anyone inspecting what data looks like above will note that the points in the triangle aren't proper probability vectors as they add

up to our `scale` value of 200 instead of one. That's immaterial for this application. A higher scale is chosen to create a sharper, exact border between between the best response regions.

Chapter 14

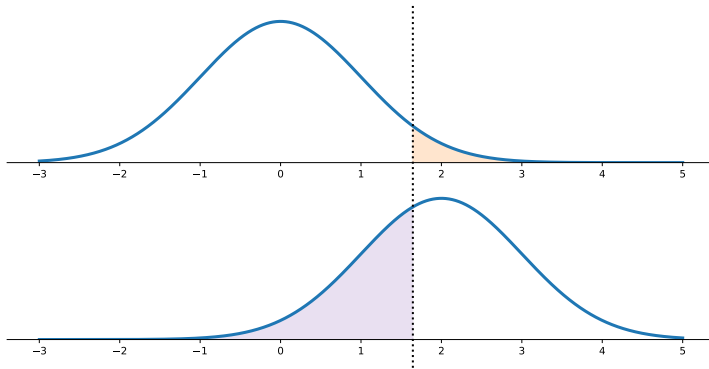
Intro Statistics

Since the fall of 2022, I've been teaching intro stats and I've been very particular about my graphs. According to my college ethics professor, the experience of beauty is a human good and statistics is not.¹ That doesn't provide a good prioritization for developing an intro stats syllabus, but as an instructor, I've at least tried to make my slides and notes a kind of beautiful. Watching a student pinch to zoom on her iPad without a particular graph pixelating or otherwise revealing any defect was a beautiful moment.

I make all of the figures for my class materials in Python. Working in Python offers the advantage of, well, working in Python. We are able to mix statistical functions from Python in our code, so we gain more precision than is available from anything hand-drawn (whether on pencil and paper or by dragging a cursor). We can also create high quality images in a vector format like PDF or SVG, meaning we can zoom without the image quality degrading.

It's true that you might achieve the same goals using languages like R or *Tikz*. Indeed, for probability trees, I still use *Tikz*. Otherwise, I offer no apology for choosing Python. In this chapter, I include a few common applications that arise in statistics.

¹Shoutout to the late Alfonso Gómez-Lobo.



14.1 Probability Diagrams

Below, we produce Venn diagrams to consider set operations for two sets, A and B . These sets are outlined as circles, either by plotting the outline or by using the Circle artist object. The result of the set operation will be filled with gray. This can be tricky because I insist that the circle outlines (the Artist object's edge) are always visible, meaning we have to be mindful of how one circle overlapping another could obscure the other's outline.

First, we plot A and B , filling only A . A and B are Circle objects and I plot B second so that it is in front of A . This means that the outline of B is not lost underneath A . We want the opposite from the fill color of B —we do not want it to overlap A . To prevent this, B is filled with `facecolor = (1, 1, 1, 0)`, which is the RGBA color for white (RGB 1,1,1) and an alpha or opacity of 0%, leaving it transparent. We could also have used an alpha parameter, but this would apply to both the `facecolor` and `edgecolor` arguments, which we want to treat separately.

```

1 fig, ax = plt.figure(), plt.axes()
2 ax.set_aspect(1)
3 ax.set_xlim(-1.6,1.6)
4 ax.set_ylim(-1.1,1.4)
5 ax.axis('off')
6
7 # Add circles with color fill
8 left_circle = plt.Circle((-0.5, 0), 1,
9                           edgecolor = 'black',
10                          facecolor = 'gray',
11                          linewidth = 2)
12 right_circle = plt.Circle((0.5, 0), 1,
13                           edgecolor = 'black',

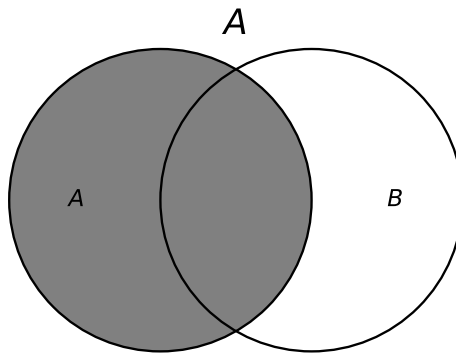
```

```

14                                     facecolor = (1, 1, 1, 0), #
    transparent
15                                     linewidth = 2,
16                                     zorder = 5)
17 ax.add_artist(left_circle)
18 ax.add_artist(right_circle)
19
20 # Labels
21 ax.text(0, 1.05,
22         s = r"$A$",
23         va = 'bottom',
24         ha = 'center',
25         size = 30)
26 ax.text(-1, 0,
27         s = r'$A$',
28         ha = 'right',
29         va = 'center',
30         size = 20)
31 ax.text(1, 0,
32         s = r'$B$',
33         ha = 'left',
34         va = 'center',
35         size = 20)

```

event-A.py



Next, we consider the union of A and B . This presents a similar challenge in not having the fill of one circle cover the outline of the other circle. First, we create two Circle objects, filled gray and with a black outline. If we stopped there, we would have a visible overlap. To recover the outline of A , we add a third circle with a black outline but a transparent fill.

```

1 fig, ax = plt.figure(), plt.axes()

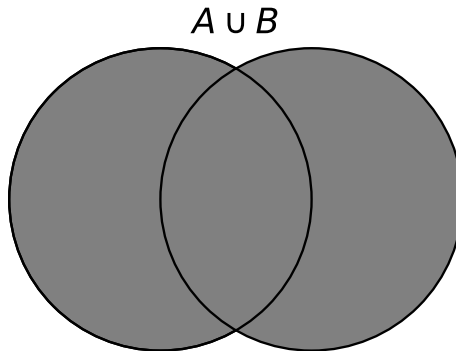
```

```

2 ax.set_aspect(1)
3 ax.set_xlim(-1.6,1.6)
4 ax.set_ylim(-1.1,1.4)
5 ax.axis('off')
6
7 # Add circles with color fill
8 left_circle = plt.Circle((-0.5, 0), 1,
9                           edgecolor = 'black',
10                          facecolor = 'gray',
11                          linewidth = 2)
12 left_circle_helper = plt.Circle((-0.5, 0), 1,
13                                 edgecolor = 'black',
14                                 facecolor = (1, 1, 1, 0),
15                                 linewidth = 2)
16 right_circle = plt.Circle((0.5, 0), 1,
17                            edgecolor = 'black',
18                            facecolor = 'gray',
19                            linewidth = 2)
20 ax.add_artist(left_circle)
21 ax.add_artist(right_circle)
22 ax.add_artist(left_circle_helper)
23
24 # Label
25 ax.text(0, 1.05,
26         s = r"$A \cup B$",
27         va = 'bottom',
28         ha = 'center',
29         size = 30)

```

union.py



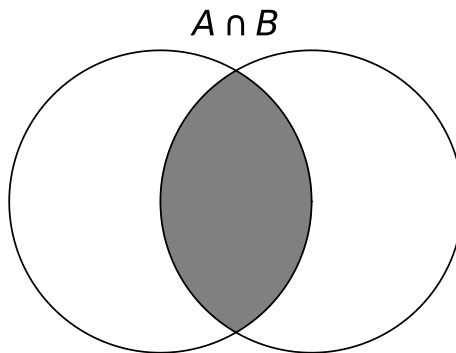
To plot the intersection of A and B , we abandon `Circle` objects for the `plot()` and `fill_between()` Axes methods. Filling the intersection is done in two pieces.


```

1 fig, ax = plt.figure(), plt.axes()
2 ax.set_aspect(1)
3 ax.set_xlim(-1.6,1.6)
4 ax.set_ylim(-1.1,1.4)
5 ax.axis('off')
6
7 # Add Circles
8 k = 10_000
9 angles = np.linspace(0, 2*np.pi, k)
10 x = np.cos(angles)
11 y = np.sin(angles)
12 ax.plot(x - 0.5, y, color = 'black')
13 ax.plot(x + 0.5, y, color = 'black')
14
15 # fill circles in two pieces
16 x = np.cos(angles) - 0.5
17 t = 0
18 while x[t] > 0:
19     t += 1
20 ax.fill_between(x[:t], -y[0:t], y[0:t],
21                color = 'gray', zorder = -1)
22 ax.fill_between(-x[:t], -y[0:t], y[0:t],
23                color = 'gray', zorder = -1)
24
25 # Label
26 ax.text(0, 1.05,
27         s = r"$A \cap B$",
28         va = 'bottom',
29         ha = 'center',
30         size = 30)

```

intersection.py



Finally, we plot $A \cap B^C$ or $A \setminus B$. For this, we again need a third transparent circle. We allow a white-filled B to cover $A \cap B$,

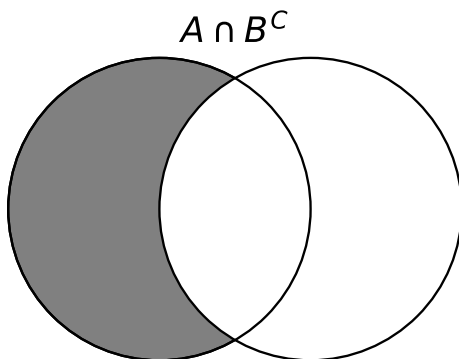
essentially removing the fill from that region. Then, we use the `Circle` `left_circle_helper` to restore the outline of A .

```

1 fig, ax = plt.figure(edgecolor = 'black'), plt.axes()
2 ax.set_aspect(1)
3 ax.set_xlim(-1.6,1.6)
4 ax.set_ylim(-1.1,1.4)
5 ax.axis('off')
6
7 # Add circles with color fill
8 left_circle = plt.Circle((-0.5, 0), 1,
9                           edgecolor = 'black',
10                          facecolor = 'gray',
11                          linewidth = 2)
12 left_circle_helper = plt.Circle((-0.5, 0), 1,
13                                 edgecolor = 'black',
14                                 facecolor = (1,1,1,0),
15                                 linewidth = 2)
16 right_circle = plt.Circle((0.5, 0), 1,
17                            edgecolor = 'black',
18                            facecolor = 'white',
19                            linewidth = 2)
20 ax.add_artist(left_circle)
21 ax.add_artist(right_circle)
22 ax.add_artist(left_circle_helper)
23
24 # Label
25 ax.text(0, 1.05,
26         s = r"$A \cap B^C$",
27         va = 'bottom',
28         ha = 'center',
29         size = 30)

```

A-minus-B.py

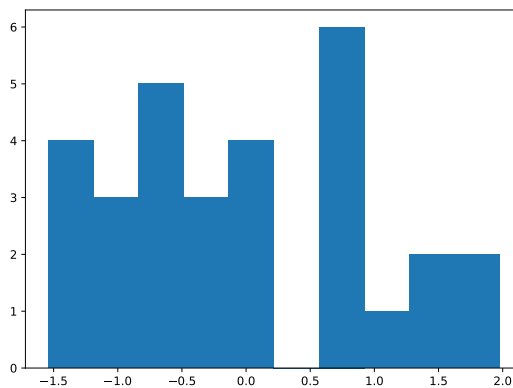


14.2 Distributions

We use the `hist()` Axes method to create a histogram. Beautifying the plot is only a matter of modifying the background elements, like the spines and labels, and then choosing the right number of bins and colors for the histogram. A histogram includes Patch objects, meaning we can specify both an `edgecolor` and `facecolor`. Below, we create a histogram using the defaults.

```
1 data = np.random.normal(size = 30)
2 fig, ax = plt.figure(), plt.axes()
3 ax.hist(data)
```

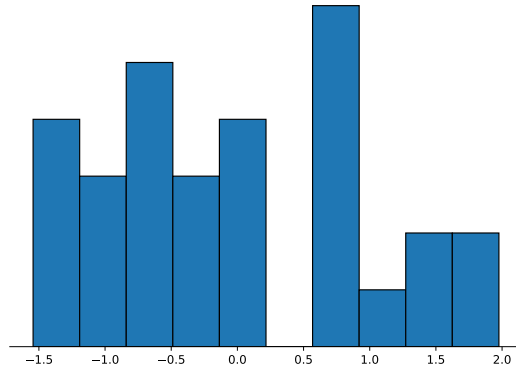
[default-hist.py](#)



When you are only interested in the shape of the data, you will find the y -axis information unimportant. Accordingly, we'll remove the y -axis to (repeat after me) reduce clutter.

```
1 #data = np.random.normal(size = 30)
2 fig, ax = plt.figure(), plt.axes()
3 ax.hist(data, edgecolor = 'black')
4 ax.set_yticks([])
5 for s in 'left', 'top', 'right':
6     ax.spines[s].set_visible(False)
```

[clean-hist.py](#)



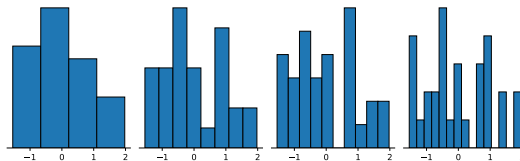
The last modification considered is the simple use of the `bins` parameter to adjust the number of bins in the histogram.

```

1 #data = np.random.normal(size = 30)
2 fig, ax = plt.subplots(1,4, figsize = (9,3))
3
4 for key, b in enumerate([4, 8, 10, 15]):
5     ax[key].hist(data, edgecolor = 'black', bins = b)
6     ax[key].set_yticks([])
7     for s in 'left', 'top', 'right':
8         ax[key].spines[s].set_visible(False)

```

[bins-hist.py](#)



Next, density plots can be produced the SciPy package and its statistics module, which has been imported already with `import scipy.stats as stats`. Below is a simple plot for a normal distribution.

```

1 x = np.linspace(-4, 4, 100_000)
2 y = stats.norm.pdf(x)
3 fig, ax = plt.figure(), plt.axes()
4 ax.set_aspect(5)
5
6 ax.plot(x,y, linewidth = 2, color = 'C0')
7 ax.fill_between(x,y, color = 'C0')

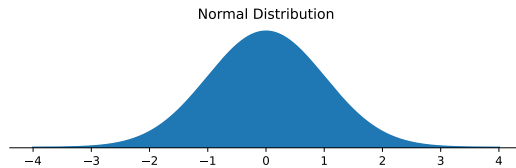
```

```

8
9 ax.set_yticks([])
10 for s in 'left', 'top', 'right':
11     ax.spines[s].set_visible(False)
12
13 ylims = ax.get_ylim()
14 ax.set_ylim(0, ylims[1])
15 ax.set_title("Normal Distribution")

```

[normal-pdf.py](#)



To motivate sampling distributions, it might be useful to label the x -axis not with numeric quantities but with μ and $\bar{x}$ for the true mean and the observed sample mean. This is done by with the `set_xticklabels()` Axes method, aligning the strings with ticks that are also manually set with `set_xticks()`.

```

1 fig, ax = plt.figure(), plt.axes()
2 x = np.linspace(-4,4, 100_000)
3 norm_pdf = stats.norm.pdf(x)
4 ax.plot(x, norm_pdf)
5 ax.set_title("Sampling Distribution")
6
7 # Clean background elements
8 ax.set_yticks([])
9 for s in 'left', 'top', 'right':
10     ax.spines[s].set_visible(False)
11
12 # Start y-axis at 0
13 ylims = ax.get_ylim()
14 ax.set_ylim(0, ylims[1])
15
16 # Set x-axis ticks and labels
17 ax.set_xticks([-1, 0,1])

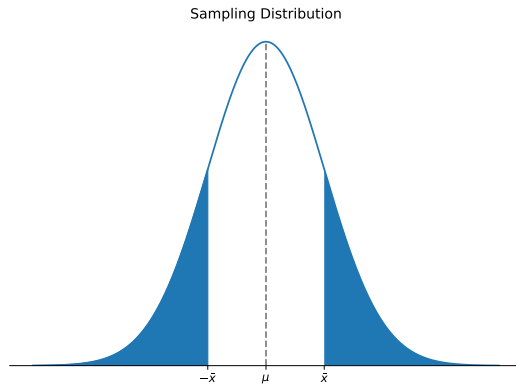
```

```

18 ax.set_xticklabels([r"$-\bar{x}$", r'$\mu$', r'$\bar{x}$',
19                    ''])
20 # Vertical dashed line
21 x0 = 0,0
22 y0 = [0,stats.norm.pdf(0)]
23 ax.plot(x0, y0, linestyle = 'dashed', color = 'gray')
24
25 # Fill area for a more extreme sample mean
26 x_right = x[x > 1]
27 y_right = norm_pdf[-len(x_right):]
28 ax.fill_between(x_right, y_right)
29
30 x_left = x[x < -1]
31 y_left = norm_pdf[:len(x_left)]
32 ax.fill_between(x_left, y_left, color = 'C0')

```

sampling-dist.py



Next, we conduct and illustrate a one-tailed z -test for a proportion,

$$H_0 : p = \frac{1}{2}$$

$$H_a : p > \frac{1}{2}.$$

We set $\alpha = 0.05$ and suppose we observed 18 successes from 30 trials. The test statistic and p -value are calculated with the help of the statsmodels library and the `proportions_ztest()` function.

```

1 # One-sided test

```

```

2 from statsmodels.stats.proportion import
   proportions_ztest
3 z_oneside, pval_oneside = proportions_ztest(
4     count = 18,
5     nobs = 30,
6     value = .5,
7     prop_var = 0.5,
8     alternative = 'larger')
9
10 # one tail norm
11 x = np.linspace(-4,4, 100_000)
12 norm_pdf = stats.norm.pdf(x)
13
14 # Make empty figure
15 fig, axx = plt.subplots(2, 1,
16     figsize = (10,5),
17     sharey = True)
18
19 for ax in axx:
20     # add normal curve
21     ax.plot(x, norm_pdf, lw = 3)
22
23     # add z stat
24     ax.axvline(z_oneside, linestyle = 'dashed')
25     ax.text(z_oneside, 0.3, ' $z$ ', ha = 'left', size =
26         12, color = 'C0')
27
28     # Clean plot
29     for s in 'left', 'right', 'top':
30         ax.spines[s].set_visible(False)
31     ax.yaxis.set_ticks([])
32     ax.set_ylim(0,.42)
33     ax.set_xlim(-3.1, 3.1)
34
35 # draw rejection region
36 ax = axx[0]
37 alpha = .05
38 critical_value = stats.norm.ppf(1 - alpha) # for one-
   sided
39 rejection_x = x[x>critical_value]
40 ax.fill_between(rejection_x, norm_pdf[-len(rejection_x)
41     :],
42     color = 'red',
43     alpha = 0.2)
44 ax.text(3.1, .14,
45     s = 'Rejection Region',
46     fontsize = 19,
47     color = (.6, .2, .2),
48     ha = 'right')
49
50 # shade p-value
51 ax = axx[1]
52 more_extreme_x = x[x>z_oneside]

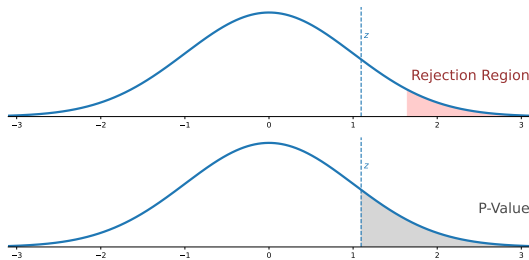
```

```

51 ax.fill_between(more_extreme_x, norm_pdf[-len(
    more_extreme_x):],
52                 color = (.2, .2, .2),
53                 alpha = 0.2)
54 ax.text(3.1, .13,
55         s = 'P-Value',
56         fontsize = 19,
57         color = (.3, .3, .3),
58         ha = 'right')

```

one-tail-norm.py



In the above, the two plots share the same x -axis scaling and I'd prefer the vertical line for the test statistic extend from one graph to the other, emphasizing that the two distributions are identical. We again face this challenge when using stacked plots to illustrate Type I and Type II error, and it is in that context that we solve the problem. This can be solved by using a Transform object. The x -axis coordinates are naturally expressed in data coordinates. Because we want the vertical line extending from one subplot Axes to the other, we should work in Figure coordinates. This requires just two special lines in the code below. The blended transformation is created with the `blended_transform_factory()` function and then we use the `transform` parameter in the Axes plotting method.

```

1 x = np.linspace(-3,5, 100_000)
2
3 fig = plt.figure(figsize = (10,5))
4 ax0 = fig.add_subplot(211, zorder = 99)
5 ax1 = fig.add_subplot(212)
6
7 for center, ax in zip([0, 2], [ax0, ax1]):
8     # add normal curve
9     norm_pdf = stats.norm.pdf(x, center, 1)
10    ax.plot(x, norm_pdf, lw = 3)
11
12    # Clean plot
13    for s in 'left', 'right', 'top':

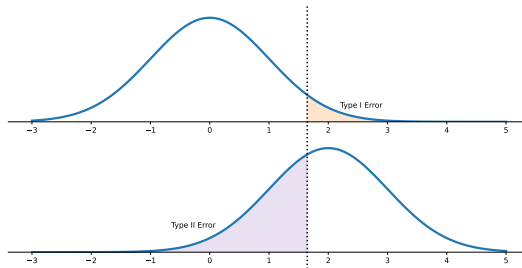
```



```

14         ax.spines[s].set_visible(False)
15         ax.yaxis.set_ticks([])
16         ax.set_ylim(0, .42)
17
18 # draw rejection region
19 ax = ax0
20 alpha = 0.05
21 critical_value = stats.norm.ppf(1 - alpha) # for one-
           sided
22 rejection_x = x[x > critical_value]
23 ax.fill_between(rejection_x,
24                 stats.norm.pdf(x, 0, 1)[-len(rejection_x
           ):],
25                 color = 'C1',
26                 alpha = 0.2)
27 # label
28 ax.text(2.2, stats.norm.pdf(2), 'Type I Error')
29
30 # Draw critical value line through both plots
31 trans = mpl.transforms.blended_transform_factory(ax.
           transData, fig.transFigure)
32 ax.plot([critical_value, critical_value],
33         [0, 1],
34         color='black',
35         linestyle = 'dotted',
36         lw=2,
37         transform = trans,
38         clip_on= False,
39         zorder = 99)
40
41 # Draw type II error region
42 fail_to_reject_region = x[x <= critical_value]
43 ax1.fill_between(fail_to_reject_region,
44                 stats.norm.pdf(x, 2, 1)[0:len(
           fail_to_reject_region)],
45                 color = 'C4',
46                 alpha = 0.2)
47 ax1.text(.1, stats.norm.pdf(.3, 2, 1), 'Type II Error',
           ha = 'right')

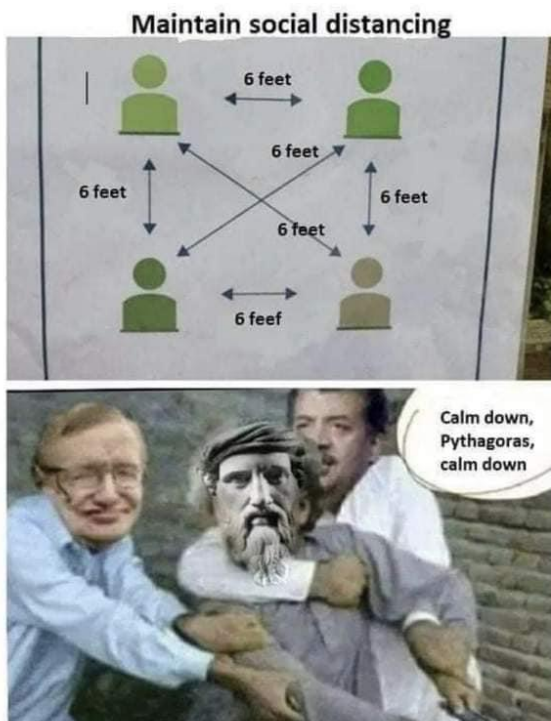
```

[error-stack.py](#)

Chapter 15

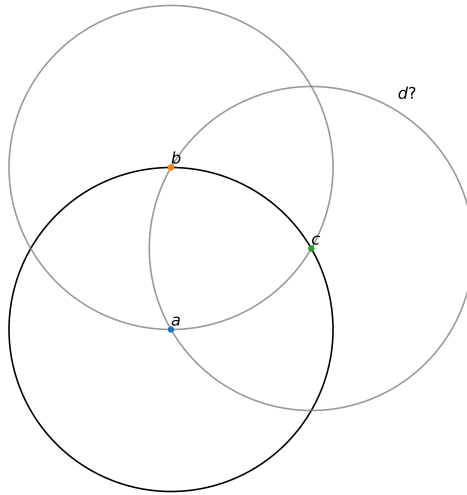
Multi-dimensional Scaling

Multi-dimensional scaling is a kind of dimensionality reduction, allowing one to translate a set of items and their predetermined pairwise distances into an arrangement of points in Cartesian space, typically in two dimensions. This is not a trivial task. Consider the meme below, which references social distancing guidance from the coronavirus pandemic. We have four individuals, forming six possible pairs, and any two individuals must maintain a six-foot distance. The graphic shows impossible right triangles that violate the Pythagorean Theorem when the distances are all exactly six feet. Indeed, there is no way to arrange four individuals in two-dimensional space so that they are all exactly six feet apart. This highlights one difficulty in multi-dimensional scaling—we must accept some error in our output.

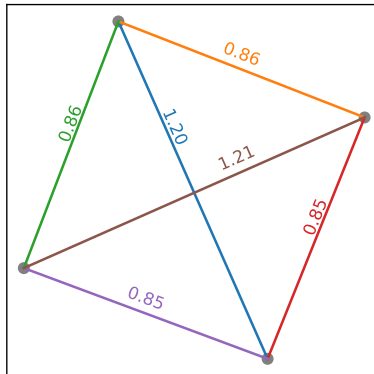


The following diagram helps show why no alternative arrangement could be perfect. Each point a, b, c is in the middle of a circle of six-foot radius. For b to be six feet away from a , b must lie on the circle centered at a . Similarly, for b to be six feet away from c , b must also lie on the circle centered at c . For three points, this arrangement is possible. But if we introduce d , d must lie on all three of the circles centered at a , b , and c . But there is no point where all three circles intersect.¹

¹The code is (link TK).



Suppose we have four points, with unit pairwise distances for any two distinct points. Applying multi-dimensional scaling creates what is roughly a square.



TKTK

Bibliography

- Borg, I. (2018). *Applied multidimensional scaling and unfolding*. Springer.
- Harper, M. (2020). Python-ternary: Ternary plots in python. *Zenodo* 10.5281/zenodo.594435. <https://doi.org/10.5281/zenodo.594435>
- Knafllic, C. N. (2015). *Storytelling with data: A data visualization guide for business professionals*. John Wiley & Sons.
- Orwell, G. (2013). *Politics and the english language*. Penguin UK.
- Schwabish, J. (2014). An economist's guide to visualizing data. *Journal of Economic Perspectives*, 28(1), 209–34.
- Schwabish, J. (2021). *Better data visualizations: A guide for scholars, researchers, and wonks*. Columbia University Press.
- Scruton, R. (2015). Poetry and truth. In J. Gibson (Ed.), *The philosophy of poetry*. Oxford University Press.
- Turrell, A., & contributors. (2021). *Coding for economists*. Online. <https://aeturrell.github.io/coding-for-economists>
- VanderPlas, J. (2016). *Python data science handbook: Essential tools for working with data*. " O'Reilly Media, Inc."

About the Author

His name is Alexander. He is a data scientist at LinkedIn and is an adjunct assistant professor at Columbia University. He holds a Ph.D. in Economics from the University of Wisconsin–Madison and is an alumnus of the Insight Health Data Science Fellows Program.

